Logic

Course notes

Sasha Sutton

Contents

Preface and Context **5**

I Propositional logic **7**

1 Syntax and semantics **8**

1.1 Syntax . . . 8

1.2 Semantics . . . 9

1.3 Extending the syntax . . . 10

1.4 Satisfiability and validity . . . 11

1.5 Logical consequences and equivalences . . . 11

2 Transformations of formulas **13**

2.1 Propositional substitutions . . . 13

2.2 Classical equivalences . . . 14

2.3 Normal forms . . . 15

2.3.1 Negation normal form . . . 15

2.3.2 Conjunctive normal form . . . 15

2.3.3 Disjunctive normal form . . . 16

3 Satisfiability and model search **17**

3.1 Model search by enumeration . . . 17

3.2 Model search by simplification . . . 18

3.3 The algorithm of Davis, Putnam, Logemann and Loveland . . . 19

3.4 The compactness theorem . . . 20

4 Validity and sequent calculus **22**

4.1 Propositional sequent calculus . . . 22

4.2 Proof search . . . 24

4.2.1 Finite branches . . . 24

4.2.2 Invertibility . . . 24
4.3 Completeness . . . 25
4.4 Other sequent calculi . . . 25

II First-order logic **26**

5 First-order logic: syntax and semantics **27**
5.1 Signatures . . . 27
5.2 Syntax of formulas . . . 28
5.3 Free and bound variables . . . 28
5.4 Interpretation of a signature . . . 28
5.5 Valuations and semantics of formulas . . . 29
5.6 Models, satisfiability and validity . . . 29
5.7 Queries on a database . . . 30
5.8 Logical equivalence . . . 31

6 Transformation of first-order formulas **32**
6.1 Substitutions . . . 32
6.2 Renaming . . . 33
6.3 Equivalences to know . . . 34
6.4 Normal forms . . . 34
6.4.1 Negation normal form . . . 34
6.4.2 Prenex normal form . . . 34

7 Theories **36**
7.1 Logical theories . . . 36
7.1.1 Theories of structures . . . 37
7.1.2 Axiomatic theories . . . 37
7.1.3 Consistency, completeness and decidability . . . 37
7.2 Quantifier elimination and decidability . . . 38
7.2.1 Dense unbounded linear orders . . . 39
7.2.2 Rational linear arithmetic . . . 39
7.2.3 Gödel's incompleteness theorems . . . 40

8 Satisfiability modulo theories **41**
8.1 Modulo-theory solvers . . . 41
8.2 Modelling examples . . . 42

8.2.1 The McNugget number . . . 42
8.2.2 Program invariant synthesis . . . 42

9 First-order sequent calculus 44
9.1 Soundness . . . 45
9.2 Completeness . . . 46
9.3 Admissible rules . . . 46

III Exercise sheets with solutions 47

Exercise sheet 1: Propositional logic 48

Exercise sheet 2: Transformation of formulas 50

Exercise sheet 3: Model search 53

Exercise sheet 4: Sequent calculus 56

Exercise sheet 5: First-order logic 59

Exercise sheet 6: Transformation of first-order formulas 62

Exercise sheet 7: Logical theories 64

Exercise sheet 8: Modelling in first-order logic 67

Exercise sheet 9: First-order sequent calculus 69

10 Links with constraint programming 72
10.1 A dictionary . . . 72
10.2 From propositional logic to constraints . . . 72
10.3 From first-order logic to constraint models . . . 73
10.4 Proof-theoretic point of view . . . 74
10.5 Soundness, completeness, termination . . . 74

A Basic mathematical concepts 75
A.1 Sets . . . 75
A.2 Functions . . . 75
A.3 Relations . . . 76
A.4 Graphs . . . 76

B Induction **77**

B.1 The example of the natural numbers 77

B.2 Inductive definitions 77

B.3 Inductively defined functions 78

B.4 Proofs by induction 78

C Greek alphabet **79**

Preface and Context

This course aims at reasoning about the logical formalisms at the heart of mathematics and computer science. Formal deduction systems are studied, both to help understand the notion of proof and to discuss the possible automation of proofs. The topics are:

- syntax and semantics of propositional logic, normal forms;
- SAT modelling and model search with the DPLL algorithm, SAT solvers;
- sequent calculus for propositional logic: soundness and completeness;
- syntax and semantics of first-order logic;
- logical theories: axiomatic theories, dense orders and arithmetic;
- satisfiability modulo theories, use of SMT solvers;
- sequent calculus for first-order logic.

Skills to acquire. Use the basics of logic to validate or refute a reasoning; write proofs concisely and rigorously; use the fundamental concepts of computer science (formal languages, logic, graphs) for programming and modelling.

Context

Logic comes from the Greek *logos*, meaning at the same time "reason", "language" and "reasoning". It is the study of the formal rules that every correct argument must obey.

Ancient philosophers such as Aristotle wanted to determine whether an argument is *conclusive*. For instance "All men are mortal, Socrates is a man, hence Socrates is mortal" is conclusive, because (1) both premises are true and (2) the inference of the conclusion from the premises is valid. These two ingredients are independent: "All mice are green, Yoda is a mouse, hence Yoda is green" is a *valid* argument that is not conclusive (a premise is false), while "All humans are mortal, Socrates is mortal, hence Socrates is human" is a *fallacious* argument (the inference is invalid). Logic is concerned with the inference rules that make sure an argument is valid.

Logic took off as a mathematical discipline in the 19th century (Boole, De Morgan, Frege). This led to the "foundational crisis", when paradoxes challenged the naive use of sets. Russell's paradox, popularised as the *barber paradox*: a barber shaves exactly the men who do not shave themselves; does he shave himself? Either answer gives a contradiction, so such a barber does not exist. In set-theoretic form: if $y = \{x \mid x \notin x\}$, then $y \in y \Leftrightarrow y \notin y$. The 20th century then saw several attempts to formalise logic and mathematics, whose limits were revealed by Gödel, Turing and Church.

In this course we take a more computer-science-oriented point of view. Logical formulas are very simple objects—*trees*—that can be manipulated by programs, which makes it possible to solve automatically a huge number of problems:

- **Logic circuits.** Circuits in processors are hardware realisations of propositional formulas, so reasoning on formulas lets us determine their functionality, decide whether two circuits are equivalent, minimise them, etc.
- **Combinatorial problems.** Many hard problems reduce to SAT, the satisfiability problem of propositional logic, which is NP-complete. In practice highly optimised SAT solvers (MiniSat, Glucose, Sat4j) can solve them, e.g. Sudoku grids.
- **Databases.** Codd's theorem relates relational algebra to relational calculus (first-order logic); an SQL query is a first-order formula evaluated on the structure associated with the database, and optimising queries amounts to finding equivalent formulas.
- **Program verification.** Specifications are written as logical formulas, e.g. "whenever a client makes a request at time t, the server answers at some later time t'": $\forall t.\ \mathit{Request}(t) \Rightarrow \exists t'.\ t \leq t' \wedge \mathit{Access}(t')$. Another approach annotates programs with pre- and post-conditions (Hoare logic) to prove, for example, that a loop computes $r = n!$.

There are many logics (modal, temporal, description, higher-order, intuitionistic, linear, separation, hybrid, infinitary, ...). This course studies the two classical logics at the basis of all the others: *propositional logic* and *first-order logic*. We use software tools (MiniSat for propositional logic, Z3 for first-order logic): the difficulty is not to design an algorithm but to *model* a problem in logic.

Part I

Propositional logic

Chapter 1

Syntax and semantics

Summary

For a set P_0 of propositions, propositional formulas are defined by $\varphi, \psi ::= p \mid \neg\varphi \mid \varphi \vee \psi \mid \varphi \wedge \psi$ with $p \in P_0$. We write $\varphi \Rightarrow \psi$ for $\neg\varphi \vee \psi$, $\varphi \Leftrightarrow \psi$ for $(\varphi \Rightarrow \psi) \wedge (\psi \Rightarrow \varphi)$, and $\top, \bot$ for $p \vee \neg p$, $p \wedge \neg p$. A *literal* is p or $\neg p$.
A formula is *satisfiable* if some interpretation (assigning 0=false or 1=true to propositions) satisfies it; *valid* if every interpretation does. $\mathrm{Sat}(\varphi)$ is the set of interpretations satisfying φ. The symbol $\models$ is used in several ways: $I \models \varphi$; $\models \varphi$ (φ valid); $S \models \varphi$ (φ is a logical consequence of S). Two formulas are equivalent, $\varphi \equiv \psi$, if $\varphi \models \psi$ and $\psi \models \varphi$, i.e. if $\psi \Leftrightarrow \varphi$ is valid.

Propositional logic studies how truth propagates through elementary logical connectives. The basic elements are *atomic propositions* (simply "propositions"): assertions that may be true or false and whose truth value cannot be reduced to that of smaller propositions. For instance "sulphuric acid boils at 337 °C" and "the platypus is omnivorous" are atomic; their truth is a matter for physics and zoology. But if the physicist says the first is true and the zoologist says the second is false, logic alone tells us that "sulphuric acid boils at 337 °C *and* the platypus is omnivorous" is false. More generally, "p and q" is true iff p and q are both true. Propositions are therefore represented by letters $p, q, r, \ldots$ and formulas are built with the connectives $\neg$ (not), $\wedge$ (and), $\vee$ (or).

1.1 Syntax

Definition 1.1. Let P_0 be a set of proposition symbols. The set of *propositional formulas* is the smallest set containing P_0 and such that if φ, ψ are formulas then so are $\neg\varphi$, $\varphi \vee \psi$ and $\varphi \wedge \psi$. Inductively,

$$\varphi, \psi ::= p \mid \neg\varphi \mid \varphi \vee \psi \mid \varphi \wedge \psi \qquad (p \in P_0).$$

$\neg$ is *negation*, $\vee$ *disjunction*, $\wedge$ *conjunction*.

A formula can thus be seen as a finite *tree*: leaves (base case of the induction) are labelled by propositions, internal nodes are labelled by $\neg$ (one child) or by $\vee, \wedge$ (two children). E.g. the tree with root $\vee$, left child p, right child $\wedge$ whose children are $\neg q$ and r is the formula $p \vee (\neg q \wedge r)$. Drawing trees is tedious, so we use a linear notation with parentheses, $(p \vee ((\neg q) \wedge r))$, and we drop parentheses using the priorities: $\neg$ binds tighter than $\wedge$, which binds tighter than $\vee$. (We will see that $\vee, \wedge$ are associative, allowing further omissions.) So the formula can be written $p \vee \neg q \wedge r$, although $p \vee (\neg q \wedge r)$ is more readable.

Example 1.2. p, $p \wedge (q \vee r)$ and $p \vee q$ are formulas; $\neg(\vee q)$ and $f(x) \vee g(x)$ are not: $\vee$ needs two children and $f(x)$ is not a proposition symbol.

Trees lend themselves to inductive definitions of functions. The set $\mathrm{prop}(\varphi) \subseteq P_0$ of propositions occurring in φ is

$$\mathrm{prop}(p) \stackrel{\text{def}}{=} \{p\}, \quad \mathrm{prop}(\neg\varphi) \stackrel{\text{def}}{=} \mathrm{prop}(\varphi), \quad \mathrm{prop}(\varphi \vee \psi) = \mathrm{prop}(\varphi \wedge \psi) \stackrel{\text{def}}{=} \mathrm{prop}(\varphi) \cup \mathrm{prop}(\psi).$$

For $\varphi = p \wedge (q \vee p)$: $\mathrm{prop}(\varphi) = \{p\} \cup (\{q\} \cup \{p\}) = \{p, q\}$. The set of *subformulas* is

$$\mathrm{SF}(p) \stackrel{\text{def}}{=} \{p\}, \quad \mathrm{SF}(\neg\varphi) \stackrel{\text{def}}{=} \{\neg\varphi\} \cup \mathrm{SF}(\varphi), \quad \mathrm{SF}(\varphi \star \psi) \stackrel{\text{def}}{=} \{\varphi \star \psi\} \cup \mathrm{SF}(\varphi) \cup \mathrm{SF}(\psi) \ (\star \in \{\vee, \wedge\}).$$

Thus $\mathrm{SF}(p \vee (\neg q \wedge r)) = \{p, q, r, \neg q, \neg q \wedge r, p \vee (\neg q \wedge r)\}$. A subformula is simply a subtree.

Definition 1.3. A *literal* is a formula of the form p or $\neg p$ with p a proposition.

For instance the literals occurring in $q \vee (p \wedge \neg q)$ are $q, p, \neg q$.

1.2 Semantics

Semantics is the study of the interpretation of formulas. First we give a truth value to atomic propositions: an *interpretation* is a map $I : P_0 \to \{0, 1\}$, where 0 means "false" and 1 "true" (Boolean values, after George Boole).

Definition 1.4. For an interpretation I, the value $[\![\varphi]\!]_I$ of a formula is defined inductively by

$$[\![p]\!]_I \stackrel{\text{def}}{=} I(p) \qquad [\![\neg\varphi]\!]_I \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if } [\![\varphi]\!]_I = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$[\![\varphi \vee \psi]\!]_I \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if } [\![\varphi]\!]_I = 1 \text{ or } [\![\psi]\!]_I = 1 \\ 0 & \text{otherwise} \end{cases} \qquad [\![\varphi \wedge \psi]\!]_I \stackrel{\text{def}}{=} \begin{cases} 1 & \text{if } [\![\varphi]\!]_I = 1 \text{ and } [\![\psi]\!]_I = 1 \\ 0 & \text{otherwise.} \end{cases}$$

This corresponds to the usual truth tables:

p	$\neg p$
0	1
1	0

p	q	$p \wedge q$
0	0	0
0	1	0
1	0	0
1	1	1

p	q	$p \vee q$
0	0	0
0	1	1
1	0	1
1	1	1

For $I(p) = 1$, $I(q) = I(r) = 0$ we get $[\![p \vee \neg q \wedge r]\!]_I = 1$. All values over all interpretations can be gathered in one truth table:

p	q	r	$\neg q$	$\neg q \wedge r$	$p \vee \neg q \wedge r$
0	0	0	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	1	0	0	1

Just as the value of an arithmetic expression depends only on the variables occurring in it, the value of a formula depends only on its propositions:

Property 1.5. *For every formula φ and interpretations I, I', if $I(p) = I'(p)$ for all $p \in \mathrm{prop}(\varphi)$ then $[\![\varphi]\!]_I = [\![\varphi]\!]_{I'}$.*

Proof. By induction on φ, assuming $I(p) = I'(p)$ for all $p \in \mathrm{prop}(\varphi)$.

- $\varphi = p$: then $p \in \mathrm{prop}(\varphi)$ so $I(p) = I'(p)$, hence $[\![p]\!]_I = I(p) = I'(p) = [\![p]\!]_{I'}$.
- $\varphi = \neg\psi$: $\mathrm{prop}(\varphi) = \mathrm{prop}(\psi)$, so by induction $[\![\psi]\!]_I = [\![\psi]\!]_{I'}$ and therefore $[\![\neg\psi]\!]_I = [\![\neg\psi]\!]_{I'}$ by definition of negation.
- $\varphi = \psi \vee \psi'$: $\mathrm{prop}(\varphi) = \mathrm{prop}(\psi) \cup \mathrm{prop}(\psi')$, so I and I' agree on both, and by induction $[\![\psi]\!]_I = [\![\psi]\!]_{I'}$, $[\![\psi']\!]_I = [\![\psi']\!]_{I'}$; the definition of $\vee$ then gives equal values.
- $\varphi = \psi \wedge \psi'$ is similar. □

We may therefore use *partial* interpretations, provided their domain $\mathrm{dom}(I)$ contains the propositions of the formula. For distinct $p_1, \ldots, p_n$ and $b_1, \ldots, b_n \in \{0, 1\}$ we write $[b_1 \leftarrow p_1, \ldots, b_n \leftarrow p_n]$ for the partial interpretation with domain $\{p_1, \ldots, p_n\}$ sending p_j to b_j. E.g. $I = [1 \leftarrow p, 0 \leftarrow q, 0 \leftarrow r]$ gives $[\![p \vee \neg q \wedge r]\!]_I = 1$.

1.3 Extending the syntax

Other Boolean operations could be used: there are $2^{2^2} = 16$ Boolean functions of two arguments. Some have names: exclusive or $\oplus$ (xor), implication $\Rightarrow$, equivalence $\Leftrightarrow$, "not and" $\uparrow$ (nand):

p	q	$p \oplus q$	$p \Rightarrow q$	$p \Leftrightarrow q$	$p \uparrow q$
0	0	0	1	1	1
0	1	1	1	0	1
1	0	1	0	0	1
1	1	0	1	1	0

All these are expressible with $\neg, \wedge, \vee$: for all φ, ψ and I,

$$[\![\varphi \oplus \psi]\!]_I = [\![(\varphi \vee \psi) \wedge \neg(\varphi \wedge \psi)]\!]_I \qquad [\![\varphi \Rightarrow \psi]\!]_I = [\![\neg\varphi \vee \psi]\!]_I$$
$$[\![\varphi \Leftrightarrow \psi]\!]_I = [\![(\neg\varphi \vee \psi) \wedge (\varphi \vee \neg\psi)]\!]_I \qquad [\![\varphi \uparrow \psi]\!]_I = [\![\neg(\varphi \wedge \psi)]\!]_I.$$

On implication. Note the particular definition of $\Rightarrow$: "if $1+1=1$ then the capital of France is Marseille" is true, since $\varphi \Rightarrow \psi$ is true whenever φ is false. In everyday language "if the weather is nice I go fishing" is often read as a necessary condition ("I only go fishing if the weather is nice"), but logically it only expresses a *sufficient* condition. If the premise is false, the whole implication is true: this is the principle *ex falso sequitur quodlibet* (from falsehood anything follows). On the other hand, one cannot prove something false from a true hypothesis.

In fact *every* Boolean function, of any arity, can be written with $\neg, \wedge, \vee$:

Theorem 1.6 (Functional completeness)**.** *For every $n > 0$ and every Boolean function $f : \{0,1\}^n \to \{0,1\}$ there is a formula φ on propositions $p_1, \ldots, p_n$ such that $[\![\varphi]\!]_I = f(I(p_1), \ldots, I(p_n))$ for all I.*

Proof. By induction on n. For $n = 1$ there are four functions: identity (p_1), negation ($\neg p_1$), constant 1 ($p_1 \vee \neg p_1$), constant 0 ($p_1 \wedge \neg p_1$).

For $n > 1$, assume the result for $n-1$ and let $f : \{0,1\}^n \to \{0,1\}$. Define $g_0(b_1, \ldots, b_{n-1}) = f(b_1, \ldots, b_{n-1}, 0)$ and $g_1(b_1, \ldots, b_{n-1}) = f(b_1, \ldots, b_{n-1}, 1)$. By induction there are formulas φ_0, φ_1 encoding g_0, g_1. Put $\varphi = (\neg p_n \wedge \varphi_0) \vee (p_n \wedge \varphi_1)$. Fix I.

- If $I(p_n) = 1$: $[\![\neg p_n \wedge \varphi_0]\!]_I = 0$, so $[\![\varphi]\!]_I = [\![\varphi_1]\!]_I = g_1(I(p_1), \ldots, I(p_{n-1})) = f(I(p_1), \ldots, I(p_n))$.
- If $I(p_n) = 0$: symmetric, using g_0. □

Applied to $p \Leftrightarrow q$ the proof yields $(p \wedge q) \vee (\neg p \wedge \neg q)$.

We also add the constants $\top$ ("true") and $\bot$ ("false"), denoting the constant functions 1 and 0; in the non-extended syntax they are $p \vee \neg p$ and $p \wedge \neg p$.

1.4 Satisfiability and validity

Definition 1.7. *I satisfies φ* (or I is a *model* of φ), written $I \models \varphi$, if $[\![\varphi]\!]_I = 1$. Inductively: $I \models p$ iff $I(p) = 1$; $I \models \neg\varphi$ iff not $I \models \varphi$; $I \models \varphi \vee \psi$ iff $I \models \varphi$ or $I \models \psi$; $I \models \varphi \wedge \psi$ iff both. We write $I \not\models \varphi$ if $I \models \varphi$ fails, and $\mathrm{Sat}(\varphi)$ for the set of interpretations satisfying φ.

Example 1.8. For $\varphi = (p \vee q) \wedge (p \vee \neg r)$, $\mathrm{Sat}(\varphi)$ (over p, q, r) consists of $(0,1,0)$, $(1,0,0)$, $(1,0,1)$, $(1,1,0)$, $(1,1,1)$.

Definition 1.9 (Satisfiability)**.** φ is *satisfiable* if it has a model ($\mathrm{Sat}(\varphi) \neq \emptyset$), *unsatisfiable* otherwise.

Definition 1.10 (Validity)**.** φ is *valid* (a *tautology*), written $\models \varphi$, if every interpretation is a model of it.

Property 1.11 (Excluded middle)**.** *For every φ, $\varphi \vee \neg\varphi$ is valid.*

Proof. Let I be an interpretation. If $[\![\varphi]\!]_I = 1$ then $[\![\varphi \vee \neg\varphi]\!]_I = 1$; if $[\![\varphi]\!]_I = 0$ then $[\![\neg\varphi]\!]_I = 1$ and again $[\![\varphi \vee \neg\varphi]\!]_I = 1$. □

Definition 1.12. For a set S of formulas, $I \models S$ if $I \models \varphi$ for all $\varphi \in S$. S is *satisfiable* if some I satisfies S, unsatisfiable otherwise.

Example 1.13. Let $S = \{p \vee q \vee \neg r,\ q \vee r,\ \neg p \vee \neg q \vee r,\ \neg p \vee \neg r,\ p \vee \neg q\}$. Let I be an interpretation. If $I(p) = 1$: if $I(r) = 1$ then $I \not\models \neg p \vee \neg r$; else if $I(q) = 1$ then $I \not\models \neg p \vee \neg q \vee r$; else $I \not\models q \vee r$. If $I(p) = 0$: if $I(q) = 1$ then $I \not\models p \vee \neg q$; else if $I(r) = 1$ then $I \not\models p \vee q \vee \neg r$; else $I \not\models q \vee r$. In all cases $I \not\models S$, so S is unsatisfiable.

Satisfiability and validity are dual:

Property 1.14. *φ is satisfiable iff $\neg\varphi$ is not valid; φ is valid iff $\neg\varphi$ is not satisfiable.*

Proof. First statement. If φ is satisfiable, take $I \models \varphi$; then $I \not\models \neg\varphi$, so $\neg\varphi$ is not valid. Conversely if $\neg\varphi$ is not valid, some I has $I \not\models \neg\varphi$, i.e. $I \models \varphi$. The second statement is similar. □

1.5 Logical consequences and equivalences

Definition 1.15. φ is a *logical consequence* of ψ, written $\psi \models \varphi$, if every model of ψ is a model of φ ($\mathrm{Sat}(\psi) \subseteq \mathrm{Sat}(\varphi)$). For a set S: $S \models \psi$ if every I with $I \models S$ satisfies $I \models \psi$.

Beware of the notation: $\models$ is used (i) as $I \models \varphi$ / $I \models S$; (ii) as $\models \varphi$ for validity; (iii) as $S \models \varphi$ for consequence ($\{\psi\} \models \varphi$ is written $\psi \models \varphi$).

If $S = \emptyset$: every interpretation satisfies all formulas of $\emptyset$, so $\emptyset \models \varphi$ says exactly that φ is valid.

Property 1.16. *φ is valid iff $\emptyset \models \varphi$.*

Property 1.17 (Deduction)**.** *$S \cup \{\varphi\} \models \psi$ iff $S \models \varphi \Rightarrow \psi$. In particular $\varphi \models \psi$ iff $\varphi \Rightarrow \psi$ is valid.*

Proof. ($\Rightarrow$) Assume $S \cup \{\varphi\} \models \psi$ and let $I \models S$. If $I \not\models \varphi$ then $I \models \varphi \Rightarrow \psi$. Otherwise $I \models S \cup \{\varphi\}$ so $I \models \psi$, hence again $I \models \varphi \Rightarrow \psi$. ($\Leftarrow$) Assume $S \models \varphi \Rightarrow \psi$ and let $I \models S \cup \{\varphi\}$. Then $I \models \varphi \Rightarrow \psi$ and $I \models \varphi$, so $I \models \psi$. □

Definition 1.18. φ and ψ are *logically equivalent*, $\varphi \equiv \psi$, if $\varphi \models \psi$ and $\psi \models \varphi$, i.e. $\mathrm{Sat}(\varphi) = \mathrm{Sat}(\psi)$.

By the deduction property, $\varphi \equiv \psi$ iff $\varphi \Leftrightarrow \psi$ is valid, which can be tested with truth tables.

Example 1.19. $p \equiv (p \vee \neg q) \wedge p$:

p	q	$\neg q$	$p \vee \neg q$	$(p \vee \neg q) \wedge p$
0	0	1	1	0
0	1	0	0	0
1	0	1	1	1
1	1	0	1	1

The columns of p and $(p \vee \neg q) \wedge p$ coincide.

This approach must test all interpretations of $\mathrm{prop}(\varphi) \cup \mathrm{prop}(\psi)$, an exponential number in the number of propositions. Later techniques explore only a small part of this search space.

Link with constraint programming

A model of a formula is a *solution* of a constraint problem: constraints are formulas, variables are the decision variables, and "satisfiable" means "the CSP has a solution". See Chapter 10.

Chapter 2

Transformations of formulas

Summary

A *propositional substitution* is a function τ (with finite domain) mapping each proposition p to a formula $\tau(p)$; $[\varphi_1 \leftarrow p_1, \ldots, \varphi_n \leftarrow p_n]$ sends p_i to φ_i. If I is an interpretation, I_τ is the interpretation mapping p to $[\![\tau(p)]\!]_I$. The *substitution lemma* states $[\![\tau(\varphi)]\!]_I = [\![\varphi]\!]_{I_\tau}$. A formula is in *negation normal form* (NNF) if negations only appear in front of propositions; it is in *conjunctive normal form* (CNF) if it is a conjunction of disjunctions of literals, $\bigwedge_i \bigvee_j \ell_{i,j}$. The *clausal form* is the set-of-sets notation of a CNF.

In this chapter we study transformations of formulas to ensure that we only handle formulas with certain features, which simplifies algorithms. Another goal is a first technique to show equivalence of formulas: get equivalences for small formulas, then apply them to larger ones by replacing propositions by more complex formulas. This requires *substitutions*.

2.1 Propositional substitutions

As in arithmetic: by commutativity of $+$, $2 \times (3 + x) = 2 \times (x + 3)$ for any x; formally, start from $2 \times y$, note $3 + x$ and $x + 3$ are equal and substitute them for y. Similarly, since $p \vee \neg p$ is valid for every proposition p, $\varphi \vee \neg\varphi$ is valid for every formula φ.

Definition 2.1. A *propositional substitution* is a function τ associating to each $p \in P_0$ a formula $\tau(p)$ such that $\mathrm{dom}(\tau) = \{p \mid \tau(p) \neq p\}$ is finite. We write $[\varphi_1 \leftarrow p_1, \ldots, \varphi_n \leftarrow p_n]$ for the substitution of domain $\{p_1, \ldots, p_n\}$ (distinct p_i) sending p_i to φ_i.

It extends to all formulas by $\tau(\neg\varphi) \stackrel{\text{def}}{=} \neg\tau(\varphi)$, $\tau(\varphi \vee \psi) \stackrel{\text{def}}{=} \tau(\varphi) \vee \tau(\psi)$, $\tau(\varphi \wedge \psi) \stackrel{\text{def}}{=} \tau(\varphi) \wedge \tau(\psi)$.

Example 2.2. For $\varphi = p \wedge q$ and $\tau = [p \vee \neg q \leftarrow p,\ p \leftarrow q]$ we get $\tau(\varphi) = (p \vee \neg q) \wedge p$.

Definition 2.3. For an interpretation I and a substitution τ, I_τ is the interpretation sending each p to $[\![\tau(p)]\!]_I$.

Example 2.4. If $I(p) = I(q) = 0$ and $\tau = [p \vee \neg q \leftarrow p, p \leftarrow q]$, then $I_\tau(p) = [\![p \vee \neg q]\!]_I = 1$ and $I_\tau(q) = [\![p]\!]_I = 0$.

Lemma 2.5 (Propositional substitution lemma)**.** *For every formula φ, substitution τ and interpretation I: $[\![\tau(\varphi)]\!]_I = [\![\varphi]\!]_{I_\tau}$.*

Proof. Induction on φ. Base: $[\![\tau(p)]\!]_I = I_\tau(p) = [\![p]\!]_{I_\tau}$ by definition. Negation: $[\![\tau(\neg\psi)]\!]_I = [\![\neg\tau(\psi)]\!]_I$; by induction $[\![\tau(\psi)]\!]_I = [\![\psi]\!]_{I_\tau}$, hence by the semantics of $\neg$ this equals $[\![\neg\psi]\!]_{I_\tau}$. Disjunction: $[\![\tau(\psi\vee\psi')]\!]_I = [\![\tau(\psi)\vee\tau(\psi')]\!]_I$, which is 1 iff $[\![\tau(\psi)]\!]_I = 1$ or $[\![\tau(\psi')]\!]_I = 1$, iff (induction) $[\![\psi]\!]_{I_\tau} = 1$ or $[\![\psi']\!]_{I_\tau} = 1$, iff $[\![\psi \vee \psi']\!]_{I_\tau} = 1$. Conjunction is similar. □

Example 2.6. With the previous I, τ, φ: $[\![\tau(\varphi)]\!]_I = [\![(p \vee \neg q) \wedge p]\!]_{[0\leftarrow p, 0\leftarrow q]} = 0$ and $[\![\varphi]\!]_{I_\tau} = [\![p \wedge q]\!]_{[1\leftarrow p, 0\leftarrow q]} = 0$.

Corollary 2.7. *If φ is valid and τ is a substitution, then $\tau(\varphi)$ is valid.*

Proof. For any I, $[\![\tau(\varphi)]\!]_I = [\![\varphi]\!]_{I_\tau} = 1$ since φ is valid. □

2.2 Classical equivalences

By a truth table, $\neg(p\vee q) \Leftrightarrow \neg p\wedge\neg q$ is valid; applying $[\varphi \leftarrow p, \psi \leftarrow q]$ we get $\neg(\varphi\vee\psi) \equiv \neg\varphi\wedge\neg\psi$ for all formulas φ, ψ. In the same way:

Theorem 2.8. *For all formulas $\varphi, \psi, \psi', \psi_1, \psi_2$:*

$\varphi \wedge \psi \equiv \psi \wedge \varphi$	$\varphi \vee \psi \equiv \psi \vee \varphi$	*(commutativity)*
$\varphi \wedge (\psi_1 \wedge \psi_2) \equiv (\varphi \wedge \psi_1) \wedge \psi_2$	$\varphi \vee (\psi_1 \vee \psi_2) \equiv (\varphi \vee \psi_1) \vee \psi_2$	*(associativity)*
$\top \wedge \varphi \equiv \varphi \wedge \top \equiv \varphi$	$\bot \vee \varphi \equiv \varphi \vee \bot \equiv \varphi$	*(neutral elements)*
$\varphi \wedge \varphi \equiv \varphi$	$\varphi \vee \varphi \equiv \varphi$	*(idempotence)*
$\varphi \wedge (\varphi \vee \psi) \equiv \varphi$	$\varphi \vee (\varphi \wedge \psi) \equiv \varphi$	*(absorption)*
$\varphi \wedge \bot \equiv \bot \wedge \varphi \equiv \bot$	$\varphi \vee \top \equiv \top \vee \varphi \equiv \top$	*(absorbing element)*
$\varphi \wedge (\psi \vee \psi') \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \psi')$	$\varphi \vee (\psi \wedge \psi') \equiv (\varphi \vee \psi) \wedge (\varphi \vee \psi')$	*(distributivity)*
$\varphi \wedge \neg\varphi \equiv \neg\varphi \wedge \varphi \equiv \bot$	$\varphi \vee \neg\varphi \equiv \neg\varphi \vee \varphi \equiv \top$	*(complement)*
$\neg\neg\varphi \equiv \varphi$		*(involution)*
$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi$	$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$	*(De Morgan's laws)*
$\varphi \Rightarrow \psi \equiv \neg\psi \Rightarrow \neg\varphi$		*(contraposition)*
$\varphi \Rightarrow (\psi \Rightarrow \psi') \equiv (\varphi \wedge \psi) \Rightarrow \psi'$		*(currying)*

These equivalences let us simplify formulas, thanks to:

Property 2.9. *If $\tau(p) \equiv \tau'(p)$ for every proposition p, then $\tau(\varphi) \equiv \tau'(\varphi)$ for every formula φ.*

Proof. Let I be an interpretation. For every p, $I_\tau(p) = [\![\tau(p)]\!]_I = [\![\tau'(p)]\!]_I = I_{\tau'}(p)$, so $I_\tau = I_{\tau'}$. By the substitution lemma, $[\![\tau(\varphi)]\!]_I = [\![\varphi]\!]_{I_\tau} = [\![\varphi]\!]_{I_{\tau'}} = [\![\tau'(\varphi)]\!]_I$. □

Consequently, one may replace a subformula ψ of φ by an equivalent ψ': let φ' be φ in which ψ is replaced by a fresh proposition p, so that $\varphi = [\psi \leftarrow p](\varphi')$; with $\tau = [\psi \leftarrow p]$ and $\tau' = [\psi' \leftarrow p]$ we get $\varphi = \tau(\varphi') \equiv \tau'(\varphi')$, which is φ with ψ replaced by ψ'.

Example 2.10. We show $(p \Rightarrow q) \Rightarrow p \equiv p$:

$$\begin{aligned}
(p \Rightarrow q) \Rightarrow p &\equiv \neg(\neg p \vee q) \vee p \\
&\equiv (\neg\neg p \wedge \neg q) \vee p && \text{(De Morgan)} \\
&\equiv (p \wedge \neg q) \vee p && \text{(involution)} \\
&\equiv (p \vee p) \wedge (\neg q \vee p) && \text{(distributivity)} \\
&\equiv p \wedge (\neg q \vee p) && \text{(idempotence)} \\
&\equiv p \wedge (p \vee \neg q) && \text{(commutativity)}
\end{aligned}$$

and the last formula is equivalent to p by the earlier example (or absorption).

2.3 Normal forms

There are many equivalent ways to write a formula; some are easier to handle (e.g. p is easier to evaluate than $(p \vee \neg q) \wedge p$). Search algorithms work on formulas with restricted syntax: a *normal form* is an equivalent formula better suited to algorithmic treatment.

By associativity and commutativity, all formulas built from $\varphi_1, \ldots, \varphi_n$ with $\wedge$ alone are equivalent, so we write $\bigwedge_{i=1}^{n} \varphi_i \stackrel{\text{def}}{=} \varphi_1 \wedge \cdots \wedge \varphi_n$ (any bracketing) and likewise $\bigvee_{i=1}^{n} \varphi_i$.

2.3.1 Negation normal form

Definition 2.11. A formula is in *negation normal form* (NNF) if the only negations occur in front of propositions: $\varphi, \psi ::= p \mid \neg p \mid \varphi \vee \psi \mid \varphi \wedge \psi$.

E.g. $(p \vee \neg q) \wedge p$ is in NNF but $\neg(p \vee \neg q)$ is not. We "push" negations toward the leaves:

$$\begin{aligned}
\mathrm{NNF}(p) &\stackrel{\text{def}}{=} p & \mathrm{NNF}(\neg p) &\stackrel{\text{def}}{=} \neg p \\
\mathrm{NNF}(\varphi \vee \psi) &\stackrel{\text{def}}{=} \mathrm{NNF}(\varphi) \vee \mathrm{NNF}(\psi) & \mathrm{NNF}(\neg(\varphi \vee \psi)) &\stackrel{\text{def}}{=} \mathrm{NNF}(\neg\varphi) \wedge \mathrm{NNF}(\neg\psi) \\
\mathrm{NNF}(\varphi \wedge \psi) &\stackrel{\text{def}}{=} \mathrm{NNF}(\varphi) \wedge \mathrm{NNF}(\psi) & \mathrm{NNF}(\neg(\varphi \wedge \psi)) &\stackrel{\text{def}}{=} \mathrm{NNF}(\neg\varphi) \vee \mathrm{NNF}(\neg\psi) \\
& & \mathrm{NNF}(\neg\neg\varphi) &\stackrel{\text{def}}{=} \mathrm{NNF}(\varphi)
\end{aligned}$$

This is linear time in the size of the formula. By De Morgan and involution:

Property 2.12. $\varphi \equiv \mathrm{NNF}(\varphi)$ *for every* φ.

Example 2.13 (Peirce's law)**.** $((p \Rightarrow q) \Rightarrow p) \Rightarrow p$ is valid. In the non-extended syntax it is $\neg(\neg(\neg p \vee q) \vee p) \vee p$, and

$$\mathrm{NNF} = ((\neg p \vee q) \wedge \neg p) \vee p.$$

2.3.2 Conjunctive normal form

Definition 2.14. A *clause* is a disjunction of literals. A formula is in *conjunctive normal form* (CNF) if it is a conjunction of clauses: $\bigwedge_{i=1}^{m} \bigvee_{j=1}^{n_i} \ell_{i,j}$ with $\ell_{i,j}$ literals.

From a formula in NNF we obtain a CNF by applying distributivity:

$$\begin{aligned}
\mathrm{CNF}(\psi \wedge \psi') &\stackrel{\text{def}}{=} \mathrm{CNF}(\psi) \wedge \mathrm{CNF}(\psi') \\
\mathrm{CNF}(\varphi \vee (\psi \wedge \psi')) &\stackrel{\text{def}}{=} \mathrm{CNF}(\varphi \vee \psi) \wedge \mathrm{CNF}(\varphi \vee \psi') \\
\mathrm{CNF}((\psi \wedge \psi') \vee \varphi) &\stackrel{\text{def}}{=} \mathrm{CNF}(\psi \vee \varphi) \wedge \mathrm{CNF}(\psi' \vee \varphi)
\end{aligned}$$

The formula φ is duplicated, so the CNF can be exponentially larger and take exponential time to compute. It is still equivalent:

Property 2.15. $\varphi \equiv \mathrm{CNF}(\varphi)$.

Example 2.16. $\varphi = \neg(p \wedge (q \Rightarrow (r \vee s))) \wedge (p \vee q)$.

- Eliminate $\Rightarrow$: $\varphi \equiv \neg(p \wedge (\neg q \vee (r \vee s))) \wedge (p \vee q)$.
- NNF: $(\neg p \vee (q \wedge \neg r \wedge \neg s)) \wedge (p \vee q)$.
- CNF: $(\neg p \vee q) \wedge (\neg p \vee \neg r) \wedge (\neg p \vee \neg s) \wedge (p \vee q)$.

By idempotence, commutativity and associativity, a clause is determined by its *set* of literals ($p \vee q \vee \neg r$ is $\{p, q, \neg r\}$), and a CNF by its *set of clauses*.

Definition 2.17. The *clausal form* $\mathrm{Cl}(\varphi)$ is the set of clauses of $\mathrm{CNF}(\varphi)$, each clause seen as a set of literals.

Example 2.18. For $\varphi = (p \vee \neg q) \wedge p$, $\mathrm{Cl}(\varphi) = \{\{p, \neg q\}, \{p\}\}$, which is also the clausal form of $p \wedge (\neg q \vee p)$. Formulas with the same clausal form are equivalent.

The clausal form can be simplified by equivalence-preserving rules:

Excluded middle: delete clauses containing two opposite literals (always satisfied), e.g. $p \vee q \vee \neg r \vee \neg q$.

Factorisation: delete repeated literals in a clause.

Subsumption: if the literals of a clause C all appear in C', delete C' (any interpretation satisfying C satisfies C'). E.g. $C = p \vee q \vee r$ subsumes $C' = p \vee \neg s \vee t \vee q \vee r$.

Further remark. CNF conversion has exponential worst-case complexity but gives an equivalent formula. There is a linear-time method (Tseitin-style) that yields only an *equisatisfiable* formula (φ satisfiable iff the result is); this suffices to test satisfiability or validity and is much faster in practice.

2.3.3 Disjunctive normal form

Using distributivity the other way, from an NNF we get a *disjunctive normal form* by pushing conjunctions to the leaves:

$$\mathrm{DNF}(\varphi \wedge (\psi \vee \psi')) \stackrel{\text{def}}{=} \mathrm{DNF}(\varphi \wedge \psi) \vee \mathrm{DNF}(\varphi \wedge \psi'), \qquad \mathrm{DNF}((\psi \vee \psi') \wedge \varphi) \stackrel{\text{def}}{=} \mathrm{DNF}(\psi \wedge \varphi) \vee \mathrm{DNF}(\psi' \wedge \varphi).$$

The result has the form $\bigvee_{i=1}^{m} \bigwedge_{j=1}^{n_i} \ell_{i,j}$. Satisfiability of a DNF is decided in linear time: a conjunction of literals is satisfiable iff it contains no pair $p, \neg p$. Like CNF, DNF conversion is exponential in the worst case; moreover, unless P=NP, no polynomial algorithm can produce a DNF (even an equisatisfiable one), since it would solve SAT in polynomial time.

Chapter 3

Satisfiability and model search

Summary

The DPLL algorithm decides the satisfiability of a formula in clausal form by exhaustively searching for models. It applies the following rewriting rules on sets of clauses:

$$
\begin{aligned}
S \cup \{\{\ell\}\} &\to_{\text{dpll}} S[\top \leftarrow \ell] && && \text{(unit)}\\
S &\to_{\text{dpll}} S[\top \leftarrow \ell] && \text{where } \ell \in \text{Pur}(S) && \text{(pure)}\\
S &\to_{\text{dpll}} S[\top \leftarrow p] && \text{where } p \in \text{prop}(S) && \text{(split}_p\text{)}\\
S &\to_{\text{dpll}} S[\top \leftarrow \neg p] && \text{where } p \in \text{prop}(S) && \text{(split}_{\neg p}\text{)}
\end{aligned}
$$

The algorithm often has choices, and when a choice does not lead to a model it backtracks, yielding a search tree. It gives priority to (unit) and (pure) because they are *invertible* (no alternative needs to be explored). The system *terminates*, is *sound* (if $S \to^* \emptyset$ then S is satisfiable) and *complete* (if S is satisfiable then $S \to^* \emptyset$).

The *satisfiability problem* (SAT) is: **input** a propositional formula φ; **question** is φ satisfiable? *SAT solvers* are programs solving instances of it; if the instance is satisfiable they also return a partial interpretation I with $I \models \varphi$. They usually take a formula in clausal form, so in this chapter all formulas are in clausal form; this forces us to produce directly this form when modelling a problem.

We present the basics of SAT solvers by successive improvements of the brute-force algorithm that builds the whole truth table. The complexity is exponential in the worst case, and every current SAT solver is too: unless P = NP this cannot be avoided, since SAT is the archetypal NP-complete problem.

3.1 Model search by enumeration

SAT can be solved by enumerating the $2^{|\text{prop}(\varphi)|}$ partial interpretations with domain $\text{prop}(\varphi)$. Rather than writing the truth table we use a tree: for each proposition p we branch on $I(p) = 0$ (label $\neg p$) and $I(p) = 1$ (label p).

Example 3.1. Let $\varphi = \{\{p, q, \neg r\}, \{q, r\}, \{\neg p, \neg q, r\}, \{\neg p, \neg r\}, \{p, \neg q\}\}$. In the enumeration tree every leaf (a total interpretation on p, q, r) falsifies at least one clause—for instance the leftmost branch $\neg p, \neg q, \neg r$ falsifies $\{q, r\}$. Since no interpretation satisfies all clauses, φ is unsatisfiable.

Definition 3.2. For a partial interpretation I and a proposition p, $I[1 \leftarrow p]$ (resp. $I[0 \leftarrow p]$) is the partial interpretation mapping p to 1 (resp. 0) and every other $q \in \mathrm{dom}(I)$ to $I(q)$.

Assuming a function `evaluate`, the algorithm is

```
satisfiable(φ, I):
  if dom(I) = prop(φ):  return evaluate(φ, I)
  else:  choose p in prop(φ) \ dom(I);
      return satisfiable(φ, I[1←p]) or satisfiable(φ, I[0←p])
```

called initially with the empty partial interpretation.

3.2 Model search by simplification

The enumeration waits to have all propositions interpreted before testing clauses, which is inefficient. We can answer as soon as a branch is satisfied or falsified; evaluating all clauses at all nodes would be costly, so instead we *partially evaluate the clauses themselves* while going down a branch. We simplify the set S of clauses until we get either the empty clause $\bot$ (then S is unsatisfiable on this branch) or the empty set of clauses (then S is satisfiable).

For a literal ℓ, $\bar{\ell}$ is the opposite literal ($\overline{p} = \neg p$, $\overline{\neg p} = p$).

Definition 3.3. The *simplification* of a set S of clauses by a literal ℓ is

$$S[\top \leftarrow \ell] \stackrel{\text{def}}{=} \{C \setminus \{\bar{\ell}\} \mid C \in S,\ \ell \notin C\}.$$

We remove clauses containing ℓ (now satisfied) and remove the literal $\bar{\ell}$ from the other clauses (it is now false and cannot help satisfy them).

For example $\{\{p, q\}, \{\neg p, r\}, \{p, \neg p\}\}[\top \leftarrow p] = \{\{r\}\}$ and $[\top \leftarrow \neg p]$ gives $\{\{q\}\}$. A clause becoming empty means that none of its literals was chosen true: it is falsified by the partial interpretation. Simplification is sound because it amounts to substituting $\top$ for ℓ and $\bot$ for $\bar{\ell}$ and simplifying with $\varphi \vee \top \equiv \top$, $\varphi \vee \bot \equiv \varphi$, $\varphi \wedge \top \equiv \varphi$.

Definition 3.4. For a partial interpretation I and literal ℓ, $I[1 \leftarrow \ell]$ denotes $I[1 \leftarrow p]$ if $\ell = p$ and $I[0 \leftarrow p]$ if $\ell = \neg p$.

Example 3.5. For $S = \{\{p, \neg r, \neg q\}, \{q, \neg r\}, \{\neg q\}\}$ (satisfiable): branching on p, then q, then r gives two branches ending in the empty set of clauses, i.e. models $[0 \leftarrow p, 0 \leftarrow q, 0 \leftarrow r]$ and $[1 \leftarrow p, 0 \leftarrow q, 0 \leftarrow r]$. Branches containing $\bot$ are abandoned since $\bot$ remains forever.

Recursive algorithm:

```
satisfiable(S):
  if S = ∅:  return 1
  if ⊥ ∈ S: return 0
  choose p in prop(S)
  return satisfiable(S[⊤ ←p]) or satisfiable(S[⊤ ← ¬p])
```

Equivalently, by rewriting rules $S \to_{\text{simpl}} S[\top \leftarrow p]$ (split_p) and $S \to_{\text{simpl}} S[\top \leftarrow \neg p]$ ($\text{split}_{\neg p}$) for $p \in \mathrm{prop}(S)$. We write $S \to^*_{\text{simpl}} S'$ for a (possibly empty) sequence of rewrites. We show:

- *termination*: no infinite sequence of rewrites exists;
- *soundness*: if $S \to^*_{\text{simpl}} \emptyset$ then S is satisfiable;
- *completeness*: if S is satisfiable then $S \to^*_{\text{simpl}} \emptyset$.

Property 3.6. *For every set of clauses S, interpretation I and literal ℓ: $I \models S[\top \leftarrow \ell]$ iff $I[1 \leftarrow \ell] \models S$. (Consequence of the substitution lemma.)*

Property 3.7. *For S a set of clauses and p a proposition, $p \notin \text{prop}(S[\top \leftarrow p])$ and $p \notin \text{prop}(S[\top \leftarrow \neg p])$.*

Theorem 3.8. $\to_{\text{simpl}}$ *terminates, is sound and complete.*

Proof. Termination. If $S \to_{\text{simpl}} S'$ then $S' = S[\top \leftarrow p]$ or $S[\top \leftarrow \neg p]$ with $p \in \text{prop}(S)$; by Property 3.7, $\text{prop}(S') \subsetneq \text{prop}(S)$. So at most $\#\text{prop}(S)$ rewrites can be applied.

Soundness. The empty set of clauses is satisfiable. If $S \to S'$ and $I \models S'$ then S is satisfiable: for (split_p), Property 3.6 gives $I[1 \leftarrow p] \models S$; for ($\text{split}_{\neg p}$) it gives $I[0 \leftarrow p] \models S$. Conclude by induction on the length of the rewrite sequence.

Completeness. Let $I \models S$ and order $\text{prop}(S)$ arbitrarily as $p_1 < \cdots < p_n$. Put $S_0 = S$ and for $i = 1..n$: if $p_i \notin \text{prop}(S_{i-1})$ then $S_i = S_{i-1}$; if $I \models p_i$ then $S_i = S_{i-1}[\top \leftarrow p_i]$, and since $I = I[1 \leftarrow p_i]$, Property 3.6 gives $I \models S_i$; if $I \models \neg p_i$ then $S_i = S_{i-1}[\top \leftarrow \neg p_i]$ and again $I \models S_i$. By Property 3.7, $\text{prop}(S_i) \subseteq \{p_{i+1}, \ldots, p_n\}$, so S_n has no proposition; it cannot contain $\bot$ because $I \models S_n$. Hence $S_n = \emptyset$. □

3.3 The algorithm of Davis, Putnam, Logemann and Loveland

DPLL is a refinement of the simplification search, the basis of all current SAT solvers. It chooses cleverly the order of simplifications using two heuristics.

Definition 3.9. In a set S of clauses, a clause is *unit* if it contains a single literal; this literal is also called unit. Formally ℓ is unit in S if $S = S' \cup \{\{\ell\}\}$.

In any model of S, a unit literal must be satisfied; so simplifying by it is a good choice, and the other branch (necessarily unsatisfiable) can be ignored.

Definition 3.10. A literal ℓ is *pure* in S if $\bar{\ell}$ appears in no clause of S. $\text{Pur}(S)$ is the set of pure literals of S.

If $I \models S$ and ℓ is pure then $I[1 \leftarrow \ell] \models S$ still; simplifying by ℓ is again a good choice (the ignored branch may be satisfiable too, unlike for unit literals).

DPLL is $\to_{\text{simpl}}$ extended with the rules (unit) and (pure) of the summary.

Theorem 3.11. $\to_{\text{dpll}}$ *terminates and is sound and complete.*

Property 3.12 (Invertibility)**.** *If S is satisfiable and S' is obtained from S by (unit) or (pure), then S' is satisfiable.*

Proof. Let $I \models S$. (unit): $S = S_1 \cup \{\{\ell\}\}$, $S' = S_1[\top \leftarrow \ell]$; $I \models \ell$ so $I[1 \leftarrow \ell] = I \models S_1$ and Property 3.6 gives $I \models S'$. (pure): if $I \models \ell$ then $I[1 \leftarrow \ell] = I \models S$ and we conclude as before. Otherwise $I \not\models \ell$; every clause $C \in S$ contains a literal $\ell_C \neq \ell$ with $I \models \ell_C$; then $I[1 \leftarrow \ell] \models \ell_C$ for all C, so $I[1 \leftarrow \ell] \models S$, and Property 3.6 gives $I \models S'$. □

So if S' comes from S by (unit) or (pure), S is satisfiable iff S' is: there is no need for choice points with backtracking. These rules can be applied unrestrictedly, and the usual strategy applies them first, in this order, before (split):

```
satisfiable(S):
  if S = ∅:  return 1                              # all clauses satisfied
  if ⊥ ∈ S: return 0                          # empty clause:  unsatisfiable
  if S has a unit literal ℓ:  return satisfiable(S[⊤ ← ℓ])
  if S has a pure literal ℓ:  return satisfiable(S[⊤ ← ℓ])
  choose p in prop(S)
  return satisfiable(S[⊤ ←p]) or satisfiable(S[⊤ ← ¬p])
```

Example 3.13. For the unsatisfiable $\varphi = \{\{p, q, \neg r\}, \{q, r\}, \{\neg p, \neg q, r\}, \{\neg p, \neg r\}, \{p, \neg q\}\}$: split on p. Branch $\neg p$ gives $\{\{q, \neg r\}, \{q, r\}, \{\neg q\}\}$; $\neg q$ is unit and yields $\{\{\neg r\}, \{r\}\}$; r is unit, giving $\{\bot\}$. Branch p gives $\{\{q, r\}, \{\neg q, r\}, \{\neg r\}\}$; $\neg r$ is unit giving $\{\{q\}, \{\neg q\}\}$, then q unit gives $\{\bot\}$. So φ is unsatisfiable. For the satisfiable $\{\{p, \neg r, \neg q\}, \{q, \neg r\}, \{\neg q\}\}$: $\neg q$ unit gives $\{\{\neg r\}\}$, then $\neg r$ unit gives $\emptyset$: no non-invertible rule is needed.

3.4 The compactness theorem

Enumeration trees yield a nice result about infinite sets of formulas. A set S (possibly infinite) is satisfiable if one interpretation satisfies all its formulas simultaneously. Infinite sets can be tricky: imagine a logic stating infinitely many facts "some day it will rain; today it does not rain; tomorrow it won't; in 2 days it won't; ...". This set is unsatisfiable, but every finite subset is satisfiable (make it rain the day after the largest day mentioned). Compactness says this cannot happen in propositional logic:

Theorem 3.14 (Compactness)**.** *A set S of propositional formulas is satisfiable iff every finite subset of S is satisfiable.*

Proof. ($\Rightarrow$) is trivial. For ($\Leftarrow$) we prove the contrapositive: assume S unsatisfiable and exhibit an unsatisfiable finite subset. We assume $P_0 = \{p_0, p_1, \dots\}$ is countable (the general case uses topology, e.g. Tychonoff's theorem).

Consider the infinite binary tree where each node at depth n has two children, setting p_n to 0 or to 1. A node at depth n is a *failure node* if some $\varphi \in S$ with $\mathrm{prop}(\varphi) \subseteq \{p_0, \dots, p_{n-1}\}$ is falsified by the partial interpretation along the branch.

Every branch contains a failure node: otherwise the infinite branch defines an interpretation I of all of P_0; for $\varphi \in S$ let n be the largest index with $p_n \in \mathrm{prop}(\varphi)$; if $I \not\models \varphi$ then the node at depth $n+1$ would be a failure node. So $I \models S$, contradicting unsatisfiability.

Cut every branch at its failure node of minimal depth, labelling that leaf with a falsified formula. The resulting binary tree has only finite branches, hence is finite by König's lemma. The set E of formulas labelling the leaves is finite, and every interpretation corresponds to exactly one leaf, hence falsifies a formula of E. So $E \subseteq S$ is a finite unsatisfiable subset. □

This innocuous-looking theorem gives powerful results in remote areas, e.g. plane tilings.

Corollary 3.15. *Let T be a finite set of tiles and $H, V \subseteq T^2$. A subset $E \subseteq \mathbb{Z}^2$ can be* tiled *by T if there is $f : E \to T$ such that for every $(i, j) \in E$: if $(i+1, j) \in E$ then $(f(i, j), f(i+1, j)) \in H$ (horizontal compatibility), and if $(i, j+1) \in E$ then $(f(i, j), f(i, j+1)) \in V$ (vertical compatibility). Then $\mathbb{Z}^2$ can be tiled by T iff every square $[\![-n, n]\!]^2$, $n \in \mathbb{N}$, can be tiled by T.*

Proof idea. One direction is obvious. For the converse, take propositions $x_{(i,j),t}$ ("tile t is at position (i,j)") and the set S of formulas stating that each position carries exactly one tile, and that horizontal/vertical neighbours are compatible. A tiling of $[\![-n,n]\!]^2$ satisfies the finitely many formulas of S concerning that square; any finite subset of S mentions only a bounded square, so it is satisfiable. By compactness S is satisfiable, and a model gives a tiling of $\mathbb{Z}^2$. □

Link with constraint programming

A set of clauses is a constraint satisfaction problem with Boolean variables, and DPLL is the basic constraint-solving scheme: rule (unit) is *propagation*, rules split_p/$\text{split}_{\neg p}$ are *branching*, and $\bot \in S$ is *failure*. See Chapter 10.

Chapter 4

Validity and sequent calculus

The VALIDITY problem is: **input** a propositional formula φ; **question** is φ valid?

Summary

A *sequent* is an unordered sequence (multiset) of formulas, written after the symbol $\vdash$. A sequent $\vdash \Gamma$ is *valid*, written $\models \Gamma$, if for every interpretation I at least one formula of Γ is satisfied by I. The rules of the sequent calculus are

$$\frac{}{\vdash \Gamma, p, \neg p}\,(\text{ax}) \qquad \frac{\vdash \Gamma, \varphi, \psi}{\vdash \Gamma, \varphi \vee \psi}\,(\vee) \qquad \frac{\vdash \Gamma, \varphi \qquad \vdash \Gamma, \psi}{\vdash \Gamma, \varphi \wedge \psi}\,(\wedge)$$

where all formulas are in negation normal form. A *proof* is a tree chaining these rules, whose root is the sequent to prove. When there is a choice, try (ax) first, then ($\vee$), then ($\wedge$): it gives smaller proofs. The propositional sequent calculus is sound and complete: $\vdash \Gamma$ is provable ($\vdash_{LK_0} \Gamma$) iff it is valid ($\models \Gamma$).

One way to solve validity is through duality (Property 1.14): φ is valid iff $\neg\varphi$ is unsatisfiable, so a SAT solver can be used. But when φ is valid we would like a *certificate* explaining why, which can be checked easily (think of embedded software in an aircraft: it is better to have a certificate that an independent authority can check than to simply trust the manufacturer). The search tree of a SAT solver is easy to check but potentially huge and dependent on the solver's heuristics.

The alternative explored here is a *proof system*: deduction rules that guarantee validity. The proof itself (the derivation tree) is the certificate. It is easy to check, though it may be of exponential size.

4.1 Propositional sequent calculus

We study a *one-sided, invertible, cut-free* sequent calculus. It has few rules and suits proof search, but requires formulas in the basic syntax (no $\Rightarrow$) and in NNF; it is not the system one naturally uses by hand.

A *sequent* is an unordered sequence of formulas, with repetitions, written $\vdash \Gamma$ ("turnstile"). Order is irrelevant: sequents with the same formulas with the same number of occurrences are identified. (Formally, Γ is a finite multiset: a function $\Gamma : S \to \mathbb{N}$ with finite support; a comma denotes multiset union.) The empty sequent is written $\vdash \bot$.

A sequent $\vdash \Gamma$ is *satisfied* by I, $I \models \Gamma$, if there is a *witness* formula $\theta \in \Gamma$ with $I \models \theta$; i.e. I is a

model of the disjunction of the formulas of Γ. It is *valid*, $\models \Gamma$, if all interpretations satisfy it. If $\vdash \varphi$ is valid then φ is valid. E.g. $\vdash p, \neg p \wedge r, \neg p \vee r$ is valid.

The rules are:

$$\frac{}{\vdash \Gamma, p, \neg p}\,(\text{ax}) \qquad \frac{\vdash \Gamma, \varphi, \psi}{\vdash \Gamma, \varphi \vee \psi}\,(\vee) \qquad \frac{\vdash \Gamma, \varphi \qquad \vdash \Gamma, \psi}{\vdash \Gamma, \varphi \wedge \psi}\,(\wedge)$$

Each rule deduces a *conclusion* (below the bar) from finitely many *premises* (above); here zero, one or two premises. The order of formulas is irrelevant: in $(\vee)$, φ, ψ may be any formulas of the premise sequent and Γ stands for all the other ones. A sequent $\vdash \Gamma$ is *provable*, $\vdash_{LK_0} \Gamma$, if it has a *derivation*: a tree of rules with the conclusion at the root (drawn at the bottom).

Example 4.1 (Peirce's law)**.** $((p \Rightarrow q) \Rightarrow p) \Rightarrow p$ has NNF $((\neg p \vee q) \wedge \neg p) \vee p$, with derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \neg p, q, p}\,(\text{ax})}{\vdash \neg p \vee q, p}\,(\vee) \qquad \dfrac{}{\vdash \neg p, p}\,(\text{ax})}{\vdash (\neg p \vee q) \wedge \neg p, p}\,(\wedge)}{\vdash ((\neg p \vee q) \wedge \neg p) \vee p}\,(\vee)$$

Theorem 4.2 (Soundness)**.** *If* $\vdash_{LK_0} \Gamma$ *then* $\models \Gamma$.

Proof. By induction on a derivation, showing that for each rule, valid premises give a valid conclusion. (ax): $\Gamma = \Gamma', p, \neg p$; any I satisfies p or $\neg p$. $(\vee)$: $\Gamma = \Gamma', \varphi \vee \psi$ with premise Γ', φ, ψ valid. For any I there is a witness $\theta \in \Gamma' \cup \{\varphi, \psi\}$; if $\theta \in \{\varphi, \psi\}$ then $I \models \varphi \vee \psi$, otherwise $I \models \Gamma'$. $(\wedge)$: premises Γ', φ and Γ', ψ valid. For I, there are witnesses $\theta \in \Gamma' \cup \{\varphi\}$ and $\theta' \in \Gamma' \cup \{\psi\}$. If either lies in Γ' then $I \models \Gamma'$; else $\theta = \varphi$, $\theta' = \psi$ and $I \models \varphi \wedge \psi$. □

This gives validity certificates for formulas and logical consequences.

Example 4.3. $(p \wedge p) \Leftrightarrow p$ has NNF $(\neg p \vee \neg p \vee p) \wedge (\neg p \vee (p \wedge p))$ (no simplification by idempotence is allowed: the formula is precisely about it). Derivation:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \neg p, \neg p, p}\,(\text{ax})}{\vdash \neg p, \neg p \vee p}\,(\vee)}{\vdash \neg p \vee \neg p \vee p}\,(\vee) \qquad \dfrac{\dfrac{\dfrac{}{\vdash \neg p, p}\,(\text{ax}) \qquad \dfrac{}{\vdash \neg p, p}\,(\text{ax})}{\vdash \neg p, p \wedge p}\,(\wedge)}{\vdash \neg p \vee (p \wedge p)}\,(\vee)}{\vdash (\neg p \vee \neg p \vee p) \wedge (\neg p \vee (p \wedge p))}\,(\wedge)$$

Example 4.4. $(p \wedge (q \vee r)) \Rightarrow ((p \wedge q) \vee (p \wedge r))$ has NNF $\neg p \vee (\neg q \wedge \neg r) \vee (p \wedge q) \vee (p \wedge r)$. Applying $(\vee)$ three times we reach $\vdash \neg p, \neg q \wedge \neg r, p \wedge q, p \wedge r$. Apply $(\wedge)$ on $p \wedge q$: left premise $\vdash \neg p, \neg q \wedge \neg r, p, p \wedge r$, and right premise $\vdash \neg p, \neg q \wedge \neg r, q, p \wedge r$. The left is closed by (ax) on $\neg p, p$ after splitting $p \wedge r$ (premises $\vdash \neg p, \neg q \wedge \neg r, p, p$ and $\vdash \neg p, \neg q \wedge \neg r, p, r$, both (ax)). In the right premise, split $\neg q \wedge \neg r$ and $p \wedge r$: all leaves are closed by (ax) on $\{\neg q, q\}$, $\{\neg p, p\}$, $\{\neg r, r\}$, as required.

Example 4.5. We show $p \wedge q$ is a consequence of $\varphi_a \wedge \varphi_b \wedge \varphi_c$ where $\varphi_a = p \Rightarrow q$, $\varphi_b = \neg p \Rightarrow (\neg q \wedge \neg r)$, $\varphi_c = p \vee q \vee r$. By the deduction property it suffices that $(\varphi_a \wedge \varphi_b \wedge \varphi_c) \Rightarrow p \wedge q$ is valid, i.e. that its NNF $\tilde{\varphi}_a \vee \tilde{\varphi}_b \vee \tilde{\varphi}_c \vee (p \wedge q)$ is valid, where

$$\tilde{\varphi}_a = p \wedge \neg q, \qquad \tilde{\varphi}_b = \neg p \wedge (q \vee r), \qquad \tilde{\varphi}_c = \neg p \wedge \neg q \wedge \neg r.$$

It is provable by a (large) derivation: apply $(\vee)$ to reach $\vdash \tilde{\varphi}_a, \tilde{\varphi}_b, \tilde{\varphi}_c, p \wedge q$, split $\tilde{\varphi}_b$ with $(\wedge)$ and close all branches by (ax) after further splitting.

4.2 Proof search

Proof search addresses PROVABILITY: **input** a sequent $\vdash \Gamma$; **question** is it provable? It builds a derivation from the root toward the leaves. A *proof search branch* is a (potentially infinite) sequence $\vdash \Gamma_0, \vdash \Gamma_1, \ldots$ with $\Gamma_0 = \Gamma$ and each $\vdash \Gamma_i$ a premise of a rule with conclusion $\vdash \Gamma_{i-1}$. A *failure branch* is a finite branch ending at a *failure sequent* $\vdash \Gamma_n$ to which no rule applies. Proof search is non-deterministic: several rules may apply to a sequent, e.g. on $\vdash \neg p \vee q, r \vee p$ we may pick either disjunction as *principal formula*.

Example 4.6. $(p \vee \neg q) \wedge p$ is not provable: $(\wedge)$ yields $\vdash p \vee \neg q$ and $\vdash p$; the first leads to $\vdash p, \neg q$ via $(\vee)$. Both $\vdash p, \neg q$ and $\vdash p$ are failure sequents.

Example 4.7. $(p \vee \neg q) \vee (\neg p \wedge r)$ is not provable. After $(\vee)$ we have $\vdash p \vee \neg q, \neg p \wedge r$. Choosing $p \vee \neg q$ as principal gives $\vdash p, \neg q, \neg p \wedge r$ then premises $\vdash p, \neg q, \neg p$ (ax) and $\vdash p, \neg q, r$ (failure). Choosing $\neg p \wedge r$ first gives the same failure sequent. So the formula is not provable.

4.2.1 Finite branches

The *size* of a formula in NNF is the number of $\wedge/\vee$ nodes: $|p| = |\neg p| = 0$, $|\varphi \vee \psi| = |\varphi \wedge \psi| = 1 + |\varphi| + |\psi|$. For a multiset, $|\Gamma| = \sum_{\varphi} |\varphi| \cdot \Gamma(\varphi)$.

Property 4.8. *All branches of a proof search for* $\vdash \Gamma$ *have length at most* $|\Gamma|$.

Proof. For each rule, the size of every premise is strictly smaller than the size of the conclusion. □

A naive algorithm tries each $\varphi \in \Gamma$ as principal formula in turn (depth is bounded by $|\Gamma|$, so it terminates), but it is super-exponential since it tries all orders.

4.2.2 Invertibility

A rule is *syntactically invertible* if whenever its conclusion has a derivation, each premise has one too. Then one can apply it greedily without backtracking: if premises are not provable, the conclusion is not provable, whatever other rule is tried. (ax) is trivially invertible; so are the other two:

Lemma 4.9 (Syntactic invertibility)**.** *The rules (ax), (∨), (∧) are syntactically invertible.*

Proof. For $(\vee)$: let π derive $\vdash \Gamma, \varphi \vee \psi$; we show $\vdash_{LK_0} \Gamma, \varphi, \psi$ by induction on the depth of π. If the last rule is $(\vee)$ with $\varphi \vee \psi$ principal, the premise's sub-derivation works. Otherwise $\varphi \vee \psi$ is not principal in the last rule, which is (ax) (then Γ, φ, ψ still contains $p, \neg p$, so (ax) applies), $(\vee)$ or $(\wedge)$ on another formula; apply the induction hypothesis to the premises (which contain $\varphi \vee \psi$) to get derivations of the premises with φ, ψ instead, then reapply the same last rule. For $(\wedge)$: similarly by induction on π we derive both $\vdash \Gamma, \varphi$ and $\vdash \Gamma, \psi$; if the last rule is $(\wedge)$ on $\varphi \wedge \psi$, use its two premises; otherwise apply the induction hypothesis to each premise (twice, once for φ and once for ψ) and reapply the last rule. □

When choosing between a $\vee$ and a $\wedge$ formula as principal, prefer $\vee$ (smaller search tree). The simplified algorithm:

```
provable(⊢ Γ):
  if Γ = Δ, p, ¬p:  return 1
  if Γ = Δ, φ ∨ ψ:  return provable(⊢ Δ, φ, ψ)
  if Γ = Δ, φ ∧ ψ:  return provable(⊢ Δ, φ) and provable(⊢ Δ, ψ)
  return 0
```

4.3 Completeness

Property 4.10 (Countermodel of failure sequents)**.** *If* $\vdash \Gamma$ *is a failure sequent, it has a countermodel: an interpretation* I *with* $I \not\models \Gamma$.

Proof. No rule applies. Since neither ($\vee$) nor ($\wedge$) applies, Γ contains only literals $\ell_1, \ldots, \ell_k$; since (ax) does not apply, it never contains a literal together with its negation. So there is I with $I \not\models \ell_j$ for all j. □

E.g. the failure sequent $\vdash p, \neg q, r$ has countermodel $[0 \leftarrow p, 1 \leftarrow q, 0 \leftarrow r]$.

A rule is *semantically invertible* if validity of its conclusion implies validity of each premise; contrapositively, a countermodel of a premise is a countermodel of the conclusion.

Lemma 4.11 (Semantic invertibility)**.** *The rules of the propositional sequent calculus are semantically invertible.*

Proof. (ax) has no premise. ($\vee$): if $I \not\models \Gamma, \varphi, \psi$ then $I \not\models \Gamma$, $I \not\models \varphi$, $I \not\models \psi$ so $I \not\models \varphi \vee \psi$ and $I \not\models \Gamma, \varphi \vee \psi$. ($\wedge$): if $I \not\models \Gamma, \varphi$ (the other case is symmetric) then $I \not\models \Gamma$ and $I \not\models \varphi$, so $I \not\models \varphi \wedge \psi$ and $I \not\models \Gamma, \varphi \wedge \psi$. □

Theorem 4.12 (Completeness)**.** *If* $\models \Gamma$ *then* $\vdash_{LK_0} \Gamma$.

Proof. By contraposition: assume $\vdash \Gamma$ is not provable and exhibit a countermodel. Branches of proof search are finite, so each ends in a failure sequent or an instance of (ax). If all ended in (ax) we would have a derivation. So there is a failure branch $\vdash \Gamma_0 = \vdash \Gamma, \ldots, \vdash \Gamma_n$. By the previous property there is I with $I \not\models \Gamma_n$. By decreasing induction, using the semantic invertibility lemma, $I \not\models \Gamma_i$ for all i, so $I \not\models \Gamma$ and Γ is not valid. □

4.4 Other sequent calculi

One-sided means that sequents have the form $\vdash \Gamma$. Other calculi use *two-sided* sequents $\Gamma \vdash \Delta$, meaning that the disjunction of Δ follows from the conjunction of Γ ($\Gamma \models \Delta$); this is the calculus found in most books. *Cut-free* means that no cut (proving a lemma first and then continuing) is needed to prove everything valid.

For culture, the two-sided calculus has rules (no NNF needed): (ax) $\Gamma, \varphi \vdash \varphi, \Delta$; ($\neg_l$) from $\Gamma \vdash \varphi, \Delta$ infer $\Gamma, \neg\varphi \vdash \Delta$; ($\neg_r$) from $\Gamma, \varphi \vdash \Delta$ infer $\Gamma \vdash \neg\varphi, \Delta$; ($\vee_l$) from $\Gamma, \varphi \vdash \Delta$ and $\Gamma, \psi \vdash \Delta$ infer $\Gamma, \varphi \vee \psi \vdash \Delta$; ($\vee_r$) from $\Gamma \vdash \Delta, \varphi, \psi$ infer $\Gamma \vdash \Delta, \varphi \vee \psi$; ($\wedge_l$) from $\Gamma, \varphi, \psi \vdash \Delta$ infer $\Gamma, \varphi \wedge \psi \vdash \Delta$; ($\wedge_r$) from $\Gamma \vdash \varphi, \Delta$ and $\Gamma \vdash \psi, \Delta$ infer $\Gamma \vdash \varphi \wedge \psi, \Delta$; ($\Rightarrow_l$) from $\Gamma \vdash \varphi, \Delta$ and $\Gamma, \psi \vdash \Delta$ infer $\Gamma, \varphi \Rightarrow \psi \vdash \Delta$; ($\Rightarrow_r$) from $\Gamma, \varphi \vdash \psi, \Delta$ infer $\Gamma \vdash \varphi \Rightarrow \psi, \Delta$. It is sound and complete ($\Gamma \vdash \Delta$ provable iff $\Gamma \models \Delta$), and corresponds more closely to hand reasoning: proving $(p \wedge (q \vee r)) \Rightarrow ((p \wedge q) \vee (p \wedge r))$ amounts to taking $I \models p \wedge (q \vee r)$ and reasoning by cases: if $I \models q$ then $I \models p \wedge q$, if $I \models r$ then $I \models p \wedge r$.

Part II

First-order logic

Chapter 5

First-order logic: syntax and semantics

Summary

A first-order *signature* is a pair $\mathcal{L} = (\mathcal{F}, \mathcal{P})$ of a set $\mathcal{F}$ of function symbols and a non-empty set $\mathcal{P}$ of relation symbols, each with an *arity* (number of arguments). We write $f^{(m)}$ or $R^{(m)}$ for a symbol of arity m. Formulas over $\mathcal{L}$ and a set X of variables are

$$\begin{aligned} t &::= x \mid f(t_1, \ldots, t_m) && \text{(terms)} \\ \alpha &::= R(t_1, \ldots, t_m) && \text{(atomic formulas)} \\ \varphi, \psi &::= \alpha \mid \neg\varphi \mid \varphi \vee \psi \mid \exists x.\varphi && \text{(formulas)} \end{aligned}$$

We extend with $\wedge, \Rightarrow, \top, \bot$ and define $\forall x.\varphi = \neg\exists x.\neg\varphi$. A *literal* is an atomic formula or its negation.
The semantics depends on an *interpretation* I (a non-empty domain D_I, a function $f_I : D_I^m \to D_I$ for each $f \in \mathcal{F}_m$, a relation $R_I \subseteq D_I^m$ for each $R \in \mathcal{P}_m$) and a *valuation* $\rho : X \to D_I$. A formula is *satisfiable* if some (I, ρ) satisfies it; I is a *model* of φ ($I \models \varphi$) if $I, \rho \models \varphi$ for every ρ; φ is *valid* ($\models \varphi$) if every interpretation is a model.

Propositional logic only expresses Boolean propositions and cannot describe statements whose truth depends on individuals, e.g. "if A is the father of B and C the father of A then C is a grandfather of B" or "everyone has a father". First-order logic (*predicate calculus*) adds quantifiers $\forall, \exists$ over individuals, and a richer vocabulary of function symbols (computing on individuals) and relation symbols (relating individuals).

It is similar to propositional logic (inductive definition of formulas, interpretations to evaluate them, a sound and complete proof system), but with an important algorithmic difference: **first-order logic is undecidable**: no algorithm checks whether a formula is valid. There are unboundedly many interpretations, and interpretations typically have infinite domains, so checking that an interpretation is a model of $\forall x.\varphi$ cannot always be done.

5.1 Signatures

Definition 5.1. A first-order *signature* is a pair $\mathcal{L} = (\mathcal{F}, \mathcal{P})$ of countable sets of function symbols $\mathcal{F}$ and relation (or "predicate") symbols $\mathcal{P} \neq \emptyset$, with $\mathcal{F} \cap \mathcal{P} = \emptyset$, both equipped with an arity function $a : \mathcal{F} \cup \mathcal{P} \to \mathbb{N}$. We write $\mathcal{F}_n$, $\mathcal{P}_n$ for the symbols of arity n.

A function of arity 0 is a *constant*; a relation of arity 0 is a *proposition*.

Example 5.2 (propositional signatures)**.** $\mathcal{F} = \emptyset$, $\mathcal{P} = \{P^{(0)}, Q^{(0)}\}$: first-order logic contains propositional logic.

Example 5.3 (orders)**.** $\mathcal{F} = \emptyset$, $\mathcal{P} = \{<^{(2)}, =^{(2)}\}$.

Example 5.4 (arithmetic)**.** $\mathcal{F} = \{+^{(2)}, \times^{(2)}\}$, $\mathcal{P} = \{=^{(2)}\}$.

Example 5.5 (finite directed graphs)**.** With vertices coloured blue or not: $\mathcal{F} = \emptyset$, $\mathcal{P} = \{E^{(2)}, =^{(2)}, B^{(1)}\}$ where E represents edges and B blue vertices.

5.2 Syntax of formulas

Let X be a countable set of *variables*. Formulas are defined on three levels: *terms* combine variables and function symbols (and constants) to build composite individuals; *atomic formulas* apply a relation symbol to terms, moving into the Boolean world; finally operators combine atomic formulas, as in the summary. $T(\mathcal{F}, X)$ denotes the set of terms.

Other symbols can be defined:

$$\varphi \wedge \psi \stackrel{\text{def}}{=} \neg(\neg\varphi \vee \neg\psi), \quad \forall x.\varphi \stackrel{\text{def}}{=} \neg\exists x.\neg\varphi, \quad \varphi \Rightarrow \psi \stackrel{\text{def}}{=} \neg\varphi \vee \psi,$$
$$\top \stackrel{\text{def}}{=} \forall x.R(x,\dots,x) \vee \neg R(x,\dots,x), \quad \bot \stackrel{\text{def}}{=} \neg\top,$$

with R any symbol of $\mathcal{P}$. Formulas are trees; e.g. the *drinker's formula* $\exists x.(B(x) \Rightarrow \forall y.B(y))$.

5.3 Free and bound variables

The set $\mathrm{fv}(t)$ of variables of a term is $\mathrm{fv}(x) = \{x\}$, $\mathrm{fv}(f(t_1,\dots,t_m)) = \bigcup_i \mathrm{fv}(t_i)$. A term with no variable is *closed*; $T(\mathcal{F})$ is the set of closed terms.

A variable is *bound* when it occurs under a quantifier introducing it; the same variable may be free at one place and bound at another. Define:

$$\begin{aligned} \mathrm{fv}(R(t_1,\dots,t_m)) &= \textstyle\bigcup_i \mathrm{fv}(t_i) & \mathrm{bv}(R(t_1,\dots,t_m)) &= \emptyset \\ \mathrm{fv}(\neg\varphi) &= \mathrm{fv}(\varphi) & \mathrm{bv}(\neg\varphi) &= \mathrm{bv}(\varphi) \\ \mathrm{fv}(\varphi \vee \psi) &= \mathrm{fv}(\varphi) \cup \mathrm{fv}(\psi) & \mathrm{bv}(\varphi \vee \psi) &= \mathrm{bv}(\varphi) \cup \mathrm{bv}(\psi) \\ \mathrm{fv}(\exists x.\varphi) &= \mathrm{fv}(\varphi) \setminus \{x\} & \mathrm{bv}(\exists x.\varphi) &= \mathrm{bv}(\varphi) \cup \{x\} \end{aligned}$$

E.g. $B(x) \Rightarrow \forall y.B(y)$ has free variable x and bound variable y; in $B(x) \Rightarrow \forall x.B(x)$ the first x is free and the second bound. A formula without free variable is *closed*. If $\mathrm{fv}(\varphi) = \{x_1,\dots,x_n\}$ we write $\varphi(x_1,\dots,x_n)$.

5.4 Interpretation of a signature

As in propositional logic, we assign values to variables, but taken from an arbitrary set of individuals, which is also the range of the quantifiers. Moreover relations and functions must be interpreted. We distinguish: the *interpretation*, which defines the allowed values and the meaning of relation and function symbols; and the *valuation* of variables, which gives each variable a value among the allowed ones.

Definition 5.6. An *interpretation* I of $\mathcal{L} = (\mathcal{F}, \mathcal{P})$ consists of a non-empty set D_I (its *domain*), a function $f_I : D_I^{a(f)} \to D_I$ for each $f \in \mathcal{F}$, and a relation $R_I \subseteq D_I^{a(R)}$ for each $R \in \mathcal{P}$. We write $I = (D_I, (f_I)_{f\in\mathcal{F}}, (R_I)_{R\in\mathcal{P}})$.

If $=\in \mathcal{P}$, we always assume $=_I$ is equality on D_I ("normal" interpretations). The domain is non-empty so that, with a valid φ, $\exists x.\varphi$ is valid too.

Example 5.7. Propositional: $D_I = \{\bullet\}$, $P_I = 1$, $Q_I = 0$ represents $[1 \leftarrow p, 0 \leftarrow q]$. Orders: $D_I = \mathbb{Q}$ with the usual $<$. Arithmetic: $I = (\mathbb{N}, +, \times)$. Graph: $D_I = \{0, 1, 2\}$, $E_I = \{(0,1), (0,2), (1,0), (2,1), (2,2)\}$, $B_I = \{1, 2\}$.

5.5 Valuations and semantics of formulas

Definition 5.8. A *valuation* in I is a function $\rho : X \to D_I$. $\rho[e \leftarrow x]$ maps x to e and every other y to $\rho(y)$.

Definition 5.9 (Semantics of terms and formulas)**.**

$$
\begin{aligned}
[\![x]\!]_{I,\rho} &= \rho(x), \qquad [\![f(t_1, \ldots, t_m)]\!]_{I,\rho} = f_I([\![t_1]\!]_{I,\rho}, \ldots, [\![t_m]\!]_{I,\rho}) \\
[\![R(t_1, \ldots, t_m)]\!]_{I,\rho} &= 1 \text{ if } ([\![t_1]\!]_{I,\rho}, \ldots, [\![t_m]\!]_{I,\rho}) \in R_I, \text{ else } 0 \\
[\![\neg\varphi]\!]_{I,\rho} &= 1 \text{ iff } [\![\varphi]\!]_{I,\rho} = 0 \\
[\![\varphi \vee \psi]\!]_{I,\rho} &= 1 \text{ iff } [\![\varphi]\!]_{I,\rho} = 1 \text{ or } [\![\psi]\!]_{I,\rho} = 1 \\
[\![\exists x.\varphi]\!]_{I,\rho} &= 1 \text{ iff there is } e \in D_I \text{ with } [\![\varphi]\!]_{I,\rho[e\leftarrow x]} = 1 \\
[\![\forall x.\varphi]\!]_{I,\rho} &= 1 \text{ iff for all } e \in D_I,\ [\![\varphi]\!]_{I,\rho[e\leftarrow x]} = 1.
\end{aligned}
$$

We write $I, \rho \models \varphi$ if $[\![\varphi]\!]_{I,\rho} = 1$; inductively, $I, \rho \models R(\vec{t})$ iff $([\![\vec{t}]\!]_{I,\rho}) \in R_I$; $I, \rho \models \neg\varphi$ iff $I, \rho \not\models \varphi$; $I, \rho \models \varphi \vee \psi$ iff $I, \rho \models \varphi$ or ψ; $I, \rho \models \exists x.\varphi$ iff $I, \rho[e \leftarrow x] \models \varphi$ for some e; $I, \rho \models \forall x.\varphi$ iff for all e. Beware: the semantics of a term is an element of D_I, that of a formula is a truth value, despite the same notation.

Example 5.10. Signature $(\{+, \times\}, \{=\})$, $I = (\mathbb{N}, +, \times)$, formula $\exists x.\, y = x \times x$ (y free, x bound), and ρ with $\rho(x) = 3$, $\rho(y) = 9$. Then $[\![x \times x]\!]_{I,\rho} = 9$, so the *open* subformula $y = x \times x$ has value 1 under ρ. For the whole formula we need some $e \in \mathbb{N}$ with $[\![y = x \times x]\!]_{I,\rho[e\leftarrow x]} = 1$, i.e. $9 = e \times e$: take $e = 3$. Hence $[\![\exists x.\, y = x \times x]\!]_{I,\rho} = 1$. In general the valuations making it true are exactly those where $\rho(y)$ is a perfect square $(0, 1, 4, 9, 16, \ldots)$; the value $\rho(x)$ is irrelevant since it is overwritten by the quantifier (e.g. $\rho(y) = 4$, $\rho(x) = 10$ gives 1, while $\rho(y) = 8$ gives 0).

5.6 Models, satisfiability and validity

Definition 5.11. φ is *satisfiable* if there are I, ρ with $I, \rho \models \varphi$. I is a *model* of φ, $I \models \varphi$, if $I, \rho \models \varphi$ for every valuation ρ. φ is *valid*, $\models \varphi$, if every interpretation is a model of it.

Definition 5.12. For a set S of formulas: $I, \rho \models S$ if $I, \rho \models \psi$ for all $\psi \in S$. $S \models \varphi$ if every (I, ρ) with $I, \rho \models S$ satisfies φ. S is *unsatisfiable* if no (I, ρ) satisfies it, equivalently $S \models \bot$.

Property 5.13. *For any formula φ (resp. term t), interpretation I, valuations ρ, ρ' agreeing on* $\mathrm{fv}(\varphi)$ *(resp.* $\mathrm{fv}(t)$*):* $[\![\varphi]\!]_{I,\rho} = [\![\varphi]\!]_{I,\rho'}$ *(resp. for t). (By induction.)*

In particular, if $\mathrm{fv}(\varphi) = \{x_1, \ldots, x_n\}$, φ is satisfiable iff the closed formula $\exists x_1 \ldots \exists x_n.\varphi$ is, and valid iff $\forall x_1 \ldots \forall x_n.\varphi$ is.

Example 5.14. Peirce's law, over the propositional signature, is still valid: if $p_I = 1$ then p holds and so does the formula; if $p_I = 0$ then $p \Rightarrow q$ holds, so $(p \Rightarrow q) \Rightarrow p$ fails, and the whole formula holds.

Example 5.15 (Drinker's formula)**.** Over $(\emptyset, \{B^{(1)}\})$, $\varphi = \exists x.(B(x) \Rightarrow \forall y.B(y))$ ("there is someone such that if they drink then everybody drinks") is **valid**. Let I, ρ; we need $e \in D_I$ with $I, \rho[e \leftarrow x] \not\models B(x)$ or $I, \rho[e \leftarrow x] \models \forall y.B(y)$. If $B_I \subsetneq D_I$ pick a "sober" $e \notin B_I$. If $B_I = D_I$, pick any e (the domain is non-empty).

Example 5.16 (Queries on a graph)**.** In the graph I above, "there is a vertex of out-degree 1": $\exists x.\exists y.(E(x,y) \wedge \forall z.(y = z \vee \neg E(x,z)))$ is true: take $x = 1$, $y = 0$ and check for $z = 0, 1, 2$ that $y = z \vee \neg E(1,z)$ (because $(1,1), (1,2) \notin E_I$).

Example 5.17 (Orders)**.** On $(\mathbb{Q}, <)$ (or $\mathbb{Z}$), $\forall x.\exists y.\, y < x$ "there is no minimal element" holds; on $\mathbb{N}$ it fails: the formula is satisfiable but not valid. The closed formula $\forall x.\forall y.(x < y \Rightarrow \exists z.(x < z \wedge z < y))$ expresses *density*: true on $\mathbb{Q}$, false on $\mathbb{Z}$; satisfiable but not valid.

5.7 Queries on a database

First-order logic expresses database queries (it is another way to write SQL queries). A relational database is an interpretation with finite domain over a *relational signature*: suitable constants, one relation symbol per table, and equality. Let the constants be

$$\mathcal{F} = \{\mathit{shining}, \mathit{player}, \mathit{easyrider}, \mathit{apocalypsenow},\\ \mathit{kubrick}, \mathit{altman}, \mathit{hopper}, \mathit{nicholson}, \mathit{robbins},\\ \mathit{coppola}, \mathit{chambord}, \mathit{varietes}\}$$

and $\mathcal{P} = \{\mathit{Film}^{(3)}, \mathit{Seance}^{(2)}, =\}$ with

$$\begin{aligned}\mathit{Film}_I = \{&(\text{Shining}, \text{Kubrick}, \text{Nicholson}), (\text{The Player}, \text{Altman}, \text{Robbins}),\\ &(\text{Easy Rider}, \text{Hopper}, \text{Nicholson}),\\ &(\text{Easy Rider}, \text{Hopper}, \text{Hopper}), (\text{Apocalypse Now}, \text{Coppola}, \text{Hopper})\},\\ \mathit{Seance}_I = \{&(\text{Le Chambord}, \text{Shining}), (\text{Le Chambord}, \text{Easy Rider}), (\text{Le Chambord}, \text{The Player}),\\ &(\text{Les Variétés}, \text{Easy Rider})\}.\end{aligned}$$

For $\varphi(x_1, \ldots, x_n)$ the result of the query is $\varphi(I) = \{(e_1, \ldots, e_n) \mid I, [e_1 \leftarrow x_1, \ldots, e_n \leftarrow x_n] \models \varphi\}$. Examples:

- Film titles: $\exists r.\exists i.\, \mathit{Film}(x, r, i)$ (SQL: `SELECT DISTINCT titre FROM Films`).
- Cinemas showing a film with Hopper: $\exists t.\exists r.(\mathit{Film}(t, r, \mathit{hopper}) \wedge \mathit{Seance}(x, t))$, result {Le Chambord, Les Variétés}.
- Cinemas showing a Kubrick film with Hopper: $\exists t.(\mathit{Film}(t, \mathit{kubrick}, \mathit{hopper}) \wedge \mathit{Seance}(x, t))$, result $\emptyset$.
- Actors in a film directed by Kubrick or Coppola: $\exists t.(\mathit{Film}(t, \mathit{kubrick}, x) \vee \mathit{Film}(t, \mathit{coppola}, x))$, result {Nicholson, Hopper}.
- Directors who played in a film they directed: $\exists t.\mathit{Film}(t, x, x)$, result {Hopper}.
- Directors and the cinemas showing their films: $\exists t.\exists i.(\mathit{Film}(t, x, i) \wedge \mathit{Seance}(y, t))$.

- Directors whose films are shown in all cinemas: $\forall c.((\exists t.Seance(c,t)) \Rightarrow \exists t.\exists i.(Film(t,x,i) \wedge Seance(c,t)))$, result {Hopper} (SQL needs `NOT EXISTS ... EXCEPT`).
- Actors who played in only one film of the base: $\exists t.\exists r.(Film(t,r,x) \wedge \forall t'.\forall r'.(Film(t',r',x) \Rightarrow t = t'))$, result {Robbins}.

5.8 Logical equivalence

Definition 5.18. Two first-order formulas φ, ψ are *logically equivalent*, $\varphi \equiv \psi$, if $\varphi \models \psi$ and $\psi \models \varphi$.

Equivalently, for all I, ρ: $I, \rho \models \varphi$ iff $I, \rho \models \psi$; as in Property 1.17 this holds iff $\varphi \Leftrightarrow \psi$ is valid.

Chapter 6

Transformation of first-order formulas

Summary

A *substitution* is a function σ with finite domain mapping each variable $x \in X$ to a term $\sigma(x) \in T(\mathcal{F}, X)$; $\sigma(t)$ is the term t where each variable x is replaced by $\sigma(x)$. A substitution is *applicable* to φ if it does not interact with the bound variables of φ; $\sigma(\varphi)$ is then φ with each x replaced by $\sigma(x)$. Up to *renaming* of bound variables (without capture) a substitution can always be applied. The *renaming* of $\exists x.\psi$ with x' not occurring in ψ is $\exists x'.[x' \leftarrow x](\psi)$.
For an interpretation I and valuation ρ, ρ_σ is the valuation $x \mapsto [\![\sigma(x)]\!]_{I,\rho}$; the *substitution lemma* states $[\![\sigma(\varphi)]\!]_{I,\rho} = [\![\varphi]\!]_{I,\rho_\sigma}$.
A formula is in *negation normal form* if $\ell ::= \alpha \mid \neg\alpha$, $\varphi, \psi ::= \ell \mid \varphi \vee \psi \mid \varphi \wedge \psi \mid \exists x.\varphi \mid \forall x.\varphi$. $\mathrm{NNF}(\varphi)$ is obtained by pushing negations to the leaves as in propositional logic.

Propositional substitutions τ replaced propositions p by formulas $\tau(p)$ and related two interpretations I and I_τ. In first-order logic, a substitution σ replaces *variables* x by *terms* $\sigma(x)$ and relates two valuations ρ and ρ_σ. This gives classical equivalences, added to the propositional identities of Theorem 2.8 (all still true). We finish with negation normal form (the only one needed later) and prenex form.

6.1 Substitutions

A *substitution* is a function $\sigma : X \to T(\mathcal{F}, X)$ with finite domain $\mathrm{dom}(\sigma) = \{x \mid \sigma(x) \neq x\}$. We write $\sigma = [\sigma(x_1) \leftarrow x_1, \ldots, \sigma(x_n) \leftarrow x_n]$. It extends to terms by $\sigma(f(t_1, \ldots, t_m)) = f(\sigma(t_1), \ldots, \sigma(t_m))$. A substitution with image in closed terms is *closed*.

Example 6.1. With $\mathcal{F} = \{+, \times\} \cup \{\underline{i}\}_{i \in \mathbb{N}}$, the substitution $\sigma = [3 + y \leftarrow x,\ z \leftarrow y]$ applied to $(x + 3) \times (y + 2)$ gives $((3 + y) + 3) \times (z + 2)$.

For formulas, bound variables matter: naively applying $[y + 1 \leftarrow x]$ to $\exists y.\, x < y$ (true for all x) gives $\exists y.\, y + 1 < y$, false in arithmetic, since y in the image gets captured by $\exists y$. Define the *range variables* $\mathrm{rv}(\sigma) = \bigcup_{x \in \mathrm{dom}(\sigma)} \mathrm{fv}(\sigma(x))$; e.g. $\mathrm{rv}([3 + y \leftarrow x, z \leftarrow y]) = \{y, z\}$.

Definition 6.2. σ is *applicable* to φ if $(\mathrm{dom}(\sigma) \cup \mathrm{rv}(\sigma)) \cap \mathrm{bv}(\varphi) = \emptyset$. Then $\sigma(\varphi)$ is defined by $\sigma(R(\vec{t})) = R(\sigma(\vec{t}))$, $\sigma(\neg\varphi) = \neg\sigma(\varphi)$, $\sigma(\varphi \vee \psi) = \sigma(\varphi) \vee \sigma(\psi)$, $\sigma(\exists x.\varphi) = \exists x.\sigma(\varphi)$.

E.g. $\sigma = [3 + y \leftarrow x, z \leftarrow y]$ is applicable to $\exists u.\, u + x = z \times y$, giving $\exists u.\, u + (3 + y) = z \times z$. It is not applicable to $\forall x.\, x = 2$ ($x \in \mathrm{dom}\sigma$) nor to $\exists z.\, z = x$ ($z \in \mathrm{rv}(\sigma)$).

Substitutions work inside the logic, independently of any interpretation; valuations depend on an interpretation, but a substitution can be transferred to a valuation.

Definition 6.3. For a substitution σ and valuation ρ, ρ_σ is the valuation $x \mapsto [\![\sigma(x)]\!]_{I,\rho}$.

Property 6.4. *For every term t: $[\![\sigma(t)]\!]_{I,\rho} = [\![t]\!]_{I,\rho_\sigma}$.*

Proof. Induction. Variable: $[\![\sigma(x)]\!]_{I,\rho} = \rho_\sigma(x) = [\![x]\!]_{I,\rho_\sigma}$. For $f(t_1, \dots, t_m)$: $[\![f(\sigma(t_1), \dots, \sigma(t_m))]\!]_{I,\rho} = f_I([\![\sigma(t_1)]\!]_{I,\rho}, \dots) = f_I([\![t_1]\!]_{I,\rho_\sigma}, \dots) = [\![f(t_1, \dots, t_m)]\!]_{I,\rho_\sigma}$. □

Property 6.5. *For φ a formula and σ applicable to φ: $[\![\sigma(\varphi)]\!]_{I,\rho} = [\![\varphi]\!]_{I,\rho_\sigma}$.*

Proof. Induction on φ. Atomic, negation, disjunction follow from the term property and induction. For $\exists x.\varphi$: $[\![\sigma(\exists x.\varphi)]\!]_{I,\rho} = \max_e [\![\sigma(\varphi)]\!]_{I,\rho[e \leftarrow x]} = \max_e [\![\varphi]\!]_{I,\rho[e \leftarrow x]_\sigma}$. We show $\rho[e \leftarrow x]_\sigma(z) = \rho_\sigma[e \leftarrow x](z)$ for all $z \in \mathrm{fv}(\varphi)$, so that the last term is $[\![\exists x.\varphi]\!]_{I,\rho_\sigma}$. If $z = x$: $x \notin \mathrm{dom}(\sigma)$ since $x \in \mathrm{bv}(\exists x.\varphi)$, so $\sigma(x) = x$ and $\rho[e \leftarrow x]_\sigma(x) = e = \rho_\sigma[e \leftarrow x](x)$. If $z \neq x$: $x \notin \mathrm{rv}(\sigma)$, so $x \notin \mathrm{fv}(\sigma(z))$ and $[\![\sigma(z)]\!]_{I,\rho[e \leftarrow x]} = [\![\sigma(z)]\!]_{I,\rho} = \rho_\sigma(z)$. □

6.2 Renaming

When σ is not applicable we can rename bound variables: $B(x) \Rightarrow \forall y.B(y)$ becomes the equivalent $B(x) \Rightarrow \forall z.B(z)$. The substitution above, not applicable to $\exists z.\, z = x$, is applicable to $\exists u.\, u = x$.

Formally, an (α-)*renaming* transforms $\exists x.\varphi$ into $\exists y.[y \leftarrow x](\varphi)$ where $y \notin \mathrm{fv}(\exists x.\varphi)$ and $[y \leftarrow x]$ is applicable to φ (same for $\forall$). It can be done in any subformula: $\forall x.\exists z.\, z = x$ renames to $\forall x.\exists u.\, u = x$ then to $\forall y.\exists u.\, u = y$. We write $\varphi =_r \psi$ if φ can be renamed into ψ in one or more steps, and reason *up to renaming*.

Property 6.6 (Applicability)**.** *For every substitution σ and formula φ there is φ' with $\varphi =_r \varphi'$ and σ applicable to φ'; if also $\varphi =_r \varphi''$ with σ applicable, then $\varphi' =_r \varphi''$ and $\sigma(\varphi') =_r \sigma(\varphi'')$.*

Property 6.7. *If $\varphi =_r \psi$ then $\varphi \equiv \psi$.*

We therefore write $\sigma(\varphi)$ for $\sigma(\varphi')$ with $\varphi =_r \varphi'$ and σ applicable to φ'.

Lemma 6.8 (Substitution lemma)**.** *For every term t, formula φ, substitution σ, interpretation I and valuation ρ: $[\![\sigma(t)]\!]_{I,\rho} = [\![t]\!]_{I,\rho_\sigma}$ and $[\![\sigma(\varphi)]\!]_{I,\rho} = [\![\varphi]\!]_{I,\rho_\sigma}$.*

Corollary 6.9. *If φ is a valid first-order formula over propositional-like symbols and τ a propositional substitution, then $\tau(\varphi)$ is valid.*

Example 6.10. In the arithmetic signature with $I = (\mathbb{N}, +, \times)$ one can define in the logic:

- the number 0: $zero(x) \stackrel{\text{def}}{=} x + x = x$. (If φ is not closed with $\mathrm{fv}(\varphi) = \{x_1, \dots, x_n\}$ we write $\varphi(x_1, \dots, x_n)$; $zero(y)$ denotes $[y \leftarrow x](zero)$.) Then $I \models \exists x.(zero(x) \wedge \forall y.(zero(y) \Rightarrow x = y))$: exactly one element, 0.
- the number 1: $one(x) \stackrel{\text{def}}{=} \forall y.\, x \times y = y$;
- the strict order: $x < y \stackrel{\text{def}}{=} \exists z.(\neg zero(z) \wedge y = x + z)$;

- evenness: $even(x) \stackrel{\text{def}}{=} \exists y.\, x = y + y$;
- primality: $prime(x) \stackrel{\text{def}}{=} \neg zero(x) \wedge \neg \exists y. \exists z. (x = y \times z \wedge \neg one(y) \wedge \neg one(z))$ (this also accepts 1, so one should add $\neg one(x)$ for the usual notion);
- a closed formula saying every prime greater than 2 is odd: $\forall x. ((prime(x) \wedge \exists y. (x > y + y \wedge one(y))) \Rightarrow \neg even(x))$.

6.3 Equivalences to know

Theorem 6.11. *For all first-order formulas φ, ψ and variables x, y:*

$$\forall x.\varphi \equiv \neg\exists x.\neg\varphi \qquad \exists x.\varphi \equiv \neg\forall x.\neg\varphi \qquad (\exists/\forall \text{ conversion})$$
$$\exists x.(\varphi \vee \psi) \equiv (\exists x.\varphi) \vee (\exists x.\psi) \qquad \forall x.(\varphi \wedge \psi) \equiv (\forall x.\varphi) \wedge (\forall x.\psi) \qquad \text{(distribution)}$$
$$\exists x.\exists y.\varphi \equiv \exists y.\exists x.\varphi \qquad \forall x.\forall y.\varphi \equiv \forall y.\forall x.\varphi \qquad \text{(permutation)}$$

Moreover if $x \notin \mathrm{fv}(\psi)$:

$$\forall x.(\varphi \wedge \psi) \equiv (\forall x.\varphi) \wedge \psi \qquad \forall x.(\varphi \vee \psi) \equiv (\forall x.\varphi) \vee \psi$$
$$\exists x.(\varphi \wedge \psi) \equiv (\exists x.\varphi) \wedge \psi \qquad \exists x.(\varphi \vee \psi) \equiv (\exists x.\varphi) \vee \psi.$$

Warning: $\exists x.(\varphi \wedge \psi)$ is *not* equivalent to $(\exists x.\varphi) \wedge (\exists x.\psi)$ in general (and dually $\forall x.(\varphi \vee \psi) \not\equiv (\forall x.\varphi) \vee (\forall x.\psi)$).

6.4 Normal forms

6.4.1 Negation normal form

A first-order formula is in *NNF* if $\ell ::= \alpha \mid \neg\alpha$ and $\varphi, \psi ::= \ell \mid \varphi \vee \psi \mid \varphi \wedge \psi \mid \exists x.\varphi \mid \forall x.\varphi$: negations only occur in front of atomic formulas. Pushing them to the leaves:

$$\mathrm{NNF}(\alpha) = \alpha \qquad \mathrm{NNF}(\neg\alpha) = \neg\alpha$$
$$\mathrm{NNF}(\varphi \star \psi) = \mathrm{NNF}(\varphi) \star \mathrm{NNF}(\psi) \qquad \mathrm{NNF}(\neg(\varphi \vee \psi)) = \mathrm{NNF}(\neg\varphi) \wedge \mathrm{NNF}(\neg\psi)$$
$$\mathrm{NNF}(Qx.\varphi) = Qx.\mathrm{NNF}(\varphi) \qquad \mathrm{NNF}(\neg(\varphi \wedge \psi)) = \mathrm{NNF}(\neg\varphi) \vee \mathrm{NNF}(\neg\psi)$$
$$\mathrm{NNF}(\neg\exists x.\varphi) = \forall x.\mathrm{NNF}(\neg\varphi) \qquad \mathrm{NNF}(\neg\forall x.\varphi) = \exists x.\mathrm{NNF}(\neg\varphi)$$
$$\mathrm{NNF}(\neg\neg\varphi) = \mathrm{NNF}(\varphi)$$

($\star \in \{\vee, \wedge\}$, $Q \in \{\exists, \forall\}$.) This is linear time and preserves semantics. We write $\overline{\varphi} \stackrel{\text{def}}{=} \mathrm{NNF}(\neg\varphi)$.

Example 6.12. The drinker's formula has NNF $\exists x.(\neg B(x) \vee \forall y.B(y))$; its negation has NNF $\forall x.(B(x) \wedge \exists y.\neg B(y))$.

6.4.2 Prenex normal form

It is sometimes simpler to avoid alternation between quantifiers and Boolean operations; the drinker's formula is equivalent to $\exists x.\forall y.(\neg B(x) \vee B(y))$. A formula is in *prenex normal form* if it has the form $Q_1 x_1 \cdots Q_n x_n.\varphi$ with $Q_i \in \{\forall, \exists\}$ and φ quantifier-free.

Property 6.13. *Every formula is equivalent to a formula in prenex normal form.*

Proof. Assume φ in NNF; induct on φ. A literal is already prenex. For $\varphi \vee \varphi'$ (resp. $\wedge$): by induction, $\varphi \equiv Q_1 x_1 \dots Q_n x_n.\psi$ and $\varphi' \equiv Q'_1 y_1 \dots Q'_m y_m.\psi'$; renaming bound variables we may assume $\{x_i\} \cap \mathrm{fv}(\psi') = \emptyset$ and $\mathrm{fv}(\psi) \cap \{y_j\} = \emptyset$. Using the pass-through laws repeatedly (induction on $n+m$), the disjunction is equivalent to $Q_1 x_1 \dots Q_n x_n . Q'_1 y_1 \dots Q'_m y_m . (\psi \vee \psi')$. For $\exists x.\varphi$ or $\forall x.\varphi$ simply prefix the quantifier. $\square$

Example 6.14. Density $\forall x.\forall y.(x < y \Rightarrow \exists z.(x < z \wedge z < y))$ has NNF $\forall x.\forall y.((\neg x < y) \vee \exists z.(x < z \wedge z < y))$ and prenex form $\forall x.\forall y.\exists z.((\neg x < y) \vee (x < z \wedge z < y))$.

Example 6.15. $\forall c.((\exists t.\mathit{Seance}(c,t)) \Rightarrow \exists t.\exists i.(\mathit{Film}(t,x,i) \wedge \mathit{Seance}(c,t)))$ has NNF $\forall c.((\forall t.\neg \mathit{Seance}(c,t)) \vee \exists t.\exists i.(\dots))$. Rename one t to t' and obtain the prenex form

$$\forall c.\forall t.\exists t'.\exists i.(\neg \mathit{Seance}(c,t) \vee (\mathit{Film}(t',x,i) \wedge \mathit{Seance}(c,t'))).$$

Chapter 7

Theories

Summary

A *theory* is a set T of closed formulas over a first-order signature $(\mathcal{F}, \mathcal{P})$ such that if φ is closed and $T \models \varphi$ then $\varphi \in T$: T contains all closed logical consequences of its formulas. Theories restrict the interpretations considered to those satisfying closed formulas known to be true.
For a class K of interpretations over the same signature (a *structure*), $\mathrm{Th}(K) = \{\varphi \text{ closed} \mid \forall I \in K,\ I \models \varphi\}$. Since describing K is hard, we use *axiomatic theories*: a set A of *axioms* (closed formulas) and $\mathrm{Th}(A) = \{\varphi \text{ closed} \mid A \models \varphi\}$.

- *Equivalence relations*, for $E^{(2)} \in \mathcal{P}$: A_{eq} = reflexivity $\forall x.E(x,x)$; symmetry $\forall x.\forall y.E(x,y) \Rightarrow E(y,x)$; transitivity $\forall x.\forall y.\forall z.(E(x,y) \wedge E(y,z)) \Rightarrow E(x,z)$.
- *Congruences*: $A_{cgr}(\mathcal{L})$ adds to A_{eq}, for each $f \in \mathcal{F}_n$: $\forall \vec{x}.\forall \vec{y}.(E(x_1,y_1) \wedge \cdots \wedge E(x_n,y_n) \Rightarrow E(f(\vec{x}),f(\vec{y})))$ (f-congruence), and for each $R \in \mathcal{P}_m \setminus \{E\}$: $\forall \vec{x}.\forall \vec{y}.((E(x_1,y_1) \wedge \cdots \wedge E(x_m,y_m) \wedge R(\vec{x})) \Rightarrow R(\vec{y}))$ (R-congruence).
- *Strict linear orders*, over $(\emptyset, \{<,=\})$: irreflexivity $\forall x.\neg(x<x)$; transitivity; totality $\forall x.\forall y.(x<y \vee x=y \vee y<x)$.
- *Dense unbounded strict linear orders*: add density $\forall x.\forall y.\exists z.((x<y) \Rightarrow (x<z \wedge z<y))$ and unboundedness $\forall x.\exists y.\exists z.(z<x \wedge x<y)$.

We are mostly interested in theories that are *consistent* (they have a model; an inconsistent theory contains all closed formulas) and *decidable* (an algorithm tells whether $\varphi \in T$). Dense unbounded linear orders, rational linear arithmetic and real arithmetic are decidable; we use the latter two a lot afterwards.

Taken in isolation, a first-order formula uses uninterpreted symbols that can be interpreted in arbitrary, possibly counter-intuitive ways. In the signature $(\emptyset, \{<,=\})$ of strict orders the closed formula $\exists x.(x<x)$ is satisfiable, e.g. by $D_I = \{e\}$, $<_I = \{(e,e)\}$. We want to restrict interpretations to forbid such counter-intuitive ones.

7.1 Logical theories

For closed S and closed φ, $S \models \varphi$ is equivalent to: for every interpretation I, $I \models S$ implies $I \models \varphi$.

Definition 7.1. A *theory* is a set T of closed formulas over the same signature such that $T \models \varphi$ implies $\varphi \in T$ for any closed φ.

The set of all closed formulas is a theory, but of limited interest.

7.1.1 Theories of structures

Definition 7.2. $\mathrm{Th}(I) = \{\varphi \text{ closed} \mid I \models \varphi\}$, and for a class K: $\mathrm{Th}(K) = \{\varphi \text{ closed} \mid \forall I \in K,\ I \models \varphi\}$.

Property 7.3. *For any set K of interpretations, $\mathrm{Th}(K)$ is a theory.*

Proof. If $\mathrm{Th}(K) \models \varphi$: for each $I \in K$, $I \models \mathrm{Th}(K)$ by definition, so $I \models \varphi$, so $\varphi \in \mathrm{Th}(K)$. □

Example 7.4. Signature $(\emptyset, \{P^{(0)}, Q^{(0)}\})$, $D_I = \{\bullet\}$, $P_I = 1$, $Q_I = 0$: $\mathrm{Th}(I)$ contains P, $\neg Q$, $P \vee Q \vee Q$, $\neg\neg P$, $Q \vee \exists x.P$, etc.

7.1.2 Axiomatic theories

Theories of structures do not solve our problem: how do we define the class K in practice? It is convenient to specify in the logic the expected properties, e.g. irreflexivity $\forall x.\neg(x < x)$.

Definition 7.5. An *axiomatisation* A is a set of closed formulas over a signature. The *theory of A* is $\mathrm{Th}(A) = \{\varphi \text{ closed} \mid A \models \varphi\}$, an *axiomatic theory*. If A is finite, $\mathrm{Th}(A)$ is *definable*.

Example 7.6 (Equivalence relations)**.** Equivalence relations are crucial in computer science and mathematics: congruence modulo n on integers, or "belonging to the same residual languages of L" for words, which has finitely many classes iff L is regular. For $\mathcal{P} = \{E^{(2)}\}$, A_{eq} (reflexivity, symmetry, transitivity as in the summary) is such that $I \models A_{eq}$ iff E_I is an equivalence relation on D_I. For example $D_I = \{a, b, c\}$, $E_I = \{(a,a), (a,b), (b,a), (b,b), (c,c)\}$ is a model.

Example 7.7 (Congruences)**.** Some equivalences are stronger: they pass through functions and relations. Congruence modulo n is compatible with $+$ and $\times$: if $m \equiv m'$ and $p \equiv p'$ then $m + p \equiv m' + p'$ and $mp \equiv m'p'$. For $\mathcal{L} = (\mathcal{F}, \mathcal{P})$ with $E \in \mathcal{P}$, $A_{cgr}(\mathcal{L})$ as in the summary satisfies $I \models A_{cgr}(\mathcal{L})$ iff E_I is a congruence. Equality, forced to be the semantics of $=$, is always an equivalence and a congruence.

Example 7.8 (Strict orders)**.** For $(\emptyset, \{<, =\})$, A_{ord} (irreflexivity, transitivity) characterises strict partial orders; elements may be incomparable.

Example 7.9 (Linear orders)**.** A_{lin} adds totality. E.g. $\mathbb{N}$, $\mathbb{R}$ are models.

Example 7.10 (Dense unbounded orders)**.** A_{dense} adds density (between any two elements, one can insert another) and unboundedness (an element always exists above and below).

7.1.3 Consistency, completeness and decidability

Definition 7.11. A theory T is *consistent* if it does not contain both φ and $\neg\varphi$.

An inconsistent theory has no model, but contains every closed formula (as $T \models \psi$ vacuously).

Property 7.12. *A theory is consistent iff it has a model iff it does not contain all closed formulas. Hence if $K \neq \emptyset$ then $\mathrm{Th}(K)$ is consistent; and $\mathrm{Th}(A)$ is consistent iff A has a model.*

Definition 7.13. T is *complete* if for every closed φ, $\varphi \in T$ or $\neg\varphi \in T$.

For a single interpretation I, $\mathrm{Th}(I)$ is always complete since $I \models \varphi$ or $I \models \neg\varphi$. In contrast $\mathrm{Th}(A)$ need not be complete: for A_{eq} and $\varphi = \forall x.\forall y.E(x,y)$ (one equivalence class), neither $A_{eq} \models \varphi$ (models with several classes) nor $A_{eq} \models \neg\varphi$ (models with one class).

Definition 7.14. T is *decidable* if there is an algorithm that, given a closed φ, answers whether $\varphi \in T$ (the DECISION OF T problem, the *Entscheidungsproblem*).

If $T = \mathrm{Th}(A)$ with A finite, then $\varphi \in T$ iff $A \models \varphi$ iff $(\bigwedge_{\psi \in A} \psi) \Rightarrow \varphi$ is valid: the problem reduces to VALIDITY. For $T = \mathrm{Th}(\emptyset)$, deciding T is exactly first-order validity (close the formula by universal quantifiers). For $T = \mathrm{Th}(I)$, decidability amounts to the problem EVALUATION IN I: given closed φ, does $I \models \varphi$? Easy for propositional logic, not always for first-order logic.

Decidable theories. The theory of dense unbounded linear orders $\mathrm{Th}(A_{dense})$ (Section 7.2.1); rational linear arithmetic $\mathrm{Th}(\mathbb{Q}, 1, (q\cdot)_{q\in\mathbb{Q}}, +, <)$, with domain $\mathbb{Q}$, the constant 1, unary functions $x \mapsto q \cdot x$, addition and the strict order (a refinement of the previous one: Section 7.2.2); and real arithmetic $\mathrm{Th}(\mathbb{R}, +, \times)$ (multiplication of two numbers is allowed, but no constant 1, hence no way to isolate the integers).

Undecidable theories. The most famous is elementary arithmetic, with $\mathcal{F} = \{0^{(0)}, s^{(1)}, +^{(2)}, \times^{(2)}\}$ and $\mathcal{P} = \{=\}$ together with its natural axioms; in fact every theory containing elementary arithmetic is undecidable. What makes rational linear arithmetic decidable is its linearity: one cannot multiply two numbers, only a number by a constant. Also $\mathrm{Th}(\mathbb{Z}, +, \times)$ is undecidable.

7.2 Quantifier elimination and decidability

A formula is *quantifier-free* if $\psi ::= \ell \mid \psi \vee \psi \mid \psi \wedge \psi \mid \top \mid \bot$.

A theory T allows *quantifier elimination* if for every φ there is a quantifier-free ψ with φ and ψ equivalent modulo T, i.e. $T \models \varphi \Leftrightarrow \psi$. It is *effective* if an algorithm computes ψ. If φ is closed, ψ has no variable, and it is (generally) easy to decide whether $T \models \psi$.

A formula is *primitive existential* if it has the form $\varepsilon = \exists x.(\ell_1 \wedge \cdots \wedge \ell_n)$ with ℓ_i literals and $x \in \mathrm{fv}(\ell_i)$ for all i.

Lemma 7.15. *If a quantifier-free* $\mathrm{qe}_T(\varepsilon)$ *equivalent to* ε *modulo* T *can be computed for every primitive existential* ε*, then one can be computed for every formula* φ*.*

Proof. Define $\mathrm{qe}_T(\varphi)$ by induction. Atomic: $\mathrm{qe}_T(\alpha) = \alpha$. Negation: $\mathrm{qe}_T(\neg\varphi) = \mathrm{NNF}(\neg \mathrm{qe}_T(\varphi))$. Disjunction: $\mathrm{qe}_T(\varphi \vee \psi) = \mathrm{qe}_T(\varphi) \vee \mathrm{qe}_T(\psi)$. For $\exists x.\varphi$, we have $T \models (\exists x.\varphi) \Leftrightarrow (\exists x.\mathrm{qe}_T(\varphi))$ and:

1. put $\mathrm{qe}_T(\varphi)$ into DNF (literals as propositions): $T \models (\exists x.\varphi) \Leftrightarrow \exists x. \bigvee_j \bigwedge_i \ell_{i,j}$;
2. as $\exists x.(\psi \vee \psi') \equiv (\exists x.\psi) \vee (\exists x.\psi')$, it suffices to treat each $\exists x. \bigwedge_i \ell_{i,j}$;
3. as $\exists x.(\psi \wedge \psi') \equiv \psi \wedge \exists x.\psi'$ when $x \notin \mathrm{fv}(\psi)$, split the literals into $I_j = \{i \mid x \in \mathrm{fv}(\ell_{i,j})\}$ and its complement $\overline{I_j}$: $\exists x. \bigwedge_i \ell_{i,j} \equiv \bigwedge_{i\in\overline{I_j}} \ell_{i,j} \wedge \exists x. \bigwedge_{i\in I_j} \ell_{i,j}$;
4. $\exists x. \bigwedge_{i\in I_j} \ell_{i,j}$ is primitive existential, so by hypothesis we compute its qe_T and set $\mathrm{qe}_T(\varphi) = \bigvee_j (\bigwedge_{i\in\overline{I_j}} \ell_{i,j} \wedge \mathrm{qe}_T(\exists x. \bigwedge_{i\in I_j} \ell_{i,j}))$. □

7.2.1 Dense unbounded linear orders

Signature $(\emptyset, \{<, =\})$. The interpretation $(\mathbb{Q}, <)$ is a model of A_{dense}, so the theory is consistent.

Lemma 7.16. $\mathrm{Th}(A_{dense})$ *effectively allows quantifier elimination.*

Proof. Preliminary remark: a negative literal is either $\neg(x = y)$, equivalent to $x < y \vee y < x$, or $\neg(x < y)$, equivalent to $x = y \vee y < x$ (by totality). By Lemma 7.15 it suffices to treat $\varepsilon = \exists x.\psi$ primitive existential, and we may assume all literals positive.

(A) If $x < x$ occurs in ψ then $\varepsilon \equiv \bot$ by irreflexivity.

(B) If $x = y$ occurs in ψ with y different from x: $\varepsilon \equiv [y \leftarrow x](\psi)$ (transitivity and congruence of $=$).

(C) A literal $x = x$ can be deleted.

(D) Only literals $y < x$ and $x < z$ remain: $\varepsilon \equiv \exists x.\, \bigwedge_{i \le m} y_i < x \wedge \bigwedge_{j \le n} x < z_j$ with x different from all y_i, z_j. (a) If $m = 0$ or $n = 0$, as $<$ is unbounded, $\mathrm{qe}(\varepsilon) = \top$. (b) Otherwise by transitivity and density, $\mathrm{qe}(\varepsilon) = \bigwedge_{i \le m} \bigwedge_{j \le n} y_i < z_j$. □

Example 7.17. Let $\varphi = \exists x.\exists y.\exists z.(x < y \wedge y < z \wedge \neg(x < z))$, which contradicts transitivity. It is equivalent to $\exists x.\exists y.(x < y \wedge \exists z.(y < z \wedge \neg(x < z)))$. Now $\neg(x < z) \equiv z < x \vee x = z$, giving $\exists z.(y < z \wedge z < x)$ and $\exists z.(y < z \wedge x = z)$; both eliminate to $y < x$ (cases D.b and B). We obtain $\exists x.\exists y.(x < y \wedge y < x)$; the primitive $\exists y.(x < y \wedge y < x)$ gives $x < x$ (D.b), then $\exists x.\, x < x$ gives $\bot$ (A). So $\mathrm{qe}(\varphi) = \bot$.

Theorem 7.18. $\mathrm{Th}(A_{dense})$ *is decidable.*

Proof. For closed φ, compute the variable-free $\mathrm{qe}(\varphi)$. As $\mathcal{F} = \emptyset$, there are no closed terms, so it is $\top$ or $\bot$, and the answer is yes or no respectively. □

7.2.2 Rational linear arithmetic

Signature $\mathcal{F} = \{1^{(0)}, (q\cdot^{(1)})_{q \in \mathbb{Q}}, +^{(2)}\}$, $\mathcal{P} = \{<, =\}$ and the structure $(\mathbb{Q}, 1, (q\cdot)_{q \in \mathbb{Q}}, +, <)$, a model of A_{dense}.

Lemma 7.19 (Fourier–Motzkin)**.** $\mathrm{Th}(\mathbb{Q}, 1, (q\cdot)_{q \in \mathbb{Q}}, +, <)$ *effectively allows quantifier elimination.*

Proof. As before it suffices to treat $\varepsilon = \exists x.\psi$ primitive existential with positive literals $t < t'$ or $t = t'$, which can be rewritten as literals $x < t$, $t < x$ or $x = t$ (e.g. $5x + y < 2x - y + z$ becomes $x < -\frac{2}{3}y + \frac{1}{3}z$). Cases (A)–(C) are as before (for (B) use $[t \leftarrow x]\psi$). In case (D), $\varepsilon \equiv \exists x.\, \bigwedge_i t_i < x \wedge \bigwedge_j x < t'_j$ with $x \notin \mathrm{fv}(t_i), \mathrm{fv}(t'_j)$: if $m = 0$ or $n = 0$, $\mathrm{qe} = \top$ since $\mathbb{Q}$ is unbounded; otherwise by density and transitivity $\mathrm{qe} = \bigwedge_i \bigwedge_j t_i < t'_j$. □

Theorem 7.20. $\mathrm{Th}(\mathbb{Q}, 1, (q\cdot)_{q \in \mathbb{Q}}, +, <)$ *is decidable.*

Proof. Reduce φ closed to the variable-free $\mathrm{qe}(\varphi)$, a Boolean combination of $t < t'$, $t = t'$ with t, t' closed terms built from 1, $+$ and multiplications by constants; compute their values in $\mathbb{Q}$. □

7.2.3 Gödel's incompleteness theorems

A theory $\mathrm{Th}(A)$ is *recursively axiomatisable* if an algorithm decides whether a formula belongs to A. Definable theories (A finite) are, but so is Peano arithmetic (elementary arithmetic plus the induction principle, which needs infinitely many axioms).

Theorem 7.21 (First incompleteness theorem (Rosser's form))**.** *Every consistent, recursively axiomatisable theory containing elementary arithmetic is not complete.*

Peano arithmetic is consistent, hence incomplete: some statements φ are such that neither φ nor $\neg\varphi$ is provable.

Theorem 7.22 (Second incompleteness theorem)**.** *No consistent, recursively axiomatisable theory T containing Peano arithmetic contains* $\mathrm{Coh}(T)$*, the closed arithmetic formula expressing the consistency of T.*

In short: a consistent theory cannot prove its own consistency.

Link with constraint programming

Quantifier elimination is *projection* of a constraint on some of its variables (Fourier–Motzkin for linear constraints). Theories such as linear arithmetic are the domains over which CP solvers work. See Chapter 10.

Chapter 8

Satisfiability modulo theories

Summary

An SMT solver (satisfiability modulo theory) such as Z3 finds a pair (I, ρ) satisfying a given formula, where I moreover satisfies a fixed theory T.
Sometimes the theory is that of a single interpretation (e.g. rational linear arithmetic) and the solver only has to produce a valuation. In other cases the solver also finds the interpretation, or part of it: we use *uninterpreted* function or relation symbols which must satisfy some conditions, besides the other symbols which satisfy the fixed theory. If the formula is closed, the solver need not return a valuation, only the interpretation of the uninterpreted symbols.

A formula is satisfiable if there are I, ρ with $I, \rho \models \varphi$. But satisfiability of a formula in isolation is often insufficient since we want to restrict the possible interpretations of function and relation symbols. We therefore fix a theory T and consider the problem Satisfiability modulo T: **input** a formula φ; **question** are there I and ρ such that $I \models T$ and $I, \rho \models \varphi$? A *witness* is such a pair (I, ρ).

In many practical cases T is $\mathrm{Th}(I)$ for a fixed I, like $(\mathbb{Q}, 1, (q\cdot)_{q\in\mathbb{Q}}, +, <)$ (rational linear arithmetic) or $(\mathbb{R}, +, \times)$ (real arithmetic), both decidable. The problem then amounts to finding a valuation ρ with $I, \rho \models \varphi$.

8.1 Modulo-theory solvers

An *SMT solver* solves Satisfiability modulo T and returns a witness. We use Z3. In its most basic form an SMT solver handles quantifier-free formulas over a fixed T. E.g. is

$$x + y \geq 0 \ \wedge \ (x \neq z \Rightarrow y + z = -1) \ \wedge \ z > 3 \cdot t$$

satisfiable in rational linear arithmetic? Solvers really solve *conjunctions of literals*. One could write the formula as the disjunction $(x{+}y \geq 0 \wedge \neg(x \neq z) \wedge z > 3t) \vee (x{+}y \geq 0 \wedge y{+}z = -1 \wedge z > 3t)$ and solve each system of linear (in)equations over $\mathbb{Q}$, but this may be exponential. SMT solvers instead use SAT solvers for the Boolean part:

1. Associate a proposition to each literal: here $p_1 \wedge (p_2 \Rightarrow p_3) \wedge p_4$ with $p_1 : x + y \geq 0$, $p_2 : x \neq z$, $p_3 : y + z = -1$, $p_4 : z > 3t$.
2. Convert into clausal form $(\{\{p_1\}, \{\neg p_2, p_3\}, \{p_4\}\})$ and give it to a SAT solver. If it is unsatisfiable, so is the first-order formula.

3. Otherwise the SAT solver returns a model, e.g. $[1 \leftarrow p_1, 0 \leftarrow p_2, 1 \leftarrow p_3, 1 \leftarrow p_4]$, corresponding to the conjunction of literals $x + y \geq 0 \wedge \neg(x \neq z) \wedge y + z = -1 \wedge z > 3t$, which the SMT solver tries to satisfy in the decidable theory: a system of linear inequalities over $\mathbb{Q}$. If satisfiable, so is the original formula.

4. Otherwise (as in our example) conjoin the original formula with the *negation* of this conjunction of literals, a new formula satisfiable iff the original is, and go back to step 1.

Example 8.1. Continuing: step 2 now gives the CNF $p_1 \wedge (\neg p_2 \vee p_3) \wedge p_4 \wedge (\neg p_1 \vee p_2 \vee \neg p_3 \vee \neg p_4)$ and the SAT solver returns e.g. $[1 \leftarrow p_1, 0 \leftarrow p_2, 0 \leftarrow p_3, 1 \leftarrow p_4]$; step 3 gives $x + y \geq 0 \wedge \neg(x \neq z) \wedge \neg(y + z = -1) \wedge z > 3t$, satisfiable in rational linear arithmetic, e.g. by $[0 \leftarrow x, 0 \leftarrow y, 0 \leftarrow z, -\frac{1}{6} \leftarrow t]$, which therefore satisfies the original formula.

If T effectively allows quantifier elimination (dense unbounded orders, rational linear arithmetic, real arithmetic), we have an algorithm giving a quantifier-free $\mathrm{qe}_T(\varphi)$ with $T \models \varphi \Leftrightarrow \mathrm{qe}_T(\varphi)$, so the SMT solver can be applied to $\mathrm{qe}_T(\varphi)$: if $T \models \varphi \Leftrightarrow \psi$ then φ is satisfiable modulo T iff ψ is, with the *same* witnesses.

In general there is no algorithm for Satisfiability modulo T: a closed φ belongs to T iff $\neg\varphi$ is not satisfiable modulo T, so such an algorithm would decide T, yet elementary arithmetic and $\mathrm{Th}(\mathbb{Z}, +, \times)$ are undecidable.

8.2 Modelling examples

8.2.1 The McNugget number

McNuggets are sold in boxes of six, nine or twenty. One cannot buy exactly 3 or 11. We can buy exactly m iff m is a non-negative linear combination of 6, 9 and 20:

$$comb(m) \stackrel{\text{def}}{=} \exists c_1.\exists c_2.\exists c_3.\ m = 6c_1 + 9c_2 + 20c_3$$

with $c_1, c_2, c_3 \in \mathbb{N}$ (we can take rational linear arithmetic as theory, where integers can be distinguished). For large enough m such coefficients exist since 6, 9, 20 are coprime (a combinatorial theorem). The *McNugget number* is the largest m with $\neg comb(m)$; i.e. we look for m with

$$\neg comb(m) \wedge \forall p.(p > m \Rightarrow comb(p)).$$

This is an instance of Satisfiability modulo T, and Z3 correctly answers 43: every number of McNuggets from 44 on can be bought. Here Z3 only produces a valuation, the interpretation being fixed in advance.

8.2.2 Program invariant synthesis

SMT solvers can synthesise loop invariants (relations between program variables that stay true during execution) used to prove programs correct. Consider

```
int i = 0; int j = 1;
while (i < 10) { i = i + 2; j = j + 1; }
assert(j < 10);
```

We want $j < 10$ when the loop exits ($i \geq 10$). One approach computes the set $Reach \subseteq \mathbb{Z}^2$ of reachable (i, j) and checks $i \geq 10 \Rightarrow j < 10$: indeed $Reach =$

$\{(0,1),(2,2),(4,3),(6,4),(8,5),(10,6)\}$ and it does not meet $\mathit{Error} = \{(i,j) \mid i,j \geq 10\}$. But in general *Reach* cannot be computed (e.g. non-termination), and we precisely want to verify without running. Instead we look for a *safe over-approximation*: $\mathit{Approx} \supseteq \mathit{Reach}$ with $\mathit{Approx} \cap \mathit{Error} = \emptyset$. To be sure it over-approximates we require an *inductive invariant*: the initial configuration is in *Approx* and any step from *Approx* stays in *Approx*. (*Reach* is the smallest one; the largest is $\mathit{Safe} = \{(i,j) \mid (i < 10 \wedge 2j \leq i + 8) \vee (i \geq 10 \wedge j < 10)\}$.)

Take the signature $(\{+^{(2)}, (\underline{k}^{(0)})_{k\in\mathbb{Z}}\}, \{<, =, \mathit{Invar}^{(2)}\})$ with $+$, constants, $<$ and $=$ interpreted in $D_I = \mathbb{Z}$ as usual, and *Invar* an *uninterpreted* relation symbol to be found by the solver. The formula is closed, so Z3 returns only an interpretation. It must say:

- loop invariant: $\forall x.\forall y.(\mathit{Invar}(x,y) \wedge x < 10 \Rightarrow \mathit{Invar}(x+2, y+1))$;
- initial condition $i = 0, j = 1$: $\mathit{Invar}(0,1)$;
- safety: $\forall x.\forall y.(\mathit{Invar}(x,y) \wedge (x > 10 \vee x = 10) \Rightarrow y < 10)$.

Z3 answers that the formula is satisfiable and returns an interpretation of *Invar* which is a safe inductive invariant, but in general neither *Reach* nor *Safe*: an intermediate set, due to the solver's internal engine.

Link with constraint programming

An SMT problem is a CP problem: variables with an interpreted domain, constraints written as first-order formulas. Optimisation (minimising m) is done in CP by branch-and-bound instead of the universally quantified formula $\psi(m)$. See Chapter 10.

Chapter 9

First-order sequent calculus

Summary

The first-order sequent calculus aims at proofs of validity, in particular for a sequent made of a single formula. Its rules are

$$\frac{}{\vdash \Gamma, \ell, \overline{\ell}}\ (\text{ax}) \qquad \frac{\vdash \Gamma, \varphi, \psi}{\vdash \Gamma, \varphi \vee \psi}\ (\vee) \qquad \frac{\vdash \Gamma, \varphi \qquad \vdash \Gamma, \psi}{\vdash \Gamma, \varphi \wedge \psi}\ (\wedge)$$

$$\frac{\vdash \Gamma, [y \leftarrow x](\varphi)}{\vdash \Gamma, \forall x.\varphi}\ (\forall)\ (y \notin \mathrm{fv}(\Gamma, \forall x.\varphi)) \qquad \frac{\vdash \Gamma, [t \leftarrow x](\varphi), \exists x.\varphi}{\vdash \Gamma, \exists x.\varphi}\ (\exists)\ (t \in T(\mathcal{F}, X))$$

$$\frac{\vdash \Gamma, \varphi \qquad \vdash \Delta, \overline{\varphi}}{\vdash \Gamma, \Delta}\ (\text{cut})$$

with all formulas in negation normal form. A proof is a tree chaining these rules, with the sequent to prove as its root. The first-order sequent calculus is sound and complete: $\vdash \Gamma$ is provable ($\vdash_{LK} \Gamma$) iff it is valid ($\models \Gamma$). However, unlike propositional logic, no algorithm solves first-order validity (in particular because of rule ($\exists$), which does not have the finite-branch property).
The calculus can be extended with admissible rules: weakening (W), contraction (C) and extended axiom (ax$'$).

We extend the propositional calculus LK_0 to first-order logic, with the same aim of obtaining readable, easily checkable validity certificates. We work with one-sided sequents $\vdash \Gamma$, with Γ a finite multiset of formulas in NNF. Free variables are extended to sequents: $\mathrm{fv}(\Gamma) = \bigcup_{\varphi \in \Gamma} \mathrm{fv}(\varphi)$. The empty sequent is $\vdash \bot$. Recall that $\overline{\varphi}$ denotes $\mathrm{NNF}(\neg\varphi)$.

The axiom, conjunction and disjunction rules are as before, except that the axiom looks for a literal and its negation $\overline{\ell}$ (rather than a proposition).

Rule ($\forall$). It requires $y \notin \mathrm{fv}(\Gamma, \forall x.\varphi)$. This mirrors what one does by hand to prove $\forall x.\varphi$: "Let x be arbitrary. We show φ." (the renaming of x to y avoids problems explained below).

Rule ($\exists$). We introduce any term t while keeping a copy of $\exists x.\varphi$. By hand: "Let $x = t$. We show φ", which takes creativity since the term t may be hard to find.

Rule (cut). It splits the conclusion sequent and introduces a formula and its negation in the two premises. This is what we do when proving a lemma before the theorem: either the lemma is false (impossible), or the theorem is true. We shall see it is not really necessary, but it eases writing some proofs.

Every rule except (cut) has a *principal formula* in its conclusion; (cut) eliminates a *cut formula*.

A sequent is *provable*, $\vdash_{LK} \Gamma$, if it has a derivation.

Example 9.1 (Drinker's formula)**.** The drinker's formula $\exists x.(\neg B(x) \vee \forall y.B(y))$ is provable:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \neg B(x), B(y), \neg B(y), \forall y.B(y), \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\text{ax})}{\vdash \neg B(x), B(y), \neg B(y) \vee \forall y.B(y), \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\vee)}{\vdash \neg B(x), B(y), \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\exists)\text{ with } t = y}{\vdash \neg B(x), \forall y.B(y), \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\forall)}{\vdash \neg B(x) \vee \forall y.B(y), \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\vee)}{\vdash \exists x.(\neg B(x) \vee \forall y.B(y))}\,(\exists)\text{ with } t = x$$

(The first use of $(\exists)$ picks a first witness x; it is later "corrected" by the second witness y, which is why a copy of $\exists x.\varphi$ is retained.)

Remark 9.2. The condition $y \notin \mathrm{fv}(\Gamma)$ in $(\forall)$ is necessary for soundness. Otherwise we could derive $\vdash \forall x.B(x) \vee \forall x.\neg B(x)$ from the axiom $\vdash B(x), \neg B(x)$ by two (incorrect) applications of $(\forall)$. That formula is not valid: with $D_I = \{a, b\}$, $B_I = \{a\}$, we have $I \not\models \forall x.B(x)$ (take $x = b$) and $I \not\models \forall x.\neg B(x)$ (take $x = a$).

Remark 9.3. The condition $y \notin \mathrm{fv}(\forall x.\varphi)$ is also necessary. Otherwise, since $[y \leftarrow x](B(x) \vee \neg B(y)) = B(y) \vee \neg B(y)$, we could derive $\vdash \forall y.\forall x.(B(x) \vee \neg B(y))$ from $\vdash B(y) \vee \neg B(y)$. This formula is not valid: with $D_I = \{a, b\}$, $B_I = \{a\}$, the valuation $[a \leftarrow y, b \leftarrow x]$ falsifies $B(x) \vee \neg B(y)$.

Example 9.4. We show that $\forall x.\exists y.\forall z.(x < y \wedge \neg(x < z \wedge z < y))$ does not belong to $\mathrm{Th}(A_{dense})$. The theory is consistent (it has $(\mathbb{Q}, <)$ as a model), so we show that the density axiom implies the negation of this formula, i.e. that

$$(\forall x.\forall y.\exists z.((x < y) \Rightarrow (x < z \wedge z < y))) \Rightarrow \neg(\forall x.\exists y.\forall z.(x < y \wedge \neg(x < z \wedge z < y)))$$

is provable (hence valid by soundness, Theorem 9.5). Its NNF is

$$(\exists x.\exists y.\forall z.((x < y) \wedge (\neg(x < z) \vee \neg(z < y)))) \vee (\exists x.\forall y.\exists z.(\neg(x < y) \vee (x < z \wedge z < y))).$$

The derivation first applies $(\vee)$, then $(\exists)$ on x (twice, to obtain the witnesses for both disjuncts), $(\forall)$ on y giving a fresh y, $(\exists)$ for z, and so on, until we reach a sequent containing $x < y \wedge (\neg(x < z) \vee \neg(z < y))$ and $\neg(x < y) \vee (x < z \wedge z < y)$; applying $(\forall)$ on z, $(\vee)$, $(\wedge)$ and $(\vee)$ splits into leaves, each closed by (ax) on pairs $x < y, \neg(x < y)$, $x < z, \neg(x < z)$ or $z < y, \neg(z < y)$. (The full derivation is long; see the course slides for the complete tree.)

9.1 Soundness

A sequent $\vdash \Gamma$ is *satisfied* by I under ρ, $I, \rho \models \Gamma$, if there is a *witness* formula $\theta \in \Gamma$ with $I, \rho \models \theta$. It is *valid*, $\models \Gamma$, if $I, \rho \models \Gamma$ for all I, ρ.

Theorem 9.5 (Soundness)**.** *If* $\vdash_{LK} \Gamma$ *then* $\models \Gamma$.

Proof. By induction on a derivation, showing for each rule that valid premises give a valid conclusion. The propositional rules are as in Theorem 4.2. We treat quantifier rules and cut.

$(\forall)$. Assume the premise $\vdash \Gamma, [y \leftarrow x](\varphi)$ valid with $y \notin \mathrm{fv}(\Gamma, \forall x.\varphi)$. Take (I, ρ) arbitrary. For all $e \in D_I$, $I, \rho[e \leftarrow y] \models \Gamma, [y \leftarrow x](\varphi)$, so there is a witness θ_e. If for some e, $\theta_e \in \Gamma$: since $y \notin \mathrm{fv}(\Gamma)$, Property 5.18 gives $I, \rho \models \theta_e$, so $I, \rho \models \Gamma, \forall x.\varphi$. Otherwise, $I, \rho[e \leftarrow y] \models [y \leftarrow x](\varphi)$

for all e; by the substitution lemma, $I, \rho[e \leftarrow y, e \leftarrow x] \models \varphi$ for all e, hence $I, \rho[e \leftarrow y] \models \forall x.\varphi$, and since $y \notin \mathrm{fv}(\forall x.\varphi)$, $I, \rho \models \forall x.\varphi$.

($\exists$). Assume the premise valid; take (I, ρ). If the witness of $I, \rho \models \Gamma, [t \leftarrow x](\varphi), \exists x.\varphi$ is in $\Gamma \cup \{\exists x.\varphi\}$ it is also a witness for the conclusion. If it is $[t \leftarrow x](\varphi)$ then by the substitution lemma $I, \rho[[\![t]\!]_{I,\rho} \leftarrow x] \models \varphi$, hence $I, \rho \models \exists x.\varphi$.

(cut). Assume $\vdash \Gamma, \varphi$ and $\vdash \Delta, \overline{\varphi}$ valid. For (I, ρ) there are witnesses $\theta \in \Gamma \cup \{\varphi\}$ and $\theta' \in \Delta \cup \{\overline{\varphi}\}$. If $\theta \in \Gamma$ or $\theta' \in \Delta$ we are done. Otherwise $I, \rho \models \varphi$ and $I, \rho \models \overline{\varphi}$, which is absurd. □

9.2 Completeness

Completeness (every valid sequent is derivable in LK) is much more technical and is admitted here. It is shown by contraposition: an unprovable sequent $\vdash \Gamma$ has a countermodel, i.e. I, ρ with $I, \rho \not\models \varphi$ for all $\varphi \in \Gamma$.

Theorem 9.6 (Completeness)**.** *If* $\models \Gamma$ *then* $\vdash_{LK} \Gamma$.

The proof is even stronger: it shows that if $\models \Gamma$ there is a derivation of $\vdash \Gamma$ that does not use (cut).

Despite soundness and completeness, the calculus does not give an algorithm for Validity. Even though the rules (except (cut), which is unnecessary) are syntactically invertible, branches are not finite (because of ($\exists$)): proof search may not terminate. This is unavoidable:

Theorem 9.7 (Mostowski, Robinson, Tarski; Church–Turing)**.** *There is no algorithm taking a first-order formula and answering whether it is valid.*

9.3 Admissible rules

The calculus can be enriched with *admissible* rules, which do not change provability (they are not indispensable by completeness of LK, but they are sound; their correctness is proved syntactically by manipulating derivations). We admit the correctness of some useful ones: weakening, contraction and the extended axiom (studied in tutorials for the propositional calculus):

$$\frac{\vdash \Gamma}{\vdash \Gamma, \Delta}\ (\mathrm{W}) \qquad \frac{\vdash \Gamma, \varphi, \varphi}{\vdash \Gamma, \varphi}\ (\mathrm{C}) \qquad \frac{}{\vdash \Gamma, \varphi, \overline{\varphi}}\ (\mathrm{ax}')$$

Part III

Exercise sheets with solutions

Exercise sheet 1 — Propositional logic

Exercise 1. *(1) For each of three trees, say whether it represents a propositional formula and give its concrete syntax. Tree A: a root $\neg$ with two children ($\vee$ and p), the $\vee$ node having children p and $\neg q$. Tree B: root $\vee$ with children $\neg(\wedge(p, q))$ and p. Tree C: root $\wedge$ whose children are the proposition q having its own sub-tree, and a $\neg$ node, with q as an internal node.*

(2) For each expression, say whether it denotes a propositional formula unambiguously, and draw the trees: $(p \wedge q \wedge r) \vee \neg p$ and $r \wedge \neg(p \vee q \wedge \neg r)$.

Solution. (1) Tree A is not a formula: the root $\neg$ should have exactly one child. Tree B is the formula $(\neg(p \wedge q)) \vee p$. Tree C is not a formula: a proposition cannot be an internal node, propositions must be leaves.

(2) The first uses associativity of $\wedge$ to omit parentheses; once fully parenthesised there are two readings, $((p \wedge q) \wedge r) \vee \neg p$ and $(p \wedge (q \wedge r)) \vee \neg p$, hence two (equivalent) trees. The second is unambiguous since negation has priority over conjunction, which has priority over disjunction: $r \wedge \neg(p \vee (q \wedge \neg r))$.

Exercise 2. *Let f be defined by induction on formulas: $f(p) = \emptyset$, $f(\neg\varphi) = \{\neg\varphi\} \cup f(\varphi)$, $f(\varphi \vee \psi) = f(\varphi \wedge \psi) = f(\varphi) \cup f(\psi)$. (1) Compute $f(r \wedge \neg(p \wedge (q \vee \neg r)))$. (2) When is $f(\varphi) = \emptyset$? Prove it by induction.*

Solution. (1) $f(r \wedge \neg(p \wedge (q \vee \neg r))) = f(r) \cup f(\neg(p \wedge (q \vee \neg r))) = \{\neg(p \wedge (q \vee \neg r))\} \cup f(p) \cup f(q \vee \neg r) = \{\neg(p \wedge (q \vee \neg r))\} \cup \{\neg r\}$.

(2) Only the negation rule adds anything, so f returns the subformulas starting with a negation. Claim: $f(\varphi) = \emptyset$ iff φ contains no negation. By induction on φ. If $\varphi = p$: $f(p) = \emptyset$ and p has no negation: both sides true. If $\varphi = \neg\psi$: $\neg\psi \in f(\varphi)$ so $f(\varphi) \neq \emptyset$, and φ contains a negation: both sides false. If $\varphi = \psi \star \psi'$: $f(\varphi) = \emptyset$ iff $f(\psi) = f(\psi') = \emptyset$ iff (induction) neither ψ nor ψ' contains a negation iff φ has none.

Exercise 3. *Find a formula φ with the truth table: $\varphi = 1$ on $(p, q, r) \in \{000, 001, 011, 111\}$ and 0 elsewhere.*

Solution. Take one conjunction per row with value 1: $(\neg p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge q \wedge r) \vee (p \wedge q \wedge r)$. Another: splitting on r, $(r \wedge (\neg p \vee q)) \vee (\neg r \wedge \neg p \wedge \neg q)$.

Exercise 4. *For each formula, say if it is valid and/or satisfiable and give a satisfying interpretation: $q \vee \neg q$, $\neg(p \wedge \neg q)$, $p \Rightarrow p$, $(p \Rightarrow q) \wedge p \wedge \neg q$.*

Solution. $q \vee \neg q$ and $p \Rightarrow p$ are valid. $\neg(p \wedge \neg q)$ is not valid ($[1 \leftarrow p, 0 \leftarrow q]$ falsifies it) but is satisfiable ($[0 \leftarrow p, 1 \leftarrow q]$). $(p \Rightarrow q) \wedge p \wedge \neg q$ is unsatisfiable (truth table: always 0).

Exercise 5. *For each pair, write truth tables and decide whether one is a consequence of the other and whether they are equivalent: (a) $p \Rightarrow q$ and $\neg q \Rightarrow \neg p$; (b) $p \wedge (\neg q \vee r)$ and $(p \wedge \neg q) \vee (p \wedge \neg r)$; (c) $p \Rightarrow q$ and $p \Rightarrow (q \vee r)$.*

Solution. (a) The truth tables coincide ($1, 1, 0, 1$ for $pq = 00, 01, 10, 11$): equivalent (contraposition).

(b) Neither is a consequence of the other: $[1 \leftarrow p, 1 \leftarrow q, 1 \leftarrow r]$ satisfies the first but not the second; $[1 \leftarrow p, 1 \leftarrow q, 0 \leftarrow r]$ satisfies the second but not the first.

(c) $p \Rightarrow (q \vee r)$ is a consequence of $p \Rightarrow q$ (every model of $p \Rightarrow q$ satisfies it), but not conversely ($[1 \leftarrow p, 0 \leftarrow q, 1 \leftarrow r]$). Not equivalent.

Exercise 6. *Let p: it rains; q: Pierre takes his umbrella; r: Pierre is wet. Translate: (1) If it rains Pierre takes his umbrella. (2) If Pierre takes his umbrella, he is not wet. (3) If it does not rain, Pierre does not take his umbrella and is not wet. Show that "Pierre is not wet" is a logical consequence of the three.*

Solution. $\varphi_1 = p \Rightarrow q$, $\varphi_2 = q \Rightarrow \neg r$, $\varphi_3 = \neg p \Rightarrow (\neg q \wedge \neg r)$. A truth table over p, q, r shows that $\mathrm{Sat}(\{\varphi_1, \varphi_2, \varphi_3\}) = \{[0, 0, 0], [1, 1, 0]\}$ (values of p, q, r), and in both $r = 0$, so $\mathrm{Sat}(S) \subseteq \mathrm{Sat}(\neg r)$ and $S \models \neg r$.

Exercise 7. *Let φ, ψ be formulas. (1) Show φ and ψ are both valid iff $\varphi \wedge \psi$ is valid. (2) If $\varphi \vee \psi$ is valid, is φ valid or ψ valid? (3) If φ and ψ are satisfiable, what about $\varphi \vee \psi$ and $\varphi \wedge \psi$?*

Solution. (1) $\varphi \wedge \psi$ valid iff for every I, $I \models \varphi$ and $I \models \psi$ iff (for every I, $I \models \varphi$) and (for every J, $J \models \psi$) iff φ and ψ are valid. (2) No: $p \vee \neg p$ is valid but neither p nor $\neg p$ is. (3) $\varphi \vee \psi$ is satisfiable (a model of φ is a model of the disjunction). $\varphi \wedge \psi$ need not be: p and $\neg p$ are satisfiable but $p \wedge \neg p$ is not.

Exercise 8. *Let $\downarrow$ be the Boolean function with $p \downarrow q = 1$ iff $p = q = 0$ (nor). (1) Express $p \downarrow q$ with $\neg, \vee$ only. (2) Express $\neg p$ with $\downarrow$ only. (3) Express $p \vee q$ with $\downarrow$ only.*

Solution. (1) $p \downarrow q \equiv \neg(p \vee q)$. (2) $p \downarrow p \equiv \neg(p \vee p) \equiv \neg p$. (3) $p \vee q \equiv \neg\neg(p \vee q) \equiv (p \downarrow q) \downarrow (p \downarrow q)$.

Exercise 9. *Let $\mathcal{I}$ be the set of all interpretations. Show: (1) $\mathrm{Sat}(\neg\varphi) = \mathcal{I} \setminus \mathrm{Sat}(\varphi)$; (2) $\mathrm{Sat}(\varphi \vee \psi) = \mathrm{Sat}(\varphi) \cup \mathrm{Sat}(\psi)$; (3) $\mathrm{Sat}(\varphi \wedge \psi) = \mathrm{Sat}(\varphi) \cap \mathrm{Sat}(\psi)$; (4) $\models \varphi \Rightarrow \psi$ iff $\mathrm{Sat}(\varphi) \subseteq \mathrm{Sat}(\psi)$.*

Solution. For every I: (1) $I \in \mathrm{Sat}(\neg\varphi)$ iff $I \models \neg\varphi$ iff $I \not\models \varphi$ iff $I \notin \mathrm{Sat}(\varphi)$. (2) $I \models \varphi \vee \psi$ iff $I \models \varphi$ or $I \models \psi$ iff $I \in \mathrm{Sat}(\varphi) \cup \mathrm{Sat}(\psi)$. (3) Same with "and". (4) $\models \varphi \Rightarrow \psi$ iff for every I, $I \models \neg\varphi \vee \psi$ iff for every I, $I \not\models \varphi$ or $I \models \psi$ iff for every I, $(I \models \varphi \Rightarrow I \models \psi)$ iff $\mathrm{Sat}(\varphi) \subseteq \mathrm{Sat}(\psi)$.

Exercise 10. *Let f be defined inductively by $f(p) = p$, $f(\neg\varphi) = \neg f(\varphi)$, $f(\varphi \vee \psi) = f(\varphi) \vee f(\psi)$, $f(\varphi \wedge \psi) = \neg(\neg f(\varphi) \vee \neg f(\psi))$. What does f do? Prove it by induction.*

Solution. f eliminates $\wedge$ without changing semantics: for every φ, $f(\varphi)$ only uses $\neg, \vee$ and $f(\varphi) \equiv \varphi$. Induction: p is fine. For $\neg\varphi_1$: $f = \neg\psi_1$ with $\psi_1 = f(\varphi_1) \equiv \varphi_1$. For $\varphi_1 \vee \varphi_2$: $\psi_1 \vee \psi_2 \equiv \varphi_1 \vee \varphi_2$. For $\varphi_1 \wedge \varphi_2$: $f = \neg(\neg\psi_1 \vee \neg\psi_2)$ uses only $\neg, \vee$ and $\varphi_1 \wedge \varphi_2 \equiv \neg(\neg\varphi_1 \vee \neg\varphi_2) \equiv \neg(\neg\psi_1 \vee \neg\psi_2)$ (De Morgan, and $\varphi_i \equiv \psi_i$).

Exercise sheet 2 — Transformation of formulas

Exercise 1. *Let $\varphi = (p \wedge q) \vee r$, $\tau_1 = [(s \vee q) \leftarrow p,\ \neg q \leftarrow r]$, $\tau_2 = [(q \wedge p) \leftarrow q]$ and the partial interpretation $I = [1 \leftarrow p, 0 \leftarrow q, 0 \leftarrow r, 0 \leftarrow s]$. (1) Compute $\tau_1(\varphi), \tau_2(\varphi)$. (2) Compute I_{τ_1}, I_{τ_2}. (3) Check $[\![\tau_i(\varphi)]\!]_I = [\![\varphi]\!]_{I_{\tau_i}}$; why was this expected?*

Solution. (1) $\tau_1(\varphi) = ((s \vee q) \wedge q) \vee \neg q$ and $\tau_2(\varphi) = (p \wedge (q \wedge p)) \vee r$. (2) $I_{\tau_1}(p) = [\![s \vee q]\!]_I = 0$, $I_{\tau_1}(q) = 0$, $I_{\tau_1}(r) = [\![\neg q]\!]_I = 1$, $I_{\tau_1}(s) = 0$. $I_{\tau_2}(p) = 1$, $I_{\tau_2}(q) = [\![q \wedge p]\!]_I = 0$, $I_{\tau_2}(r) = 0$, $I_{\tau_2}(s) = 0$. (3) $[\![\tau_1(\varphi)]\!]_I = 1 = [\![(p \wedge q) \vee r]\!]_{I_{\tau_1}}$ and $[\![\tau_2(\varphi)]\!]_I = 0 = [\![\varphi]\!]_{I_{\tau_2}}$. This is exactly the substitution lemma.

Exercise 2. *(1) If $\varphi \equiv \psi$ and τ is a substitution, show $\tau(\varphi) \equiv \tau(\psi)$. (2) Deduce a simple proof of De Morgan's law $\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$.*

Solution. (1) Let I be any interpretation. Since $\varphi \equiv \psi$, $[\![\varphi]\!]_{I_\tau} = [\![\psi]\!]_{I_\tau}$; by the substitution lemma $[\![\tau(\varphi)]\!]_I = [\![\tau(\psi)]\!]_I$. (2) A truth table shows $\neg(p \vee q) \equiv \neg p \wedge \neg q$. Apply (1) with $\tau = [\varphi \leftarrow p, \psi \leftarrow q]$.

Exercise 3. *There are n people and proposition p_i means "person i lies". Model: (1) there is a liar; (2) everybody lies; (3) nobody lies; (4) exactly one liar; (5) at least two liars; (6) exactly two liars.*

Solution.

$$\varphi_1 = \bigvee_i p_i \qquad \varphi_2 = \bigwedge_i p_i \qquad \varphi_3 = \bigwedge_i \neg p_i$$

$$\varphi_4 = \bigvee_i \Big(p_i \wedge \bigwedge_{j \neq i} \neg p_j\Big) \qquad \varphi_5 = \bigvee_{i \neq j} (p_i \wedge p_j) \qquad \varphi_6 = \bigvee_{i \neq j} \Big(p_i \wedge p_j \wedge \bigwedge_{k \neq i,j} \neg p_k\Big)$$

Exercise 4. *Put in negation normal form: $\neg(p \vee \neg(q \wedge \neg r))$ and $(p \Rightarrow q) \Leftrightarrow (\neg q \Rightarrow \neg p)$.*

Solution. $\mathrm{NNF}(\neg(p \vee \neg(q \wedge \neg r))) = \neg p \wedge \mathrm{NNF}(\neg\neg(q \wedge \neg r)) = \neg p \wedge q \wedge \neg r$. For the second, first remove $\Rightarrow, \Leftrightarrow$: $(\neg(\neg p \vee q) \vee (q \vee \neg p)) \wedge (\neg(q \vee \neg p) \vee (\neg p \vee q))$, whose NNF is $((p \wedge \neg q) \vee (q \vee \neg p)) \wedge ((\neg q \wedge p) \vee (\neg p \vee q))$.

Exercise 5. *Put $(p_1 \wedge q_1) \vee (p_2 \wedge q_2) \vee (p_3 \wedge q_3)$ in CNF. How many clauses has its clausal form? Generalise to a formula with $2n$ propositions and 2^n clauses.*

Solution. It is already in NNF; distributing repeatedly gives the conjunction of the 8 clauses $\{r_1, r_2, r_3\}$ with $r_i \in \{p_i, q_i\}$: $(p_1 \vee p_2 \vee p_3) \wedge (p_1 \vee p_2 \vee q_3) \wedge \cdots \wedge (q_1 \vee q_2 \vee q_3)$, hence $8 = 2^3$ clauses.

In general $\varphi_n = (p_1 \wedge q_1) \vee \cdots \vee (p_n \wedge q_n)$ has $2n$ propositions and its clausal form consists of all clauses $\{r_1, \ldots, r_n\}$ with $r_i \in \{p_i, q_i\}$: 2^n clauses. So CNF conversion is exponential in the worst case.

Exercise 6 (Santa Claus)**.** *Santa has N toys $J_1, \ldots, J_N$ to give to M children $E_1, \ldots, E_M$, each with a wish list. A child is happy if he gets at least one toy from his list. All toys must be given. (1) For 2 children and 3 toys, child 1 wishing the first two toys and child 2 the last two, write a formula whose models are all the ways of distributing the toys; give a model. (2) Model the general case and describe its clausal form. (3) Add the constraint that a toy is given to a child only if it is on his list; show an example where the new formula is unsatisfiable while the previous one was satisfiable.*

Solution. Use propositions $p_{i,k}$: "toy J_i is given to child E_k".

(1) $(p_{1,1} \vee p_{2,1}) \wedge (p_{2,2} \vee p_{3,2})$ (each child gets a toy from his list) $\wedge (p_{1,1} \vee p_{1,2}) \wedge (p_{2,1} \vee p_{2,2}) \wedge (p_{3,1} \vee p_{3,2})$ (each toy is given at least once) $\wedge (\neg p_{1,1} \vee \neg p_{1,2}) \wedge (\neg p_{2,1} \vee \neg p_{2,2}) \wedge (\neg p_{3,1} \vee \neg p_{3,2})$ (no toy is given twice). A model: toys 1 and 2 to child 1, toy 3 to child 2.

(2) Let L be the set of pairs (i,k) such that J_i is in the list of E_k. The clauses are: $\{\{p_{i,k} \mid (i,k) \in L\} \mid k \leq M\}$; $\{\{p_{i,k} \mid k \leq M\} \mid i \leq N\}$; $\{\{\neg p_{i,k}, \neg p_{i,k'}\} \mid k \neq k', i \leq N\}$.

(3) Add the unit clauses $\{\neg p_{i,k}\}$ for $(i,k) \notin L$. With a single child, two toys and only toy 1 in his list, we get $\{\{p_{1,1}\}, \{p_{1,1}\}, \{p_{1,2}\}, \{\neg p_{1,2}\}\}$ which is unsatisfiable (toy 2 must be given but cannot be), whereas without the new constraint giving both toys to the child was a model.

Exercise 7 (Linear-time equisatisfiable translation)**.** *Let φ be in NNF. For each non-literal subformula φ' introduce a fresh proposition $q_{\varphi'}$; for a literal subformula $q_{\varphi'} = \varphi'$. For non-literal φ' define $\psi_{\varphi'} = q_{\varphi_1} \vee q_{\varphi_2}$ if $\varphi' = \varphi_1 \vee \varphi_2$ and $q_{\varphi_1} \wedge q_{\varphi_2}$ if $\varphi' = \varphi_1 \wedge \varphi_2$. The desired formula is $\psi = q_\varphi \wedge \bigwedge_{\varphi'} (q_{\varphi'} \Rightarrow \psi_{\varphi'})$. (1) NNF of Peirce's law. (2) Apply the algorithm. (3) Why can ψ be put in 3-clausal form? Apply it. (4) Check Peirce's law is valid; give an interpretation satisfying the ψ of (3) and one that does not. (5) Show ψ is equisatisfiable with φ.*

Solution. (1) $\neg(\neg(\neg p \vee q) \vee p) \vee p$ has NNF $((\neg p \vee q) \wedge \neg p) \vee p$.

(2) Let $\varphi_4 = \neg p \vee q$, $\varphi_5 = \varphi_4 \wedge \neg p$ and $\varphi = \varphi_5 \vee p$ with variables q_φ, q_4, q_5:

$$\psi = q_\varphi \wedge (q_\varphi \Rightarrow (q_5 \vee p)) \wedge (q_4 \Rightarrow (\neg p \vee q)) \wedge (q_5 \Rightarrow (q_4 \wedge \neg p)).$$

(3) Each implication is $q \Rightarrow (a \vee b)$, i.e. the clause $\{\neg q, a, b\}$, or $q \Rightarrow (a \wedge b)$, i.e. the two clauses $\{\neg q, a\}, \{\neg q, b\}$; with the clause $\{q_\varphi\}$ all clauses have at most 3 literals. For Peirce: $\{\{q_\varphi\}, \{\neg q_\varphi, q_5, p\}, \{\neg q_4, \neg p, q\}, \{\neg q_5, q_4\}, \{\neg q_5, \neg p\}\}$.

(4) The truth table of Peirce's law gives 1 everywhere. $I_1 = [1 \leftarrow q_\varphi, 1 \leftarrow p, 0 \leftarrow q_5, 0 \leftarrow q_4,\ q$ arbitrary] satisfies ψ; I_2 with $0 \leftarrow q_\varphi$ does not (first clause).

(5) ($\Rightarrow$) If $I \models \varphi$, extend I to I' by $I'(q_{\varphi'}) = [\![\varphi']\!]_I$. Then $I' \models q_\varphi$, and for each non-literal φ', if $I' \models q_{\varphi'}$: for $\varphi' = \varphi_1 \vee \varphi_2$ one of φ_i holds in I so $I' \models q_{\varphi_1} \vee q_{\varphi_2}$; for $\wedge$ both hold. So $I' \models \psi$. ($\Leftarrow$) If $I \models \psi$, show by induction on subformulas that $I \models q_{\varphi'}$ implies $I \models \varphi'$: for literals it is trivial; for $\varphi' = \varphi_1 \vee \varphi_2$, $I \models q_{\varphi'} \Rightarrow \psi_{\varphi'}$ gives $I \models q_{\varphi_1}$ or q_{φ_2}, so by induction $I \models \varphi_1$ or φ_2; similarly for $\wedge$. Since $I \models q_\varphi$, $I \models \varphi$.

Exercise 8. *(1) Put $(p_1 \vee \neg p_2 \vee p_3) \wedge q$ in DNF. (2) Give an algorithm to test satisfiability of a DNF. (3) Give an algorithm to test validity of a CNF.*

Solution. (1) $(p_1 \wedge q) \vee (\neg p_2 \wedge q) \vee (p_3 \wedge q)$. (2) A DNF $\bigvee_i \bigwedge_j \ell_{i,j}$ is satisfiable iff one conjunction does not contain both a proposition and its negation: scan the conjunctions looking for one with

no opposite literals. (3) A formula is valid iff its negation is unsatisfiable. The negation of a CNF is turned into a DNF by pushing negation to the leaves (De Morgan); apply (2) and answer the opposite.

Exercise sheet 3 — Model search

Reminder. DPLL decides satisfiability of a set of clauses S by rewriting with

$$\begin{aligned} S \cup \{\{\ell\}\} &\to_{\text{dpll}} S[\top \leftarrow \ell] && && \text{(unit)} \\ S &\to_{\text{dpll}} S[\top \leftarrow \ell] && \text{where } \ell \text{ is pure in } S && \text{(pure)} \\ S &\to_{\text{dpll}} S[\top \leftarrow p] && \text{where } p \in \text{prop}(S) && \text{(split}_p\text{)} \\ S &\to_{\text{dpll}} S[\top \leftarrow \neg p] && \text{where } p \in \text{prop}(S) && \text{(split}_{\neg p}\text{)} \end{aligned}$$

Rules (unit) and (pure) are invertible and have priority; the search explores a tree through the split rules. The system terminates, is correct ($S \to^*_{\text{dpll}} \emptyset$ implies S satisfiable) and complete.

Exercise 1. *Apply DPLL to decide satisfiability (and give a model if any): (1)* $\{\{p,q,r\},\{\neg p,q,r\},\{r\},\{p,\neg q,\neg r\},\{\neg p,\neg q,\neg r\},\{\neg r\}\}$*;*

(2) $\{\{p,\neg q,r\},\{p,\neg r\},\{q,r\},\{p,\neg q\}\}$*;* *(3)* $\{\{p,q,r\},\{p,\neg q,\neg r\},\{p,\neg s\},\{\neg q,\neg r,\neg s\},\{\neg p,\neg q,r\},\{t,u\},\{t,\neg u\},\{q,\neg t\},\{\neg r,\neg t\}\}$.

Solution. (1) The literal r is unit; one (unit) step gives $\{\{p,\neg q\},\{\neg p,\neg q\},\bot\}$, which contains $\bot$. As (unit) is invertible, the set is unsatisfiable.

(2) p is pure: $S \to \{\{q,r\}\}$. Then q is pure: $\{\{q,r\}\} \to \emptyset$. Hence the set is satisfiable, and $[1 \leftarrow p, 1 \leftarrow q]$ is a model.

(3) $\neg s$ is pure, removing $\{p,\neg s\}$ and $\{\neg q,\neg r,\neg s\}$:

$$\{\{p,q,r\},\{p,\neg q,\neg r\},\{\neg p,\neg q,r\},\{t,u\},\{t,\neg u\},\{q,\neg t\},\{\neg r,\neg t\}\}.$$

Split on $\neg p$: $\{\{q,r\},\{\neg q,\neg r\},\{t,u\},\{t,\neg u\},\{q,\neg t\},\{\neg r,\neg t\}\}$. Split on $\neg q$ (first branch): $\{\{r\},\{t,u\},\{t,\neg u\},\{\neg t\},\{\neg r,\neg t\}\}$; unit r gives $\{\{t,u\},\{t,\neg u\},\{\neg t\}\}$, unit $\neg t$ gives $\{\{u\},\{\neg u\}\}$, unit u gives $\{\bot\}$: this branch fails. Split on q (second branch): $\{\{\neg r\},\{t,u\},\{t,\neg u\},\{\neg r,\neg t\}\}$; unit $\neg r$ gives $\{\{t,u\},\{t,\neg u\}\}$, then t is pure, giving $\emptyset$. So the set is satisfiable with model $[0 \leftarrow s, 0 \leftarrow p, 1 \leftarrow q, 0 \leftarrow r, 1 \leftarrow t]$.

Exercise 2 (Pigeonhole principle)**.** *The principle states that k pigeons can be put in n holes with each pigeon alone in its hole iff $k \leq n$. (1) Give propositions modelling the placement. (2) A CNF stating each pigeon is in at least one hole. (3) A CNF stating each hole contains at most one pigeon. (4) Use DPLL to show that two pigeons fit in two holes (ignore the constraint that a pigeon is not in two holes). (5) Use DPLL to show that three pigeons do not fit in two holes.*

Solution. (1) A proposition $p_{i,j}$ for $1 \leq i \leq k$, $1 \leq j \leq n$: "pigeon i is in hole j". (2) $\bigwedge_{1\leq i\leq k} \bigvee_{1\leq j\leq n} p_{i,j}$. (3) $\bigwedge_{1\leq j\leq n,\ 1\leq i\neq i'\leq k} (\neg p_{i,j} \vee \neg p_{i',j})$.

(4) The clauses are $\{\{p_{1,1}, p_{1,2}\}, \{p_{2,1}, p_{2,2}\}, \{\neg p_{1,1}, \neg p_{2,1}\}, \{\neg p_{1,2}, \neg p_{2,2}\}\}$. Split on $\neg p_{1,1}$: $\{\{p_{1,2}\}, \{p_{2,1}, p_{2,2}\}, \{\neg p_{1,2}, \neg p_{2,2}\}\}$; unit $p_{1,2}$: $\{\{p_{2,1}, p_{2,2}\}, \{\neg p_{2,2}\}\}$; unit $\neg p_{2,2}$: $\{\{p_{2,1}\}\}$; unit $p_{2,1}$: $\emptyset$. Model: $[0 \leftarrow p_{1,1}, 1 \leftarrow p_{1,2}, 0 \leftarrow p_{2,2}, 1 \leftarrow p_{2,1}]$, i.e. pigeon 1 in hole 2, pigeon 2 in hole 1.

(5) The clause set is $\{\{p_{1,1}, p_{1,2}\}, \{p_{2,1}, p_{2,2}\}, \{p_{3,1}, p_{3,2}\}\}$ together with $\{\neg p_{i,j}, \neg p_{i',j}\}$ for $j \in \{1, 2\}$, $i < i' \leq 3$. *Branch* $\neg p_{1,1}$: unit $p_{1,2}$, then units $\neg p_{2,2}$, $\neg p_{3,2}$, then $p_{2,1}$ and $p_{3,1}$ are forced by their clauses but $\{\neg p_{2,1}, \neg p_{3,1}\}$ yields the empty clause (more precisely, unit propagation produces $\{\{p_{3,1}, p_{3,2}\}, \{\neg p_{3,1}\}, \{\neg p_{3,2}\}\} \to \{\{p_{3,2}\}, \{\neg p_{3,2}\}\} \to \{\bot\}$). *Branch* $p_{1,1}$: units $\neg p_{2,1}$, $\neg p_{3,1}$, then $p_{2,2}$, $p_{3,2}$ are forced and $\{\neg p_{2,2}, \neg p_{3,2}\}$ yields $\bot$. Every branch ends with a set containing $\bot$, so the set is unsatisfiable.

Exercise 3 (MAX-SAT)**.** *MAX-SAT takes a set of clauses φ and asks how many clauses can be satisfied simultaneously. (1) Modify the simplification algorithm* `satisfiable(S)` *to solve it. (2) Incorporate rule (pure). (3) Can rule (unit) be incorporated? (4) Illustrate on $\{\{p, q, r\}, \{\neg p, q, r\}, \{r\}, \{p, \neg q, \neg r\}, \{\neg p, \neg q, \neg r\}, \{\neg r\}\}$ and on $\{\{q, r\}, \{q, \neg r\}, \{p, r\}, \{\neg p, q\}, \{p, q, r\}, \{\neg q\}\}$.*

Solution. (1) Simplify until reaching $\emptyset$ or $\{\bot\}$ (value 0), and at each simplification add the number of clauses thereby satisfied:

```
function maxsat(S):
    if S = {} or S = {bottom}: return 0
    choose p in prop(S)
    k1 = maxsat(S[T <- p])  + #clauses of S containing p
    k0 = maxsat(S[T <- ~p]) + #clauses of S containing ~p
    return max(k1, k0)
```

(2) If ℓ is pure it is always best to set it true: satisfying it cannot falsify any clause. So add

```
    if S has a pure literal l:
        return maxsat(S[T <- l]) + #clauses of S containing l
```

before the branching.

(3) Not directly: satisfying a unit clause may falsify many others. In $\{\{p\}, \{\neg p, q, r\}, \{\neg p, \neg q, r\}, \{\neg p, q, \neg r\}, \{\neg p, \neg q, \neg r\}\}$, setting $p = 1$ satisfies only the first clause (the remaining $\{q, r\}, \{\neg q, r\}, \{q, \neg r\}, \{\neg q, \neg r\}$ cannot all hold), whereas $p = 0$ satisfies 4 clauses.

(4) For the first formula, branching on $\neg r$ satisfies the 3 clauses containing $\neg r$ and leaves $\{\{p, q\}, \{\neg p, q\}, \bot\}$, of value 2 ($q$ is pure and satisfies both, then $\{\bot\}$ gives 0): total $3 + 2 = 5$. Branching on r is symmetric: $3 + 2 = 5$. The answer is 5 out of 6.

For the second formula (clauses counted with multiplicity), branching on $\neg q$ satisfies $\{\neg q\}$ and leaves $\{r\}, \{\neg r\}, \{p, r\}, \{p, r\}, \{\neg p\}$, of value 4 (take $r = 1, p = 0$): total $1 + 4 = 5$. Branching on q satisfies the 4 clauses containing q and leaves $\{\{p, r\}, \bot\}$, of value 1 (p is pure): total 5. The answer is 5 out of 6, e.g. with $p = 0, q = 0, r = 1$.

Exercise 4. *Show that $\to_{\text{dpll}}$ terminates, and is correct and complete (Theorem 3.13).*

Solution. *Termination.* Each rule removes a proposition from the formula, so there are at most $|\text{prop}(S)|$ steps, as for search by simplification.

Correctness. Search by simplification is correct, so it is enough to show that if $S \to S'$ by (unit) or (pure) and $I \models S'$ then S is satisfiable. For (unit), $S = S'' \cup \{\{\ell\}\}$, $S' = S''[\top \leftarrow \ell]$:

$I[1 \leftarrow \ell] \models S''$ by Property 3.8 and it satisfies $\{\ell\}$. For (pure), $S' = S[\top \leftarrow \ell]$ and $I[1 \leftarrow \ell] \models S$ by Property 3.8.

Completeness. If S is satisfiable then $S \to^*_{\text{simpl}} \emptyset$ by completeness of simplification; this is a $\to_{\text{dpll}}$ derivation using only the split rules.

Exercise 5 (Compactness and graph colouring)**.** *A graph is k-colourable if its vertices can be coloured with k colours so that adjacent vertices differ. The four colour theorem states that every finite planar graph is 4-colourable. We prove it for infinite graphs. For a planar graph $G = (V, E)$, use propositions v_i, $v \in V$, $i \in \{0, 1, 2, 3\}$: "v has colour i". (1) Write C_v: v has exactly one colour. (2) Write $A_{u,v}$: u and v have different colours. (3) For finite G, a formula φ_G satisfiable iff G is 4-colourable. (4) For infinite G, a set S_G satisfiable iff G is 4-colourable. (5) Show S_G is equivalent to $S'_G = \{\varphi_{G'} \mid G' \text{ finite subgraph of } G\}$. (6) Deduce that every infinite planar graph is 4-colourable.*

Solution. (1) $C_v = \left(\bigvee_{i=0}^{3} v_i\right) \wedge \neg \bigvee_{0 \le i < j \le 3} (v_i \wedge v_j)$. (2) $A_{u,v} = \bigvee_{0 \le i \ne j \le 3} (u_i \wedge v_j)$. (3) $\varphi_G = \bigwedge_{v \in V} C_v \wedge \bigwedge_{(u,v) \in E} A_{u,v}$. A model gives a colouring: the colour of v is the unique i with v_i true. (4) One cannot write an infinite conjunction, so take $S_G = \{C_v \mid v \in V\} \cup \{A_{u,v} \mid (u, v) \in E\}$. (5) If $I \models S_G$, it corresponds to a 4-colouring of G, whose restriction colours every subgraph, so $I \models \varphi_{G'}$ for every finite subgraph G'. Conversely, if $I \models S'_G$: for C_v take a finite subgraph containing v; for $A_{u,v}$ take a finite subgraph containing the edge (u, v); then $I \models \varphi_{G'}$ gives $I \models C_v$, resp. $A_{u,v}$. So $I \models S_G$. (6) By (5) we must show S'_G satisfiable; by compactness it suffices to treat a finite subset $\{\varphi_{G_1}, \ldots, \varphi_{G_m}\}$ of S'_G. The union G' of the G_i is a finite subgraph of G; a subgraph of a planar graph is planar, so G' is finite planar and hence 4-colourable by the four colour theorem. Each $G_i \subseteq G'$ is then 4-colourable, so the finite subset is satisfiable. Therefore S'_G, hence S_G, is satisfiable, and G is 4-colourable.

Exercise sheet 4 — Sequent calculus

Reminder. A sequent $\vdash \Gamma$ is valid, written $\models \Gamma$, if every interpretation satisfies at least one formula of Γ. With formulas in NNF the rules are

$$\frac{}{\vdash \Gamma, p, \neg p}\,(\text{ax}) \qquad \frac{\vdash \Gamma, \varphi, \psi}{\vdash \Gamma, \varphi \vee \psi}\,(\vee) \qquad \frac{\vdash \Gamma, \varphi \qquad \vdash \Gamma, \psi}{\vdash \Gamma, \varphi \wedge \psi}\,(\wedge)$$

The calculus is sound and complete: $\vdash_{\mathrm{LK}_0} \Gamma$ iff $\models \Gamma$. When there is a choice, try (ax), then $(\vee)$, then $(\wedge)$.

Exercise 1. *For each formula, put it in NNF and give a sequent proof of validity or a counter-model (found from a failed proof): (1) $p \vee \neg p$; (2) $(p \Rightarrow q) \Rightarrow p$; (3) $p \Rightarrow (q \Rightarrow p)$; (4) $(p \Rightarrow q) \Rightarrow (q \Rightarrow p)$; (5) $(p \wedge (q \vee r)) \Rightarrow ((p \wedge q) \vee r)$.*

Solution. (1) Valid:

$$\dfrac{\dfrac{}{\vdash p, \neg p}\,(\text{ax})}{\vdash p \vee \neg p}\,(\vee)$$

(2) The NNF is $(p \wedge \neg q) \vee p$. Applying $(\vee)$ then $(\wedge)$ gives the sequents $\vdash p, p$ and $\vdash \neg q, p$, to which no rule applies. The sequent $\vdash p, p$ gives the counter-model $[0 \leftarrow p]$ (which falsifies all its formulas), hence of $(p \Rightarrow q) \Rightarrow p$: the formula is not valid.

(3) The NNF is $\neg p \vee (\neg q \vee p)$; valid:

$$\dfrac{\dfrac{\dfrac{}{\vdash \neg p, \neg q, p}\,(\text{ax})}{\vdash \neg p, \neg q \vee p}\,(\vee)}{\vdash \neg p \vee (\neg q \vee p)}\,(\vee)$$

(4) The NNF is $(p \wedge \neg q) \vee (\neg q \vee p)$. After two $(\vee)$ steps and one $(\wedge)$ we get $\vdash p, \neg q, p$ and $\vdash \neg q, \neg q, p$. The first is a failure; it gives the counter-model $[0 \leftarrow p, 1 \leftarrow q]$. Not valid.

(5) The NNF is $(\neg p \vee (\neg q \wedge \neg r)) \vee ((p \wedge q) \vee r)$; valid:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \neg p, \neg q, p, r}\,(\text{ax}) \quad \dfrac{}{\vdash \neg p, \neg q, q, r}\,(\text{ax})}{\vdash \neg p, \neg q, p \wedge q, r}\,(\wedge) \quad \dfrac{}{\vdash \neg p, \neg r, p \wedge q, r}\,(\text{ax})}{\vdash \neg p, \neg q \wedge \neg r, p \wedge q, r}\,(\wedge)}{\vdash \neg p, \neg q \wedge \neg r, (p \wedge q) \vee r}\,(\vee)}{\vdash \neg p \vee (\neg q \wedge \neg r), (p \wedge q) \vee r}\,(\vee)}{\vdash (\neg p \vee (\neg q \wedge \neg r)) \vee ((p \wedge q) \vee r)}\,(\vee)$$

Exercise 2. *Consider the formula of TD3, Exercise 2, question 4 (two pigeons a, b, two holes $1, 2$): $\varphi \stackrel{\text{def}}{=} (a_1 \vee a_2) \wedge (b_1 \vee b_2) \wedge (\neg a_1 \vee \neg b_1) \wedge (\neg a_2 \vee \neg b_2)$. (1) Which parts express that each pigeon is in a hole, and that a hole holds at most one pigeon? (2) Write $\neg\varphi$ in NNF. (3) Explain why $\neg\varphi$ is not provable without searching for a proof. (4) Find a failing branch in the proof search for $\vdash \neg\varphi$. (5) Use semantic invertibility to build an interpretation satisfying φ.*

Solution. (1) $a_1 \vee a_2$ and $b_1 \vee b_2$ say each pigeon is in some hole; $\neg a_1 \vee \neg b_1$ and $\neg a_2 \vee \neg b_2$ say holes 1 and 2 do not contain both pigeons.

(2) $\mathrm{NNF}(\neg\varphi) = (\neg a_1 \wedge \neg a_2) \vee (\neg b_1 \wedge \neg b_2) \vee (a_1 \wedge b_1) \vee (a_2 \wedge b_2)$.

(3) By soundness, if $\neg\varphi$ were provable it would be valid. But $[1 \leftarrow a_1, 0 \leftarrow b_1, 0 \leftarrow a_2, 1 \leftarrow b_2]$ satisfies none of the four conjunctions, so $\neg\varphi$ is not valid, hence not provable.

(4) Apply $(\vee)$ three times then $(\wedge)$ four times, choosing the conjuncts to keep:

$$\vdash \neg a_1 \wedge \neg a_2,\ \neg b_1 \wedge \neg b_2,\ a_1 \wedge b_1,\ a_2 \wedge b_2$$
$$\vdash \neg a_1,\ \neg b_2,\ b_1,\ a_2,$$

a sequent with four literals on distinct variables, to which no rule applies.

(5) The failing sequent $\vdash \neg a_1, \neg b_2, b_1, a_2$ gives the interpretation $[1 \leftarrow a_1, 1 \leftarrow b_2, 0 \leftarrow b_1, 0 \leftarrow a_2]$ falsifying all its formulas. By semantic invertibility of the rules it also falsifies the root, so it is a counter-model of $\neg\varphi$, i.e. a model of φ: pigeon a in hole 1 and pigeon b in hole 2.

Exercise 3. *Show that every formula appearing in a proof of $\vdash \Gamma$ is a subformula of a formula of Γ (a theorem contains what is needed for its proof). Reason by induction on the proof tree.*

Solution. *Axiom.* The tree is reduced to its root, whose formulas are those of Γ.

Rule $(\vee)$. $\Gamma = \Gamma', \varphi \vee \psi$ and the premise is $\vdash \Gamma', \varphi, \psi$. The formulas of Γ' are in Γ, and φ, ψ are subformulas of $\varphi \vee \psi \in \Gamma$. By induction every formula of the subproof is a subformula of Γ', φ, ψ, hence of Γ.

Rule $(\wedge)$. $\Gamma = \Gamma', \varphi \wedge \psi$ with premises $\vdash \Gamma', \varphi$ and $\vdash \Gamma', \psi$. The same argument applies to each subtree.

Exercise 4 (Cut)**.** *For a formula φ write $\overline{\varphi}$ for the NNF of $\neg\varphi$. The cut rule is*

$$\frac{\vdash \Gamma, \varphi \qquad \vdash \Delta, \overline{\varphi}}{\vdash \Gamma, \Delta}\ (cut)$$

(1) Show that adding it preserves soundness. (2) Is cut syntactically invertible? (3) Using cut, show that if $\vdash \Gamma, p \wedge \neg p$ is provable then $\vdash \Gamma, \varphi$ is provable for any φ. (4) Does the result of Exercise 3 still hold with cut?

Solution. (1) Suppose both premises valid and let I be an interpretation. There are witnesses $\theta \in \Gamma \cup \{\varphi\}$ and $\theta' \in \Delta \cup \{\overline{\varphi}\}$ satisfied by I. If $\theta \in \Gamma$ or $\theta' \in \Delta$ we have a witness for $\vdash \Gamma, \Delta$. Otherwise $I \models \varphi$ and $I \models \overline{\varphi} \equiv \neg\varphi$, which is impossible. So the conclusion is valid.

(2) No. The sequent $\vdash p \vee \neg p$ is provable and is the conclusion of

$$\frac{\vdash p \vee \neg p,\ q \wedge \neg q \qquad \vdash q \wedge \neg q}{\vdash p \vee \neg p}\ (\text{cut})$$

(here the cut formula is $\overline{q \wedge \neg q} = \neg q \vee q$, whose complement is $q \wedge \neg q$), but the premise $\vdash q \wedge \neg q$ is not valid, hence not provable.

(3) Note $\overline{p \wedge \neg p} = \neg p \vee p$. Given a derivation π of $\vdash \Gamma, p \wedge \neg p$:

$$\dfrac{\dfrac{\dfrac{}{\vdash \varphi, \neg p, p}\,(\mathrm{ax})}{\vdash \varphi, \neg p \vee p}\,(\vee) \qquad \begin{matrix}\pi \\ \vdash \Gamma, p \wedge \neg p\end{matrix}}{\vdash \Gamma, \varphi}\,(\mathrm{cut})$$

(4) No. The cut can introduce arbitrary formulas. For example $\vdash p \vee \neg p$ has the proof

$$\dfrac{\dfrac{\dfrac{}{\vdash p, \neg p, \neg q \wedge q}\,(\mathrm{ax})}{\vdash p \vee \neg p, \neg q \wedge q}\,(\vee) \qquad \dfrac{\dfrac{}{\vdash q, \neg q}\,(\mathrm{ax})}{\vdash q \vee \neg q}\,(\vee)}{\vdash p \vee \neg p}\,(\mathrm{cut})$$

which contains q, not a subformula of $p \vee \neg p$.

Exercise 5. *Using the two-sided calculus from the end of the chapter (no NNF needed), prove* $(p \Rightarrow q) \Rightarrow (\neg q \Rightarrow \neg p)$.

Solution.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{q, p \vdash p}\,(\mathrm{ax})}{q \vdash p, \neg p}\,(\neg_r) \qquad \dfrac{\dfrac{}{q \vdash q, \neg p}\,(\mathrm{ax})}{q, \neg q \vdash \neg p}\,(\neg_l)}{p \Rightarrow q, \neg q \vdash \neg p}\,(\Rightarrow_l)}{p \Rightarrow q \vdash \neg q \Rightarrow \neg p}\,(\Rightarrow_r)}{\vdash (p \Rightarrow q) \Rightarrow (\neg q \Rightarrow \neg p)}\,(\Rightarrow_r)$$

Exercise 6. *With the two-sided calculus, show that* $(r \Rightarrow p) \Rightarrow (r \Rightarrow q)$ *is a logical consequence of* $p \Rightarrow q$.

Solution. We prove the sequent $p \Rightarrow q \vdash (r \Rightarrow p) \Rightarrow (r \Rightarrow q)$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{r \vdash r, p, q}\,(\mathrm{ax}) \qquad \dfrac{}{p, r \vdash p, q}\,(\mathrm{ax})}{r \Rightarrow p, r \vdash p, q}\,(\Rightarrow_l) \qquad \dfrac{}{q, r \Rightarrow p, r \vdash q}\,(\mathrm{ax})}{p \Rightarrow q, r \Rightarrow p, r \vdash q}\,(\Rightarrow_l)}{p \Rightarrow q, r \Rightarrow p \vdash r \Rightarrow q}\,(\Rightarrow_r)}{p \Rightarrow q \vdash (r \Rightarrow p) \Rightarrow (r \Rightarrow q)}\,(\Rightarrow_r)$$

Exercise sheet 5 — First-order logic

Reminder. A first-order signature is a pair $(\mathcal{F}, \mathcal{P})$ of function and relation symbols with arities; $f^{(m)}$ and $R^{(m)}$ denote arity m. Terms are $t ::= x \mid f(t_1, \ldots, t_m)$, atoms $R(t_1, \ldots, t_m)$, formulas $\varphi, \psi ::= \alpha \mid \neg\varphi \mid \varphi \vee \psi \mid \exists x.\varphi$, with $\forall x.\varphi = \neg\exists x.\neg\varphi$. An interpretation I consists of a non-empty domain D_I, a function $f_I : D_I^m \to D_I$ per $f \in \mathcal{F}_m$ and a relation $R_I \subseteq D_I^m$ per $R \in \mathcal{P}_m$; a valuation $\rho : X \to D_I$ assigns variables. I is a model of φ if $I, \rho \models \varphi$ for every ρ; φ is valid if every I is a model; satisfiable if some (I, ρ) satisfies it.

Exercise 1. *Let $\mathcal{F} = \{a^{(0)}, m^{(1)}, p^{(1)}\}$ and $\mathcal{P} = \{B^{(1)}, G^{(2)}\}$, intended on Anne and her ancestors: $B(x)$ "x has blue eyes", $G(x, y)$ "x is taller than y", $m(x)$ the mother of x, $p(x)$ the father of x, a Anne. (1) For each expression decide whether it is a term, an atomic formula, a formula or ill-formed: a. $m(a)$; b. $G(a, x)$; c. $m(p(x))$; d. $p(x)$; e. $m(x, a)$; f. $G(m(x), a)$; g. $G(p(z))$; h. $m(G(x, y))$; i. $G(x, y \vee m(a))$; j. $G(x, G(y, z))$; k. $m(x) \vee p(y)$; l. $(\exists x.m(x)) \vee p(x)$; m. $\exists x.(G(x, m(x)) \wedge p(x))$; n. $\neg\exists x.(G(x, m(x)) \wedge G(x, p(x)))$; o. $\exists y.(G(x, m(a)) \vee G(x, y))$; p. $\exists x.\forall y.G(x, y)$; q. $\forall x.(G(y, x) \Rightarrow \exists y.G(z, y))$; r. $\exists y.(m(G(x, a)) \vee G(x, y))$. (2) For the well-formed formulas draw the tree and give free and bound variables.*

Solution. (1) a, c, d are terms; b and f are atomic formulas; n, o, p, q are (non-atomic) formulas. The others are ill-formed: e (m is unary), g (G is binary), h and j (G is a relation symbol, not a function symbol, so it cannot appear inside a term), i ($\vee$ is not a function symbol, so $y \vee m(a)$ is not a term), k, l, m, r (m and p are function symbols, not relations, so $m(x)$ is not a formula).

(2) (Syntax trees omitted; they are the usual trees with quantifier nodes above the connectives.) The free variables fv and bound variables bv are:

formula	fv	bv
b. $G(a, x)$	$\{x\}$	$\emptyset$
f. $G(m(x), a)$	$\{x\}$	$\emptyset$
n. $\neg\exists x.(G(x, m(x)) \wedge G(x, p(x)))$	$\emptyset$	$\{x\}$
o. $\exists y.(G(x, m(a)) \vee G(x, y))$	$\{x\}$	$\{y\}$
p. $\exists x.\forall y.G(x, y)$	$\emptyset$	$\{x, y\}$
q. $\forall x.(G(y, x) \Rightarrow \exists y.G(z, y))$	$\{y, z\}$	$\{x, y\}$

(In q, y is both free, in $G(y, x)$, and bound, in $\exists y.G(z, y)$.)

Exercise 2. *With the same signature and interpretation, translate: (1) Anne's mother has blue eyes. (2) Anne is taller than her paternal grandmother but shorter than her maternal grandmother. (3) Everybody whose parents both have blue eyes has blue eyes. (4) Anne's mother and father have blue eyes but Anne does not. (5) Nobody is taller than Anne's father. (6) Some people are not taller than their father.*

Solution. (1) $B(m(a))$. (2) $G(a, m(p(a))) \wedge G(m(m(a)), a)$. (3) $\forall x.((B(m(x)) \wedge B(p(x))) \Rightarrow$

$B(x))$. (4) $B(m(a)) \wedge B(p(a)) \wedge \neg B(a)$. (5) $\neg\exists x.G(x, p(a))$, or $\forall x.\neg G(x, p(a))$. (6) $\exists x.\neg G(x, p(x))$.

Exercise 3. *Let $\mathcal{F} = \emptyset$, $\mathcal{P} = \{<^{(2)}, =^{(2)}\}$, I with domain $\mathbb{N}$, $<$ the usual order and $=$ equality, and $\rho(x) = \rho(z) = 1$, $\rho(y) = 3$, $\rho(v) = 0$ otherwise. Let $\varphi_0 = \exists z.(z < x)$, $\varphi_1 = \forall x.\exists y.(x < y)$, $\varphi_2 = \exists y.\forall x.((x < y) \vee (x = y))$, $\varphi_3 = (x < y) \wedge \neg\exists z.((x < z) \wedge (z < y))$. (1) For each, does (I, ρ) satisfy it? Is I a model? (2) For which ρ' does $I, \rho' \models \varphi_3$? (3) Find an interpretation I' with an infinite domain that is a model of φ_2 but not of φ_1. (4) Find a model I'' of φ_1 and φ_2.*

Solution. (1) φ_0. With $\rho' = \rho[0 \leftarrow z]$ we have $0 < 1$, so $(I, \rho) \models \varphi_0$. But I is not a model: for $\rho'' = [0 \leftarrow x]$ no natural number is below 0.

φ_1. For any $e \in \mathbb{N}$ choose $f = e + 1 > e$. So $(I, \rho) \models \varphi_1$, and as φ_1 is closed, I is a model.

φ_2. Suppose some $e \in \mathbb{N}$ satisfies $\forall x.(x < e \vee x = e)$; taking $x = e + 1$ gives a contradiction. So $(I, \rho) \not\models \varphi_2$ and, being closed, I is not a model.

φ_3. $x < y$ holds ($1 < 3$), but $z = 2$ satisfies $x < z < y$, so $\exists z.(\cdots)$ holds and φ_3 fails: $(I, \rho) \not\models \varphi_3$, and I is not a model.

(2) $I, \rho' \models \varphi_3$ iff $\rho'(x) < \rho'(y)$ and no integer lies strictly between, i.e. iff $\rho'(y) = \rho'(x) + 1$.

(3) We need an element e greater than or equal to all others (for φ_2) and an element with no strictly greater one (against φ_1); take e the same element: $D_{I'} = [0, 1] \subset \mathbb{R}$ with the usual order; 1 is maximal.

(4) We need a maximum-like element for φ_2 but every element strictly below another for φ_1: impossible if $<$ is an order, but nothing forces that. Take $D_{I''} = \mathbb{N}$ with $<$ interpreted as the total relation $\mathbb{N}^2$; then φ_1 and φ_2 are trivially true.

Exercise 4. *Same signature as Exercise 1. For each formula decide: satisfiable? has a model? valid? unsatisfiable? Give an example and a counterexample for satisfiable formulas without model and for formulas with a model that are not valid, and prove validity or unsatisfiability. $\varphi_1 = \forall x.(B(x) \vee \neg B(x))$, $\varphi_2 = \forall x.(B(x) \Rightarrow B(z))$, $\varphi_3 = \forall x.\forall y.(B(x) \Leftrightarrow \neg B(y))$, $\varphi_4 = G(x, x) \Rightarrow \exists y.G(x, y)$, $\varphi_5 = (\exists y.G(x, y)) \Rightarrow G(x, x)$, $\varphi_6 = \neg B(y) \wedge \exists y.B(y)$, $\varphi_7 = \exists x.\forall y.((B(x) \vee B(y)) \Rightarrow B(y))$.*

Solution. φ_1 is valid: for every I and element e, $B(x) \vee \neg B(x)$ holds under $[e \leftarrow x]$.

φ_2 has a model, so is satisfiable, but is not valid. Any I with $B_I = D_I$ is a model. But for $D_I = \{0, 1\}$, $B_I = \{1\}$ and $\rho = [0 \leftarrow z]$, taking $x = 1$ gives $B(x)$ true and $B(z)$ false, so $I, \rho \not\models \varphi_2$.

φ_3 is unsatisfiable: if $I, \rho \models \varphi_3$, pick $e \in D_I$ (non-empty) and set $x = y = e$; then $B(x) \Leftrightarrow \neg B(y)$ fails.

φ_4 is valid: if $G(x, x)$ holds, take y to be (the value of) x.

φ_5 has a model, so is satisfiable, but is not valid. Any I with $G_I = \emptyset$ is a model, since the antecedent is always false. But for $D_I = \{0, 1\}$ and $G_I = \{(0, 1), (1, 0)\}$ there is no satisfying valuation: if $x = 0$ take $y = 1$ and the implication fails, and symmetrically.

φ_6 (y is both free and bound) is satisfiable, has no model and is not valid. $I = \{0\}$ with $B_I = \{0\}$ admits no satisfying valuation ($\neg B(y)$ is never true), so φ_6 is not valid. $I = \{0, 1\}$, $B_I = \{0\}$ and $\rho = [1 \leftarrow y]$ satisfy it ($\exists y.B(y)$ is witnessed by 0), so it is satisfiable. No I is a model: if ρ satisfies it then $\rho(y) \notin B_I$ and $B_I \neq \emptyset$; taking $\rho' = [e \leftarrow y]$ with $e \in B_I$ falsifies $\neg B(y)$.

φ_7 is valid. Let I be any interpretation. If $B_I = D_I$, then $B(y)$ always holds so $(B(x) \vee B(y)) \Rightarrow$

$B(y)$ holds for all x, y. Otherwise pick $e \notin B_I$ and set $x = e$: then $B(x)$ is false and the formula becomes $\forall y.(B(y) \Rightarrow B(y))$, which is true.

Exercise 5. *Consider the directed graph with vertices a, b, c, d viewed as an interpretation I with binary relation E, $D_I = \{a, b, c, d\}$ and $E_I = \{(a, b), (b, a), (b, c), (c, c), (b, d)\}$. For each formula give a valuation satisfying it or show it is not satisfiable in I; then show I is a model or give a valuation witnessing that it is not. $\varphi_1 = E(x, x)$, $\varphi_2 = E(x, y) \wedge E(y, x)$, $\varphi_3 = \exists y.(E(x, y) \wedge E(y, z))$, $\varphi_4 = \exists x.E(x, y)$, $\varphi_5 = \forall x.\neg E(x, y)$, $\varphi_6 = \exists y.E(x, y)$.*

Solution. φ_1: $[c \leftarrow x]$ satisfies it, $[a \leftarrow x]$ does not, so I is not a model. φ_2: $[a \leftarrow x, b \leftarrow y]$ satisfies it, $[a \leftarrow x, c \leftarrow y]$ does not. φ_3: $[a \leftarrow x, a \leftarrow z]$ satisfies it (witness $y = b$), $[d \leftarrow x, d \leftarrow z]$ does not. φ_4: $[d \leftarrow y]$ satisfies it (witness $x = b$); in fact I is a model, since every vertex has a predecessor ($a \leftarrow b$, $b \leftarrow a$, $c \leftarrow b$ and c, $d \leftarrow b$). φ_5 is the negation of φ_4, of which I is a model, so φ_5 is not satisfiable in I. φ_6: $[a \leftarrow x]$ satisfies it (witness $y = b$), $[d \leftarrow x]$ does not (d has no successor), so I is not a model.

Exercise sheet 6 — Transformation of first-order formulas

Reminder. A substitution σ with finite domain maps variables to terms. It is applicable to φ if $(\mathrm{dom}(\sigma) \cup \mathrm{rv}(\sigma)) \cap \mathrm{bv}(\varphi) = \emptyset$, where $\mathrm{rv}(\sigma)$ is the set of variables occurring in the images. Up to renaming the bound variables of φ with fresh ones, a substitution can always be applied. With $\rho_\sigma(x) = [\![\sigma(x)]\!]_{I,\rho}$, the substitution lemma says $[\![\sigma(\varphi)]\!]_{I,\rho} = [\![\varphi]\!]_{I,\rho_\sigma}$. Quantifier equivalences: $\forall x.\varphi \equiv \neg\exists x.\neg\varphi$; $\exists x.(\varphi \vee \psi) \equiv (\exists x.\varphi) \vee (\exists x.\psi)$; $\forall x.(\varphi \wedge \psi) \equiv (\forall x.\varphi) \wedge (\forall x.\psi)$; like quantifiers commute; and if x is not free in ψ: $\forall x.(\varphi \wedge \psi) \equiv (\forall x.\varphi) \wedge \psi$, $\forall x.(\varphi \vee \psi) \equiv (\forall x.\varphi) \vee \psi$, and the same for $\exists$.

Exercise 1. *Let $\mathcal{F} = \{f^{(1)}, g^{(2)}\}$, $\mathcal{P} = \{R^{(2)}\}$, $\sigma = [f(z) \leftarrow x,\ g(x,y) \leftarrow y]$. For each formula give a renaming to which σ is applicable and apply σ: $\varphi_1 = R(x,y) \wedge \exists x.R(x,x)$; $\varphi_2 = \exists x.R(f(x),y)$; $\varphi_3 = \exists z.R(f(x),z)$; $\varphi_4 = \forall x.\exists y.R(x,y)$.*

Solution. We have $\mathrm{dom}(\sigma) = \{x,y\}$ and $\mathrm{rv}(\sigma) = \{x,y,z\}$, so bound variables must avoid x, y, z.

$$\begin{aligned}
\varphi_1' &= R(x,y) \wedge \exists t.R(t,t) & \sigma(\varphi_1') &= R(f(z),g(x,y)) \wedge \exists t.R(t,t)\\
\varphi_2' &= \exists t.R(f(t),y) & \sigma(\varphi_2') &= \exists t.R(f(t),g(x,y))\\
\varphi_3' &= \exists t.R(f(x),t) & \sigma(\varphi_3') &= \exists t.R(f(f(z)),t)\\
\varphi_4' &= \forall t.\exists s.R(t,s) & \sigma(\varphi_4') &= \forall t.\exists s.R(t,s)
\end{aligned}$$

Exercise 2 (The drinker formula)**.** *Let $\varphi = \exists x.(B(x) \Rightarrow \forall y.B(y))$. (1) State φ in English; does it seem valid? (2) Eliminate the implication and use a passing law to obtain an equivalent disjunction φ'. (3) Rename y to x in φ', state the result in English; does it seem valid?*

Solution. (1) "There is somebody such that if this person drinks then everybody drinks." This looks odd, unless we remember that an implication with a false premise is true.

(2) $\exists x.(\neg B(x) \vee \forall y.B(y))$. Since x is not free in $\forall y.B(y)$, the passing law gives $\varphi' = (\exists x.\neg B(x)) \vee (\forall y.B(y))$.

(3) Subformulas can be renamed independently: $(\exists x.\neg B(x)) \vee \forall x.B(x)$, i.e. "somebody does not drink, or everybody drinks", which is obviously valid by case analysis.

Exercise 3. *(1) If x is not free in ψ, show $\forall x.(\varphi \wedge \psi) \equiv (\forall x.\varphi) \wedge \psi$ without using the passing laws (use Proposition 5.18: the semantics of ψ does not depend on $\rho(x)$ when $x \notin \mathrm{fv}(\psi)$). (2) Are $(\forall x.P(x)) \vee \neg P(x)$ and $\forall x.(P(x) \vee \neg P(x))$ equivalent?*

Solution. (1) Fix I, ρ. ($\Leftarrow$) If $I, \rho \models (\forall x.\varphi) \wedge \psi$ and $e \in D_I$, then $I, \rho[e \leftarrow x] \models \varphi$, and since $x \notin \mathrm{fv}(\psi)$, Proposition 5.18 gives $I, \rho[e \leftarrow x] \models \psi$. So $I, \rho[e \leftarrow x] \models \varphi \wedge \psi$ for all e. ($\Rightarrow$) If

$I, \rho \models \forall x.(\varphi \wedge \psi)$, then for all e, $I, \rho[e \leftarrow x] \models \varphi$ and $\models \psi$. Hence $I, \rho \models \forall x.\varphi$. As $D_I \neq \emptyset$ and $x \notin \mathrm{fv}(\psi)$, Proposition 5.18 gives $I, \rho \models \psi$.

(2) No. $\forall x.(P(x) \vee \neg P(x))$ is valid. But $(\forall x.P(x)) \vee \neg P(x)$ is not: with $D_I = \{0, 1\}$, $P_I = \{1\}$ and $\rho = [1 \leftarrow x]$, $\forall x.P(x)$ fails (as $P(0)$ is false) and $\neg P(x)$ fails (as $P(1)$ is true), so the formula is not satisfied. This does not contradict the passing law because x is free in $\neg P(x)$.

Exercise 4. *Put in NNF (first-order NNF: $\ell ::= \alpha \mid \neg\alpha$ and $\varphi ::= \ell \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \mid \exists x.\varphi \mid \forall x.\varphi$):* $\varphi_1 = \neg\forall x.\exists y.\forall z.(\neg R(x, z) \vee B(y))$ *and* $\varphi_2 = (\exists x.(B(x) \wedge \neg R(x, y))) \Rightarrow (\neg\forall y.R(y, y))$.

Solution.

$$\begin{aligned}\mathrm{NNF}(\varphi_1) &= \exists x.\mathrm{NNF}(\neg\exists y.\forall z.(\neg R(x, z) \vee B(y))) = \exists x.\forall y.\exists z.\mathrm{NNF}(\neg(\neg R(x, z) \vee B(y))) \\ &= \exists x.\forall y.\exists z.(R(x, z) \wedge \neg B(y)).\end{aligned}$$

For φ_2, first remove the implication: $(\neg\exists x.(B(x) \wedge \neg R(x, y))) \vee (\neg\forall y.R(y, y))$. Then

$$\mathrm{NNF}(\varphi_2) = (\forall x.(\neg B(x) \vee R(x, y))) \vee (\exists y.\neg R(y, y)).$$

Exercise 5. *Give prenex normal forms (renaming if needed) of the two formulas of the previous exercise.*

Solution. $\mathrm{NNF}(\varphi_1) = \exists x.\forall y.\exists z.(R(x, z) \wedge \neg B(y))$ is already prenex. For φ_2: since x is not free in $\exists y.\neg R(y, y)$, rewrite as $\forall x.(\neg B(x) \vee R(x, y) \vee \exists y.\neg R(y, y))$. Rename the bound y to z (since y is free in the rest): $\forall x.(\neg B(x) \vee R(x, y) \vee \exists z.\neg R(z, z))$. Since z is not free in $\neg B(x) \vee R(x, y)$, we get the prenex form

$$\forall x.\exists z.(\neg B(x) \vee R(x, y) \vee \neg R(z, z)).$$

Exercise sheet 7 — Logical theories

Reminder. A theory is a set T of closed formulas over a first-order signature, closed under logical consequence. For a set A of closed axioms, $\mathrm{Th}(A) = \{\varphi \text{ closed} \mid A \models \varphi\}$. The equivalence theory A_{eq} for $E^{(2)}$ has reflexivity $\forall x.E(x,x)$, symmetry $\forall x.\forall y.E(x,y) \Rightarrow E(y,x)$ and transitivity $\forall x.\forall y.\forall z.(E(x,y) \wedge E(y,z)) \Rightarrow E(x,z)$. The congruence theory A_{cgr} adds, for $f \in \mathcal{F}_n$, $\forall \vec{x}.\forall \vec{y}.(E(x_1,y_1) \wedge \cdots \wedge E(x_n,y_n) \Rightarrow E(f(\vec{x}),f(\vec{y})))$ and, for $R \in \mathcal{P}_m \setminus \{E\}$, $\forall \vec{x}.\forall \vec{y}.((E(x_1,y_1) \wedge \cdots \wedge E(x_m,y_m) \wedge R(\vec{x})) \Rightarrow R(\vec{y}))$.

Exercise 1. *(1) Show that $\varphi \in \mathrm{Th}(\emptyset)$ iff φ is closed and valid. (2) If A is a set of closed valid formulas, show $\varphi \in \mathrm{Th}(A)$ iff φ is closed and valid. (3) For closed φ, show $\psi \in \mathrm{Th}(\{\varphi\})$ iff $\varphi \Rightarrow \psi$ is closed and valid.*

Solution. (1) $\emptyset \models \varphi$ iff every model of $\emptyset$ satisfies φ; every interpretation is a model of $\emptyset$, so this means φ is valid (no valuation is needed for closed formulas). (2) If $A \models \varphi$: every interpretation satisfies A since its elements are valid, hence satisfies φ, so φ is valid. Conversely if φ is valid, every interpretation satisfying A satisfies φ, so $A \models \varphi$. (3) By the first-order extension of Property 2.11, $\mathrm{Th}(\{\varphi\}) = \{\psi \text{ closed} \mid \varphi \models \psi\} = \{\psi \text{ closed} \mid \varphi \Rightarrow \psi \text{ valid}\}$.

Exercise 2. *(1) Let $\mathcal{F} = \emptyset$, $\mathcal{P} = \{E^{(2)}, R^{(2)}\}$. For five interpretations (a)–(e) with domain $\{a,b,c\}$, say whether E_I is an equivalence relation and whether it is a congruence, naming the failing axioms. (2) Same question for $\mathcal{F} = \{f^{(2)}\}$ with E_I relating only a and b (plus reflexive pairs), and $f_I(a,a) = f_I(a,b) = f_I(b,a) = f_I(b,b) = f_I(c,c) = c$, $f_I(a,c) = f_I(c,a) = a$, $f_I(b,c) = f_I(c,b) = b$.*

Solution. (1) The conclusions are: in (a) and (e), E_I is a congruence. In (b), E_I is an equivalence relation but not a congruence: R-congruence fails, e.g. $(b,a),(c,c) \in E_I$ and $(b,c) \in R_I$ but $(a,c) \notin R_I$. In (c), E_I is not an equivalence: it is not reflexive, since $(b,b) \notin E_I$. In (d), E_I is not transitive: $(a,b),(b,c) \in E_I$ but $(a,c) \notin E_I$.

(2) E_I is an equivalence relation with classes $\{a,b\}$ and $\{c\}$. We check f-congruence: take $(e_1,f_1),(e_2,f_2) \in E_I$.

- If all four are in $\{a,b\}$ then $f_I(e_1,e_2) = f_I(f_1,f_2) = c$ and $(c,c) \in E_I$.
- If $e_1, f_1 \in \{a,b\}$ and $e_2 = f_2 = c$, then $f_I(e_1,c) = e_1$ and $f_I(f_1,c) = f_1$, and $(e_1,f_1) \in E_I$. The symmetric case is similar.
- If $e_1 = f_1 = e_2 = f_2 = c$, we get $(c,c) \in E_I$.

So E_I is a congruence.

Exercise 3. *A k-clique of an undirected graph is a set C of k vertices that are pairwise adjacent. On the signature $\mathcal{F} = \emptyset$, $\mathcal{P} = \{E^{(2)}, =^{(2)}\}$, find an axiomatisation $A_{k\text{-clique}}$ such that $I \models A_{k\text{-clique}}$ iff (D_I, E_I) is an undirected graph containing a k-clique.*

Solution. Let $\varphi_1 = \forall x.\neg E(x,x)$ (irreflexive), $\varphi_2 = \forall x.\forall y.(E(x,y) \Rightarrow E(y,x))$ (symmetric), and

$$\varphi_3 = \exists x_1 \dots \exists x_k.\Big(\bigwedge_{1 \le i < j \le k} E(x_i, x_j) \Big).$$

Since E is irreflexive, adjacent vertices are distinct, so the x_i are pairwise distinct. Take $A_{k\text{-clique}} = \{\varphi_1, \varphi_2, \varphi_3\}$.

Exercise 4. *In* $\mathrm{Th}(A_{\mathrm{lin}})$*, the theory of strict linear orders over* $(\emptyset, \{<, =\})$ *with irreflexivity* $\forall x.\neg(x < x)$*, transitivity and totality* $\forall x.\forall y.(x < y \vee x = y \vee y < x)$*, show that* $\varphi = \forall x.\exists y.(\neg(y < x) \Rightarrow \forall z.(x < z \vee x = z))$ *belongs to the theory.*

Solution. Removing the implication and using the passing law (y is not free in $\forall z.(\dots)$), then renaming z to y:

$$\varphi \equiv \forall x.\big((\exists y.\, y < x) \vee (\forall y.(x < y \vee x = y))\big).$$

Intuitively: every x either has a smaller element or is a minimum.

Let $I \models A_{\mathrm{lin}}$ and $e \in D_I$. We show (1): $I, [e \leftarrow x] \models (\exists y.\, y < x) \vee \forall y.(x < y \vee x = y)$. If the first disjunct holds we are done. Otherwise $I, [e \leftarrow x] \models \forall y.\neg(y < x)$. Totality gives $I, [e \leftarrow x] \models \forall y.(x < y \vee x = y \vee y < x)$. Since $\forall$ distributes over $\wedge$ (not over $\vee$),

$$\begin{aligned}
&\forall y.\neg(y < x) \wedge \forall y.(x < y \vee x = y \vee y < x) \\
&\equiv \forall y.\big(\neg(y < x) \wedge (x < y \vee x = y \vee y < x)\big) \\
&\equiv \forall y.\big((\neg(y < x) \wedge x < y) \vee (\neg(y < x) \wedge x = y)\big),
\end{aligned}$$

using distributivity and the fact that $\bot$ is neutral for $\vee$. A fortiori $I, [e \leftarrow x] \models \forall y.(x < y \vee x = y)$, hence (1) holds. So $\varphi \in \mathrm{Th}(A_{\mathrm{lin}})$. Only totality was used.

Exercise 5 (Stacks)**.** *Consider stacks over the finite alphabet* $\{a, b\}$ *with signature* $\mathcal{F} = \{\$^{(0)}, \mathrm{push}_a^{(1)}, \mathrm{push}_b^{(1)}, \mathrm{pop}^{(1)}\}$*,* $\mathcal{P} = \{=, \mathrm{empty}^{(1)}, \mathrm{top}_a^{(1)}, \mathrm{top}_b^{(1)}\}$*. The empty stack is* $\$$*; pushing* a *then* b *gives* $\mathrm{push}_b(\mathrm{push}_a(\$))$*. (1) Describe in first-order logic: the only empty stack is* $\$$*; the top of a stack just pushed with* σ *is* σ*; popping gives back the stack before the push. (2) The standard interpretation has domain* $\{a, b\}^*$*, the empty stack being the empty word and the top being on the right. Interpret the symbols. (3) Check the properties. (4) Show they have other interpretations, e.g. with domain* $\{a, b\}^* \cup \{a, b\}^{\mathbb{N}}$*.*

Solution. (1)

$$\begin{aligned}
\varphi_1 &= \forall s.\ \mathrm{empty}(s) \Leftrightarrow (s = \$) \\
\varphi_2 &= \forall s.\ \mathrm{top}_a(s) \Leftrightarrow \exists s'.\, s = \mathrm{push}_a(s') \\
\varphi_3 &= \forall s.\ \mathrm{top}_b(s) \Leftrightarrow \exists s'.\, s = \mathrm{push}_b(s') \\
\varphi_4 &= \forall s.\ \mathrm{pop}(\mathrm{push}_a(s)) = s \\
\varphi_5 &= \forall s.\ \mathrm{pop}(\mathrm{push}_b(s)) = s
\end{aligned}$$

(2) $\$_I = \varepsilon$, $(\mathrm{push}_a)_I(w) = wa$, $(\mathrm{push}_b)_I(w) = wb$, $\mathrm{pop}_I(\varepsilon) = \varepsilon$ and $\mathrm{pop}_I(ua) = \mathrm{pop}_I(ub) = u$. $=$ is equality; $\mathrm{empty}_I(w) = 1$ iff $w = \varepsilon$; $\mathrm{top}_a(w) = 1$ iff $w = ua$ for some u; $\mathrm{top}_b(w) = 1$ iff $w = ub$ for some u.

(3) The formulas are closed so the valuation does not matter. φ_1: $\mathrm{empty}(w)$ holds iff $w = \varepsilon = \$_I$. φ_2 (and φ_3 similarly): $\mathrm{top}_a(w)$ holds iff $w = ua$ iff $w = (\mathrm{push}_a)_I(u)$ for some u. φ_4 (φ_5 similarly): $\mathrm{pop}_I((\mathrm{push}_a)_I(w)) = \mathrm{pop}_I(wa) = w$.

(4) Keep the standard interpretation on finite words and, for an infinite sequence $u = (u_i)_{i \in \mathbb{N}}$, define $(\mathrm{push}_a)_I(u) = v$ with $v_0 = a$ and $v_{i+1} = u_i$ (similarly for b), $\mathrm{pop}_I(u) = (u_{i+1})_{i \in \mathbb{N}}$,

$\text{empty}_I(u) = 0$, $\text{top}_a(u) = 1$ iff $u_0 = a$, $\text{top}_b(u) = 1$ iff $u_0 = b$. All five formulas still hold. Nothing links the standard elements to the added ones: from one kind one never reaches the other by finitely many operations.

Exercise 6. *Show* $\text{Th}(A_{\text{eq}}) = \text{Th}(A'_{\text{eq}})$ *where* A'_{eq} *contains reflexivity and* $\forall x.\forall y.\forall z.((E(x,y) \wedge E(x,z)) \Rightarrow E(y,z))$ *(†).*

Solution. It suffices to show that each axiom of one set is in the theory of the other.

Axioms of A_{eq} *are in* $\text{Th}(A'_{\text{eq}})$. Let $I \models A'_{\text{eq}}$. Reflexivity is an axiom of A'_{eq}. *Symmetry:* let $e, f \in D_I$ with $(e,f) \in E_I$; take $z = e$. By reflexivity $(e,e) \in E_I$, so $(e,f),(e,e) \in E_I$ and (†) gives $(f,e) \in E_I$. *Transitivity:* if $(e,f),(f,g) \in E_I$, symmetry gives $(f,e) \in E_I$; (†) applied to (f,e) and (f,g) gives $(e,g) \in E_I$.

Axiom † is in $\text{Th}(A_{\text{eq}})$. Let $I \models A_{\text{eq}}$ and $(e,f),(e,g) \in E_I$. By symmetry $(f,e) \in E_I$, and by transitivity $(f,g) \in E_I$.

By closure under consequence the two theories coincide.

Exercise sheet 8 — Modelling in first-order logic

Reminder. An SMT solver such as Z3 searches for a pair (I, ρ) satisfying a formula where I additionally satisfies a fixed theory T. Sometimes the theory fixes the interpretation entirely (e.g. linear rational arithmetic) and the solver only produces a valuation; sometimes some symbols are uninterpreted and the solver must also produce their interpretation. For a closed formula, only the uninterpreted symbols have to be interpreted.

Exercise 1 (Coin change)**.** *Given coin values $v_1, \ldots, v_k$ and a sum S, find the minimal number of coins reaching exactly S. Model it as satisfiability modulo $(\mathbb{N}, 0, 1, +, \times, <)$, with $\mathcal{F} = \{0, 1, +, \times\}$ and $\mathcal{P} = \{=, <\}$. (1) Example: coins of 10, 20, 50 cents, $S = 110$. (a) Minimal number of coins? (b) Write the formula expressing that x_1, x_2, x_3 coins give the sum. (2) Generalise it to an atomic formula $\varphi_1(x_1, \ldots, x_k)$. (3) Write an atomic formula $\varphi_2(x_1, \ldots, x_k, m)$ saying that exactly m coins are used. (4) Write $\varphi(m)$ saying that S can be reached with exactly m coins. (5) Write $\psi(m)$ saying that m is the minimal number.*

Solution. (1a) 3 coins: two of 50 and one of 10; fewer than 3 is impossible. (1b) $S = 10 \times x_1 + 20 \times x_2 + 50 \times x_3$, i.e. $110 = \ldots$.

(2) $\varphi_1(x_1, \ldots, x_k) \stackrel{\text{def}}{=} S = v_1 \times x_1 + \cdots + v_k \times x_k$ (atomic since k, S and the v_i are fixed).

(3) $\varphi_2 \stackrel{\text{def}}{=} m = x_1 + \cdots + x_k$.

(4) $\varphi(m) \stackrel{\text{def}}{=} \exists x_1 \ldots \exists x_k.\ (m = x_1 + \cdots + x_k \wedge S = v_1 \times x_1 + \cdots + v_k \times x_k)$.

(5) $\psi(m) \stackrel{\text{def}}{=} \varphi(m) \wedge \forall m'.(m' < m \Rightarrow \neg\varphi(m'))$. Asking Z3 for satisfiability of $\psi(m)$ modulo $(\mathbb{N}, 0, 1, +, \times, <)$ with m free, the interpretation is fixed and the solver returns a valuation of m, namely the minimal number of coins.

Exercise 2 (Invariant synthesis)**.** *Consider the program*

```
int i = 1, s = 0;
while (i < 6) { s = s + i; i = i + 1; }
assert (s == 15)
```

(1) Give the set Reach $\subseteq \mathbb{Z}^2$ *of reachable (i, s) at each loop test. (2) Give the set* Error *of values contradicting the assertion. (3) On the signature $(\{+, (\underline{i})_{i\in\mathbb{N}}\}, \{<, =, \text{Invar}^{(2)}\})$ with the integer arithmetic fixed, give a closed formula saying that* Invar *is a safe inductive invariant. (4) If* $\text{Invar}_I = \text{Reach}$*, is I a model? Give another model.*

Solution. (1) Reach $= \{(1, 0), (2, 1), (3, 3), (4, 6), (5, 10), (6, 15)\}$. (2) Error $= \{(i, s) \mid i \geq$

$6,\ s \neq 15\}$. (3)

$$\begin{aligned}\varphi \stackrel{\text{def}}{=}\ & \mathrm{Invar}(1,0)\\ & \wedge \forall x.\forall y.(\mathrm{Invar}(x,y) \wedge x < 6 \Rightarrow \mathrm{Invar}(x+1, y+x))\\ & \wedge \forall x.\forall y.(\mathrm{Invar}(x,y) \wedge (6 < x \vee x = 6) \Rightarrow y = 15)\end{aligned}$$

These say: initialisation, preservation by one loop iteration, and safety on exit. Z3 checks satisfiability of φ modulo integer arithmetic; the only interesting interpretation is that of Invar. (4) Yes: $(1,0) \in \mathrm{Reach}$; if $(i,s) \in \mathrm{Reach}$ with $i < 6$ then $(i+1, s+i) \in \mathrm{Reach}$; and the only points with $i \geq 6$ have $s = 15$. Another model adds $(0,0)$ (its successor $(1,0)$ is in Reach), and then $(-1,1), (-2,3), (-3,6), \ldots$ and so on.

Exercise 3. *Let $\mathcal{F} = \emptyset$, $\mathcal{P} = \{E^{(2)}, =, G^{(1)}\}$ and I with $D_I = \{0,1,2,3\}$, $E_I = \{(0,0),(0,1),(0,2),(1,1),(2,0),(3,0),(3,1),(3,2)\}$, $G_I = \{0,3\}$ (a directed graph whose grey vertices are 0 and 3). (1) Is I a model of $(E(x,y) \wedge E(y,z)) \Rightarrow E(x,z)$? (2) Of $G(x) \Leftrightarrow \forall y.((E(x,y) \wedge \neg(x = y)) \Rightarrow \neg G(y))$? (3) Write a formula saying that every non-grey vertex has out-degree exactly 1; is I a model?*

Solution. (1) The formula states transitivity of E. It fails under $[2 \leftarrow x, 0 \leftarrow y, 1 \leftarrow z]$ since $(2,0), (0,1) \in E_I$ but $(2,1) \notin E_I$. Not a model.

(2) For $x = 3$ the right-hand side fails: with $y = 0$ we have $E(3,0) \wedge 3 \neq 0$ but $G(0)$ holds. So $G(3)$ is true and the right-hand side false: the equivalence fails, and I is not a model.

(3) $\forall x.(\neg G(x) \Rightarrow \exists y.(E(x,y) \wedge \forall z.(y \neq z \Rightarrow \neg E(x,z))))$. I is a model: the non-grey vertices 1 and 2 have the single outgoing edges $(1,1)$ and $(2,0)$.

Exercise 4. *On the signature $\mathcal{F} = \emptyset$, $\mathcal{P} = \{\mathrm{Eats}^{(2)}, \mathrm{Herbivore}^{(1)}, \mathrm{Plant}^{(1)}, \mathrm{Bamboo}^{(1)}, \mathrm{Panda}^{(1)}\}$, express: (1) herbivores only eat plants; (2) no herbivore eats every kind of plant; (3) some plants are eaten by no herbivore; (4) pandas are herbivores that only eat bamboo.*

Solution.

1. $\forall h.(\mathrm{Herbivore}(h) \Rightarrow \forall m.(\mathrm{Eats}(h,m) \Rightarrow \mathrm{Plant}(m)))$
2. $\neg\exists h.(\mathrm{Herbivore}(h) \wedge \forall v.(\mathrm{Plant}(v) \Rightarrow \mathrm{Eats}(h,v)))$
3. $\exists v.(\mathrm{Plant}(v) \wedge \neg\exists h.(\mathrm{Herbivore}(h) \wedge \mathrm{Eats}(h,v)))$
4. $\forall p.(\mathrm{Panda}(p) \Rightarrow (\mathrm{Herbivore}(p) \wedge \forall m.(\mathrm{Eats}(p,m) \Rightarrow \mathrm{Bamboo}(m))))$

Exercise 5. *Choose a signature and formalise: (1) at least two people prepared and passed the exam; (2) everybody received advice from Alice or Bob; (3) everybody who prepared the exam and received advice from Bob failed it; (4) somebody gave advice to exactly one person; (5) Alice received advice from somebody who received advice from Bob, and she failed the exam.*

Solution. Take constants $\mathcal{F} = \{a^{(0)}, b^{(0)}\}$ and predicates $\mathcal{P} = \{\mathrm{Prepared}^{(1)}, \mathrm{Passed}^{(1)}, \mathrm{Advises}^{(2)}, =^{(2)}\}$, where $\mathrm{Advises}(x,y)$ means x gave advice to y.

1. $\exists x.\exists y.(x \neq y \wedge \mathrm{Prepared}(x) \wedge \mathrm{Prepared}(y) \wedge \mathrm{Passed}(x) \wedge \mathrm{Passed}(y))$
2. $\forall x.(\mathrm{Advises}(a,x) \vee \mathrm{Advises}(b,x))$
3. $\forall x.((\mathrm{Prepared}(x) \wedge \mathrm{Advises}(b,x)) \Rightarrow \neg\mathrm{Passed}(x))$
4. $\exists x.\exists y.(\mathrm{Advises}(x,y) \wedge \forall z.(z \neq y \Rightarrow \neg\mathrm{Advises}(x,z)))$
5. $\exists x.(\mathrm{Advises}(x,a) \wedge \mathrm{Advises}(b,x) \wedge \neg\mathrm{Passed}(a))$

Exercise sheet 9 — First-order sequent calculus

Reminder. The first-order sequent calculus adds to (ax), (∨), (∧) (axiom now $\vdash \Gamma, \ell, \overline{\ell}$ for a literal ℓ) the rules

$$\frac{\vdash \Gamma, [y \leftarrow x](\varphi)}{\vdash \Gamma, \forall x.\varphi}\ (\forall)\ \ y \notin \mathrm{fv}(\Gamma, \forall x.\varphi) \qquad \frac{\vdash \Gamma, [t \leftarrow x](\varphi), \exists x.\varphi}{\vdash \Gamma, \exists x.\varphi}\ (\exists)\ \ t \in T(\mathcal{F}, X)$$

and the cut rule. It is sound and complete, but validity is undecidable, in particular because rule (∃) keeps $\exists x.\varphi$ in its premise and does not have the finite-branch property. Admissible rules: weakening (W) $\vdash \Gamma \ /\ \vdash \Gamma, \Delta$, contraction (C) $\vdash \Gamma, \varphi, \varphi \ /\ \vdash \Gamma, \varphi$. In the derivations below, proofs are written bottom-up as lists, each line being the conclusion of the rule indicated, from the root to the leaf.

Exercise 1. *Let $\mathcal{F} = \{a^{(0)}, f^{(1)}\}$, $\mathcal{P} = \{P^{(1)}, R^{(2)}\}$. Put in NNF and prove in the sequent calculus: (1) $\neg R(x,y) \vee \exists z.R(x,z)$; (2) $(P(x) \wedge P(y)) \vee \exists x.\neg P(x)$; (3) $\exists x.(P(f(x)) \Rightarrow P(x))$; (4) $(\forall x.P(x)) \Rightarrow (\forall x.(P(x) \vee P(y)))$; (5) $(\forall x.\forall y.R(x,y)) \Rightarrow (\forall x.\forall y.R(y,x))$.*

Solution. (1) Already in NNF.

$$\dfrac{\dfrac{\dfrac{}{\vdash \neg R(x,y), \exists z.R(x,z), R(x,y)}\,(\mathrm{ax})}{\vdash \neg R(x,y), \exists z.R(x,z)}\,(\exists)}{\vdash \neg R(x,y) \vee \exists z.R(x,z)}\,(\vee)$$

(2) Already in NNF. Write $\chi = \exists x.\neg P(x)$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash P(x), \chi, \neg P(x)}\,(\mathrm{ax})}{\vdash P(x), \chi}\,(\exists) \qquad \dfrac{\dfrac{}{\vdash P(y), \chi, \neg P(y)}\,(\mathrm{ax})}{\vdash P(y), \chi}\,(\exists)}{\vdash P(x) \wedge P(y), \chi}\,(\wedge)}{\vdash (P(x) \wedge P(y)) \vee \chi}\,(\vee)$$

(3) The NNF is $\exists x.(\neg P(f(x)) \vee P(x))$; write $\theta = \exists x.(\neg P(f(x)) \vee P(x))$. This uses the copy of the existential formula kept in the premise of (∃): we instantiate x by x and then by $f(x)$. Bottom-up:

$\vdash \theta$	root
$\vdash \theta,\ \neg P(f(x)) \vee P(x)$	(∃) with $t = x$
$\vdash \theta,\ \neg P(f(x)) \vee P(x),\ \neg P(f(f(x))) \vee P(f(x))$	(∃) with $t = f(x)$
$\vdash \theta,\ \neg P(f(x)), P(x),\ \neg P(f(f(x))) \vee P(f(x))$	(∨)
$\vdash \theta,\ \neg P(f(x)), P(x),\ \neg P(f(f(x))), P(f(x))$	(∨)

and the last sequent is an axiom ($\neg P(f(x))$ and $P(f(x))$).

(4) The NNF is $(\exists x.\neg P(x)) \vee (\forall x.(P(x) \vee P(y)))$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \exists x.\neg P(x), \neg P(x), P(x), P(y)}\,(\text{ax})}{\vdash \exists x.\neg P(x), P(x), P(y)}\,(\exists)}{\vdash \exists x.\neg P(x),\ P(x) \vee P(y)}\,(\vee)}{\vdash \exists x.\neg P(x),\ \forall x.(P(x) \vee P(y))}\,(\forall)}{\vdash (\exists x.\neg P(x)) \vee (\forall x.(P(x) \vee P(y)))}\,(\vee)$$

Rule ($\exists$) is used with $t = x$ (keeping a copy of $\exists x.\neg P(x)$), and rule ($\forall$) is legitimate because x is bound, not free, in the sequent $\vdash \exists x.\neg P(x), P(x) \vee P(y)$.

(5) The NNF is $(\exists x.\exists y.\neg R(x,y)) \vee (\forall x.\forall y.R(y,x))$. Write $\chi = \exists x.\exists y.\neg R(x,y)$. Bottom-up:

$\vdash \chi \vee \forall x.\forall y.R(y,x)$	root
$\vdash \chi,\ \forall x.\forall y.R(y,x)$	($\vee$)
$\vdash \chi,\ \forall y.R(y,u)$	($\forall$) with fresh u
$\vdash \chi,\ R(v,u)$	($\forall$) with fresh v
$\vdash \chi,\ \exists y.\neg R(v,y),\ R(v,u)$	($\exists$) $t = v$
$\vdash \chi,\ \exists y.\neg R(v,y),\ \neg R(v,u),\ R(v,u)$	($\exists$) $t = u$

The last sequent is an axiom. Fresh variable names avoid clashes, and the ($\forall$) rules must be applied lower (nearer the root) than the ($\exists$) rules so that the instantiating terms v, u are available.

Exercise 2. *(1) Put $(P(a) \wedge \forall x.(P(x) \Rightarrow P(f(x)))) \Rightarrow P(f(f(a)))$ in NNF. (2) Prove it without admissible rules. (3) Give a second proof using contraction (C) and cut on $P(f(a))$, reasoning by cases on $P(f(a))$.*

Solution. (1) $\neg P(a) \vee \big(\exists x.(P(x) \wedge \neg P(f(x)))\big) \vee P(f(f(a)))$.

(2) Let $\varphi = \exists x.(P(x) \wedge \neg P(f(x)))$ and $\Gamma_0 = \neg P(a), P(f(f(a)))$. After two ($\vee$) steps we reach $\vdash \Gamma_0, \varphi$. Instantiate x by a with ($\exists$), then ($\wedge$):

- left: $\vdash \Gamma_0, \varphi, P(a)$, an axiom ($\neg P(a)$, $P(a)$);
- right: $\vdash \Gamma_0, \varphi, \neg P(f(a))$, to which we apply ($\exists$) with $x := f(a)$ and ($\wedge$) once more: $\vdash \Gamma_0, \varphi, \neg P(f(a)), P(f(a))$ is an axiom, and $\vdash \Gamma_0, \varphi, \neg P(f(a)), \neg P(f(f(a)))$ is an axiom ($\neg P(f(f(a)))$ with $P(f(f(a))) \in \Gamma_0$).

(3) The cut formula is $P(f(a))$, which links $P(a)$ and $P(f(f(a)))$. First apply (C) to duplicate φ, so that it is available in both premises of the cut: from $\vdash \neg P(a), \varphi, \varphi, P(f(f(a)))$ we get $\vdash \neg P(a), \varphi, P(f(f(a)))$ by (C). The cut yields $\vdash \neg P(a), \varphi, \varphi, P(f(f(a)))$ from

- π_1: $\vdash \varphi, P(f(f(a))), \neg P(f(a))$ — apply ($\exists$) with $t = f(a)$ and ($\wedge$): the leaves $\vdash \varphi, \neg P(f(f(a))), P(f(f(a))), \neg P(f(a))$ and $\vdash \varphi, P(f(a)), P(f(f(a))), \neg P(f(a))$ are axioms;
- π_2: $\vdash \neg P(a), \varphi, P(f(a))$ — apply ($\exists$) with $t = a$ and ($\wedge$): the leaves $\vdash \neg P(a), \varphi, P(a), P(f(a))$ and $\vdash \neg P(a), \varphi, \neg P(f(a)), P(f(a))$ are axioms.

The proof is then closed with ($\vee$) twice. It proceeds by case analysis on the truth of $P(f(a))$.

Exercise 3. *(1) Show that* $\varphi_1 = (\exists x.P(x)) \Rightarrow (\forall x.P(x))$ *and* $\varphi_2 = (\exists x.(P(x) \vee Q(x))) \Rightarrow (\exists x.P(x))$ *are not valid and put them in NNF. (2) Find all errors in the rule applications of two derivations: the left one proves* $\vdash (\forall x.\neg P(x)) \vee (\forall x.P(x))$ *by applying* $(\forall)$ *to* $\vdash \neg P(x), P(x)$ *to get* $\vdash \forall x.\neg P(x), P(x)$ *and again to get* $\vdash \forall x.\neg P(x), \forall x.P(x)$*; the right one proves* $\vdash (\forall x.(\neg P(x) \wedge \neg Q(x))) \vee (\exists x.P(x))$ *through* $\vdash \neg P(x), \neg Q(x), P(x)$*, then a unary* $(\wedge)$ *giving* $\vdash \neg P(x) \wedge \neg Q(x), P(x)$*, then* $(\forall)$*, then* $(\exists)$*, then* $(\vee)$*.*

Solution. (1) I with $D_I = \{0, 1\}$, $P_I = \{0\}$ satisfies $\exists x.P(x)$ but not $\forall x.P(x)$, so φ_1 is not valid. Its NNF is $(\forall x.\neg P(x)) \vee (\forall x.P(x))$. I with $D_I = \{0\}$, $P_I = \emptyset$, $Q_I = \{0\}$ satisfies $\exists x.(P(x) \vee Q(x))$ but not $\exists x.P(x)$, so φ_2 is not valid. Its NNF is $(\forall x.(\neg P(x) \wedge \neg Q(x))) \vee (\exists x.P(x))$.

(2) Left derivation: the topmost $(\forall)$ is invalid because x is free in the rest of the sequent $(P(x))$. Right derivation: the same error for $(\forall)$ (x is free in $P(x)$), and moreover $(\wedge)$ must have two premises ($\vdash \ldots, \neg P(x)$ and $\vdash \ldots, \neg Q(x)$), not one.

Exercise 4 (Socrates' syllogism)**.** *"All humans are mortal, Socrates is human, so Socrates is mortal". With* $\mathcal{F} = \{s^{(0)}\}$, $\mathcal{P} = \{H^{(1)}, M^{(1)}\}$*, write it as a formula and prove its validity.*

Solution. $\varphi = ((\forall x.(H(x) \Rightarrow M(x))) \wedge H(s)) \Rightarrow M(s)$, whose NNF is $(\exists x.(H(x) \wedge \neg M(x))) \vee \neg H(s) \vee M(s)$. Let $\chi = \exists x.(H(x) \wedge \neg M(x))$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \chi, H(s), \neg H(s), M(s)}\,(\text{ax}) \quad \dfrac{}{\vdash \chi, \neg M(s), \neg H(s), M(s)}\,(\text{ax})}{\vdash \chi, H(s) \wedge \neg M(s), \neg H(s), M(s)}\,(\wedge)}{\vdash \chi, \neg H(s), M(s)}\,(\exists)}{\vdash \chi \vee \neg H(s) \vee M(s)}\,(\vee) \times 2$$

Exercise 5 (Admissibility of weakening)**.** *To show (W) admissible, prove by induction on the depth of the derivation that* $\vdash_{\text{LK}} \Gamma$ *implies* $\vdash_{\text{LK}} \Gamma, \Delta$*. (1) For* $\Gamma = \{P(x), \forall x.(P(x) \vee \neg P(x))\}$ *and* $\Delta = \{P(y) \wedge \neg P(y)\}$*, prove* $\vdash \Gamma$ *then* $\vdash \Gamma, \Delta$*. (2) Give the details for the cases (ax),* $(\vee)$*,* $(\forall)$*, admitting that if* $\vdash \Gamma$ *has a derivation of depth* k *then* $\vdash \sigma(\Gamma)$ *has one of depth* k *for any substitution* σ*.*

Solution. (1) A proof of $\vdash \Gamma$ (the bound x is renamed y to apply $(\forall)$, since x is free elsewhere): $\vdash P(x), P(y), \neg P(y)$ is an axiom; $(\vee)$ gives $\vdash P(x), P(y) \vee \neg P(y)$; $(\forall)$ gives $\vdash P(x), \forall x.(P(x) \vee \neg P(x))$. For $\vdash \Gamma, \Delta$ we change the fresh variable to z to avoid a clash with y free in Δ: the axiom $\vdash P(x), P(z), \neg P(z), P(y) \wedge \neg P(y)$, then $(\vee)$, then $(\forall)$ give $\vdash P(x), \forall x.(P(x) \vee \neg P(x)), P(y) \wedge \neg P(y)$.

(2) *Axiom.* Adding Δ to an axiom sequent keeps the complementary pair, so it is still an axiom. *Rule* $(\vee)$*.* $\Gamma = \Gamma', \varphi \vee \psi$ with premise $\vdash \Gamma', \varphi, \psi$ of lesser depth. By induction we have $\vdash \Gamma', \varphi, \psi, \Delta$, and $(\vee)$ gives $\vdash \Gamma', \varphi \vee \psi, \Delta$. *Rule* $(\forall)$*.* $\Gamma = \Gamma', \forall x.\varphi$ and the premise is $\vdash \Gamma', [y \leftarrow x](\varphi)$ with $y \notin \text{fv}(\Gamma)$. Choose $z \notin \text{fv}(\Gamma, \Delta)$. By the admitted renaming property, there is a derivation of $\vdash \Gamma', [z \leftarrow x](\varphi)$ of the same depth, hence of depth below that of the whole proof. By induction, $\vdash \Gamma', [z \leftarrow x](\varphi), \Delta$, and $(\forall)$ (legitimate because $z \notin \text{fv}(\Gamma, \Delta, \forall x.\varphi)$) gives $\vdash \Gamma', \forall x.\varphi, \Delta$. The other rules are similar.

Chapter 10

Links with constraint programming

This chapter collects the connections between the logic course and constraint programming (CP). In CP one states a problem as *variables*, *domains* and *constraints*, and a solver alternates *propagation* (removing values that cannot occur in any solution) and *search* (branching on a variable). Logic provides the language in which constraints are written and the proof/search procedures that underlie many solvers.

10.1 A dictionary

Logic (this course)	**Constraint programming**
Propositions, atoms, terms with variables	Variables with a domain
Interpretation I and valuation ρ	Assignment of values to variables
Model of φ (a pair (I, ρ) with $I, \rho \models \varphi$)	Solution of the constraint satisfaction problem (CSP)
Conjunction of formulas (a set S of formulas)	Conjunction of constraints (the constraint store)
Satisfiable / unsatisfiable	Consistent / inconsistent CSP
Logical consequence $A \models \varphi$	Entailed constraint
Unit rule, $S[\top \leftarrow \ell]$	Propagation after fixing a literal
Split rules split_p, $\text{split}_{\neg p}$	Branching on a variable (or value)
Backtracking in the DPLL tree	Backtracking search
Pure literal rule	Dominance / symmetry-type pruning
Empty clause $\bot \in S$	Failure (empty domain)
Theory T and SMT	Global constraints and constraint solvers over a theory (linear arithmetic, integers, ...)
Quantifier elimination (Fourier–Motzkin)	Projection of a constraint onto some variables
MAX-SAT	Over-constrained problems, soft constraints, optimisation
Uninterpreted symbols (e.g. Invar)	Unknown to be synthesised

10.2 From propositional logic to constraints

SAT is a CSP with Boolean domains. A clause set S is a CSP whose variables are the propositions with domain $\{0, 1\}$ and whose constraints are the clauses. Search by simplification and DPLL (Chapter 3) are then exactly a backtracking search with *propagation*: the (unit) rule is the propagation of a clause in which all literals but one are false, and $\bot \in S$ detects a domain wipe-out. Modern CP and SAT solvers add learning of clauses (CDCL) on top of this.

Modelling with Boolean variables. Exercises of TD2 and TD3 are typical CP modelling tasks:

- "exactly one" and "at most one" constraints (the liars, the pigeons: $\bigwedge_{i<i'}(\neg p_{i,j} \vee \neg p_{i',j})$) are the Boolean encoding of the global constraints `alldifferent` and `exactly`$(1, \dots)$;
- the distribution of toys to children (TD2, Exercise 6) is an *assignment* problem;
- the pigeonhole principle (TD3) is the classical hard example for resolution-based search, and the reason why dedicated `alldifferent` propagation (matching-based) is much stronger than clause propagation;
- graph k-colouring (TD3, Exercise 5) is the textbook CSP: variables v with domain $\{0, \dots, k-1\}$ and binary constraints $u \neq v$ on edges. The one-hot encoding v_i with C_v and $A_{u,v}$ is how the CSP is turned into SAT. Compactness (Chapter 3) extends the finite statement to infinite graphs.

Encoding finite-domain variables. A variable x with domain $\{0, \dots, k-1\}$ is encoded by propositions x_i ("$x = i$") with C_x = "exactly one x_i holds". A constraint $c(x, y)$ is encoded by clauses forbidding the incompatible pairs (i, j): $\neg x_i \vee \neg y_j$.

Tseitin transformation (TD2, Exercise 7). Solvers do not convert formulas to CNF by distribution (exponential, TD2 Exercise 5) but with fresh variables for subformulas, as in the equisatisfiable transformation of the exercise.

10.3 From first-order logic to constraint models

Constraints are first-order formulas over a fixed interpretation. A CSP over integers, e.g. $x + y = 10 \wedge x < y \wedge x \neq 3$, is the question: is there a valuation ρ such that $(\mathbb{Z}, 0, 1, +, <, =), \rho \models \varphi$? This is the *satisfiability modulo a theory* of Chapter 8, where the interpretation of the symbols is imposed. The free variables are the CP decision variables, and a model ρ is a solution.

Existential and universal quantifiers. In a CSP all variables are implicitly existentially quantified. Universal quantification appears in *quantified CSPs* and in properties to check (e.g. the inductive invariants of TD8, Exercise 2). Quantifier elimination (Chapter 7) is a form of *projection*: eliminating $\exists x$ from a conjunction of linear inequalities by Fourier–Motzkin is the CP operation of projecting the constraint store on the remaining variables.

Optimisation. TD8 Exercise 1 (coin change) models an optimisation problem by a satisfiability formula $\psi(m) = \varphi(m) \wedge \forall m'.(m' < m \Rightarrow \neg\varphi(m'))$. A CP solver instead states φ and a cost m to *minimise*, using branch-and-bound: whenever a solution of cost m is found, add the constraint $m' < m$ and continue. The two formulations are logically equivalent; CP uses the incremental one. MAX-SAT (TD3, Exercise 3) is the optimisation version of SAT.

Global constraints. Constraints such as `alldifferent`, `cumulative` or `table` are formulas that are expressible in first-order logic (e.g. `alldifferent`$(x_1, \dots, x_n)$ is $\bigwedge_{i<j} x_i \neq x_j$) but for which a dedicated propagator exploits their structure. The logical statement fixes their meaning; the algorithm is the propagator.

10.4 Proof-theoretic point of view

Unsatisfiability of a CSP is established by a refutation: propagation-and-branching trees correspond to sequent-calculus or resolution proofs (Chapters 4 and 9). The failed proof search in the sequent calculus (TD4) produces a counter-model, as a CP solver extracts a solution from a branch where nothing fails. The *cut* rule corresponds to branching on a lemma/variable ($P(f(a))$ in TD9, Exercise 2) and to learned constraints in conflict-driven solvers.

10.5 Soundness, completeness, termination

A CP solver is expected to be *sound* (every returned assignment is a solution), *complete* (if there is a solution it finds one) and to terminate. For SAT these are Theorem 3.13 (DPLL), and for the theory of linear rational arithmetic, decidability follows from quantifier elimination (Chapter 7). For full arithmetic with $\times$ over $\mathbb{N}$ (TD8, Exercise 1 uses it) satisfiability is undecidable in general (Gödel, Chapter 7), but the coin-change instance is a bounded problem since $x_i \leq S/v_i$, and thus decidable by finite domain search — the typical situation in CP, where every variable has a finite domain.

Appendix A

Basic mathematical concepts

A.1 Sets

For $A, B \subseteq E$, $e \in E$ and $n > 0$:

$e \in A$	membership	e belongs to A
$A \subseteq B$	inclusion	$\forall x.\, x \in A \Rightarrow x \in B$
$\emptyset$	empty set	$\forall x.\, x \notin \emptyset$
$A \cup B$	union	$\{x : x \in A \vee x \in B\}$
$A \cap B$	intersection	$\{x : x \in A \wedge x \in B\}$
$A \times B$	Cartesian product	$\{(a, b) \mid a \in A,\ b \in B\}$
A^n	iterated product	$\{(a_1, \ldots, a_n) \mid a_1, \ldots, a_n \in A\}$
A^0	null product	$\{()\} \neq \emptyset$
$A \setminus B$	difference	$\{x : x \in A \wedge x \notin B\}$
$\mathcal{P}(A)$	power set	$\{X : X \subseteq A\}$
$\#A$	cardinality (finite A)	number of elements

To show two sets equal: $A = B$ iff $A \subseteq B$ and $B \subseteq A$.

Property A.1. *For $A, B, C \subseteq E$: $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$; $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$; $E \setminus (A \cap B) = (E \setminus A) \cup (E \setminus B)$; $E \setminus (A \cup B) = (E \setminus A) \cap (E \setminus B)$.*

Example A.2. For $A = \{1, 2, 3\}$, $B = \{0, 1\}$: $A \cap B = \{1\}$, $A \cup B = \{0, 1, 2, 3\}$, $A \times B = \{(1, 0), (1, 1), (2, 0), (2, 1), (3, 0), (3, 1)\}$, $A \setminus B = \{2, 3\}$, $B \setminus A = \{0\}$.

Property A.3. *If $\#A = n$, $\#B = m$: $\#(A \times B) = nm$, $\#(A^k) = n^k$, $\#\mathcal{P}(A) = 2^n$.*

A.2 Functions

An n-ary function on E is a function $E^n \to E$; it may be *partial* (not defined everywhere). Example: $f(n) = n + 1$ on $\mathbb{N}$. A 0-ary function $E^0 \to E$ is identified with an element of E, a constant.

A.3 Relations

An n-ary relation on E is a subset of E^n (write $R(a_1, \ldots, a_n)$ iff $(a_1, \ldots, a_n) \in R$). Beware: $R = \{(1,1),(2,2),(3,3)\}$ is binary on $\mathbb{N}$ but unary on $\mathbb{N}^2$.

A binary relation R on E is *reflexive* if $R(x,x)$ for all x; *symmetric* if $R(x,y) \Rightarrow R(y,x)$; *antisymmetric* if $R(x,y) \wedge R(y,x) \Rightarrow x = y$; *transitive* if $R(x,y) \wedge R(y,z) \Rightarrow R(x,z)$. It is an *equivalence relation* if reflexive, symmetric and transitive; an *order relation* if reflexive, antisymmetric and transitive. E.g. $=$ is an equivalence and $\leq$ an order on $\mathbb{Z}$; on words, "same length" is an equivalence and "is a prefix of" an order.

Functional relations. A function f of arity n is represented by the $(n+1)$-ary relation $R_f = \{(\vec{x}, f(\vec{x}))\}$. A relation R of arity $n+1$ is *functional* if for every $\vec{x} \in E^n$ there is at most one y with $(\vec{x}, y) \in R$, and *total functional* if there is exactly one.

A.4 Graphs

A graph is a pair $G = (V, E)$ with V a set of nodes (vertices) and $E \subseteq V \times V$ a set of arcs. E.g. $G = (\{a,b,c,d\}, \{(a,b),(b,a),(b,c),(d,b),(d,d)\})$. A graph is thus a set with a binary relation. By default graphs are directed; undirected graphs are those where E is symmetric.

Appendix B

Induction

Mathematical objects must be represented finitely, even if potentially infinite. *Inductive definitions* do so using an inherent (partial) order on the objects.

B.1 The example of the natural numbers

$\mathbb{N}$ is the smallest set containing Z (zero) and such that $n \in \mathbb{N} \Rightarrow S(n) \in \mathbb{N}$. Among sets with this property P there is a smallest one $Y = \{Z, S(Z), S(S(Z)), \dots\}$: Y must contain at least these, and this set itself satisfies P. We then write $0 = Z$, $1 = S(Z)$, etc. This gives proofs by induction:

Theorem B.1. *Let $P(n)$ be a property of the integer n. If (i) $P(0)$ and (ii) for all n, $P(n) \Rightarrow P(n+1)$, then $P(n)$ holds for all n.*

Proof. The set X of integers satisfying P contains 0 and is closed under successor; as $\mathbb{N}$ is the smallest such set, $\mathbb{N} \subseteq X \subseteq \mathbb{N}$. □

B.2 Inductive definitions

Definition B.2. Let E be a set. An *inductive definition* of a subset of E is a pair $(B, \mathcal{R})$ where $B \subseteq E$ is a non-empty *base set* (axioms) and $\mathcal{R}$ is a set of partial functions on E (*induction rules*), each of arity $n \geq 1$, $E^n \to E$.

Theorem B.3 (Fixed point theorem)**.** *For an inductive definition $(B, \mathcal{R})$ on E there is a smallest $X \subseteq E$ containing B and closed under $\mathcal{R}$.*

Proof. Let $\mathcal{F}$ be the family of subsets of E containing B and closed under $\mathcal{R}$; it is non-empty ($E \in \mathcal{F}$). Let $X = \bigcap_{Y \in \mathcal{F}} Y$. Then $B \subseteq X$, and if $r \in \mathcal{R}$ and $x_1, \dots, x_n \in X$, then each x_i is in every $Y \in \mathcal{F}$, hence $r(x_1, \dots, x_n) \in Y$ for all Y, hence in X. So $X \in \mathcal{F}$, and it is the smallest. □

We write base cases and rules separated by |.

Example B.4. Even numbers: $n ::= 0 \mid n + 2$. Dyck language (well-parenthesised words): $u ::= \varepsilon \mid (u) \mid uu$; (())() is a Dyck word, but neither)(nor)(. Base-10 numerals: $u ::= a \mid bu$ with $a \in \{0, \dots, 9\}$ and $b \in \{1, \dots, 9\}$; arithmetic expressions $E ::= u \mid E + E \mid E - E \mid E \times E \mid E/E \mid (E)$.

B.3 Inductively defined functions

For an *unambiguous* inductive definition $(B, \mathcal{R})$ of X (each element of $X \setminus B$ arises in a unique way), a function $f : X \to Y$ is defined by giving $f(x) = \gamma_x$ for each axiom $x \in B$ and, for each rule r of arity k, a function g_r with $f(r(x_1, \ldots, x_k)) = g_r(x_1, \ldots, x_k, f(x_1), \ldots, f(x_k))$.

Example B.5. $0! = 1$, $(n+1)! = (n+1) \times n!$. The number of pairs of parentheses of a Dyck word: $\ell(\varepsilon) = 0$, $\ell((u)) = 1 + \ell(u)$, $\ell(uv) = \ell(u) + \ell(v)$. The value of an arithmetic expression: $v(u)$ is the integer written u; $v(E + F) = v(E) + v(F)$, etc., $v((E)) = v(E)$.

B.4 Proofs by induction

Theorem B.6 (Induction principle)**.** *Let X be defined inductively by $(B, \mathcal{R})$ and P a property on X. If (base) $P(x)$ for all $x \in B$, and (induction) for each rule r, if P holds for the arguments then P holds for the result, then $P(x)$ for all $x \in X$.*

Proof. Let $Z = \{z \in X \mid P(z)\}$; it contains B and is closed under the rules, so by the fixed point theorem $X \subseteq Z \subseteq X$. □

Example B.7. Every Dyck word has as many closing as opening parentheses: ε has none; if w has n of each then (w) has $n+1$ of each; if w_1, w_2 have n_1, n_2 of each then $w_1 w_2$ has $n_1 + n_2$ of each.

Appendix C

Greek alphabet

Lower-case Greek letters traditionally name formulas, since Latin letters are already used for propositions, functions, predicates, variables. Letters frequently used: α (alpha), φ (phi), ψ (psi), θ (theta), τ (tau), σ (sigma), ρ (rho), ε (epsilon), ℓ (lowercase L, for literals), Γ (Gamma), Δ (Delta), π (pi), λ (lambda). Others: β beta, γ gamma, δ delta, ζ zeta, η eta, ι iota, κ kappa, μ mu, ν nu, ξ xi, o omicron, υ upsilon.